自然语言程序设计

从信息协作到协议生态与自主工程

李昌开 著

第一版完整书稿
2026 年 11 月 3 日

本书保留作者关于自然语言程序设计的思想主线，借助 AI 完成结构整理与文字展开。各章均为可直接修改的正文。方法示例与经验证的工程结论在文中分别说明。

目录

自序：会说话，不等于会设计程序——从协作中理解程序 4

前言：写在前面 8

导论 自然语言程序设计的整体命题 10

第一编 信息与协作的世界观 13

第 1 章 程序设计的对象 13

第 2 章 信息如何形成价值 16

第 3 章 协作主体与关系 19

第 4 章 工具与协作的共同演化 22

第二编 协议生态与逐层具象化 25

第 5 章 先建立协作协议 25

第 6 章 从协议到软件的逐层具象化 28

第 7 章 独立创造与共同兼容 31

第 8 章 协议生态的治理与演化 34

第三编 自然语言的语义工程 37

第 9 章 自然语言的语义规格 37

第 10 章 一个场景需要表达什么 40

第 11 章 示例与授权的不完备性 43

第 12 章 冲突裁决与责任追溯 46

第四编 自主推演与工程实现 49

第 13 章 由启发式描述推导状态模型 49

第 14 章 面向现实环境的架构搜索 52

第 15 章 鸟枪法与能力碎片的重组 55

第 16 章 从语义到自主工程 59

第 17 章 以现实验证闭合工程 62

第五编 开放协作与持续演化 65

第 18 章 跨主体与跨设备的协作 65

第 19 章 经验如何成为可复用能力 68

第 20 章 持续交付与长期合作 71

第 21 章 劳动贡献与价值的计量 74

第六编 综合实践与理论边界 77
第 22 章 固件口袋中的协议与创造 77
第 23 章 从信息协作中长出企业工具 82
第 24 章 与既有方法的关系及理论边界 87
第 25 章 面向持续演化的程序设计 90
附录一 核心概念及其边界 93
附录二 一份可以实际使用的场景描述 96
附录三 能力碎片与组合记录 97
附录四 决策与证据的简洁记录 98
附录五 全书主张的实践检验 99
参考文献与思想来源 101

自序：会说话，不等于会设计程序——从协作中理解程序

本书的思考来自我为自己和他人开发系统的实践。面对一项具体任务，我们很容易从功能开始：需要怎样的界面，处理什么数据，怎样把操作自动化。但在这些问题之前，还有一些更根本的问题值得追问：这项功能为什么需要存在，信息为什么必须经过某个人或某个环节，谁有权作出决定，又以什么结果判断工作是否完成？一个软件的出现，往往与这些关系中的困难有关；软件投入使用以后，又会改变这些关系，使新的需要继续出现。我希望把这段常常被分散在需求讨论、制度安排和技术实现中的过程连接起来，研究人怎样通过自然语言表达意图、组织协作，并使这些表达逐渐获得实际的执行能力。AI 使越来越多具体实现可以被交给新的执行者，也让我们有机会更完整地审视这段过程。《自然语言程序设计》正是从这里展开。

要理解软件为什么产生，就需要先理解信息在协作中做了什么。信息被传递出去，并不意味着接收者已经理解；理解了，也不意味着他拥有采取行动的条件与权限。即使行动发生了，还要继续判断它是否服务于值得追求的目标。同一句“任务完成”，可以表示计算结束，也可以表示结果已经保存，或者下游已经确认接收，不同含义会使后续行动完全不同。因此，设计信息流，需要同时理解信息的来源、含义、时机，以及它进入判断和行动的方式。信息量的增加、传输速度的提高，都不能直接代替价值判断：一个工具可能减少某个岗位的等待，却增加另一个岗位的负担；某项安排有利于眼前效率，却使长期协作依赖越来越多的特殊知识。价值准则因而需要参与目标的形成和过程中的选择，说明哪些结果优先，哪些代价可以接受，哪些边界应当保持。信息在具体的关系中被理解和使用，价值也在这些关系的改变中形成。

人类很早就在用自然语言安排这样的关系。说明书交代在什么条件下怎样操作，组织规则划分职责与权限，行业标准建立共同的交互和检验要求，企业绩效规则则通过指标与评价影响工作的优先次序；法律中的权利义务和程序安排，也在组织人们可以采取什么行动、怎样处理争议。比如《罗伯特议事规则》是其中一个容易理解的例子，它使能够各自表达意见的人，按照共同的程序形成决定。从组织行动的角度看，这些文本中的相关安排，可以被理解为不同形态的自然语言程序。它们依靠参与者的理解、能力与适当的制度条件得到执行，复杂性来自角色、时间、状态、例外和价值取舍之间的关系。把这些关系从习惯、临时协调和个人记忆中提取出来，变成可以共同讨论和修订的对象，便进入了程序设计。会说自然语言，不等于会用自然语言设计程序；表达了一项愿望，也还需要说明它怎样成为可共同遵循的行动安排。这正是自然语言程序设计能够被学习和训练的前提。

当人们希望把其中适合重复处理、需要更稳定执行的部分交给计算设备时，这些安排就需要进一步落实为设备能够识别的对象、状态与操作。一个工作流程中的“确认”，必须具体到确认什么、由谁触发、何时生效、失败后如何处理；一句“保存以后可以恢复”，也必须落到某种存储条件与可观察的恢复行为。计算机语言承担着这种具体化之后的技术表达，软件则使相应行为能够在特定环境中运行。自然语言程序与计算机程序由此具有相连的设计内容，同时也承担不同的表达和执行要求。长期以来，这种联系主要通过不同专业角色的工作来维持：人们用自然语言讨论需要、规则和价值，再由工程人员建立模型、选择结构、编写代码，并把运行结果带回原来的讨论。两端一直存在联系，但跨越其间的层次需要大量理解、补充和交接，也容易使最初的理由在实现过程中变得不可见。

但 AI 时代的到来，人们参与软件创造的过程和效率，正在改变这种衔接的技术条件。自然语言表达的目标、场景和约束，可以由 AI 进一步整理、推演，形成候选状态模型、接口和实现；运行中出现的问题，也可以沿着这些关系返回，促使我们修改原先的理解。原来更多依靠人工在不同表达系统之间接续的工作，因而有机会成为更连续的创造过程。我把这里的变化理解为一个逐渐贯通的设计连续谱：一端是现实活动中的意图、价值和协作关系，另一端是具体技术条件下可以运行的软件，中间是不断明确承诺、补充决定和检验结果的过程。这种连续性允许自然语言、结构化协议、状态模型、代码和测试各自发挥作用，并使它们围绕共同语义相互解释。每向下一层，都会增加原先尚未确定的选择；每得到一项新的证据，也可能要求返回上层重新判断。今天写作本书的现实起点，正是这种连接能力的扩展。

沿着这条连续谱，本书先从信息与协作进入共同协议。只要多个参与者或独立创造物需要彼此依赖，它们就需要知道哪些行为可以被期待，什么状态意味着承诺已经兑现，发生分歧时又由谁作出决定。协议使这些共同预期稳定下来，同时把不影响共同关系的内部选择留给各自实现。随后，架构把承诺落实为责任与能力的分配，接口使相关行为能够被触发和观察，设计规范传递可复用的经验与建议，具体软件则把这些选择变成运行中的能力。自然语言可以贯穿其间，说明每项安排的理由、效力与适用条件。这里的顺序表达的是概念之间的联系：局部原型也可以帮助我们发现共同规则，实现中的困难也可以要求重新划分责任。协议的优先，尤其体现在共同依赖形成之后，参与者需要能够依赖明确承诺，而不是只能模仿某个现有实现的偶然做法。这样，独立创造才有共同存在的基础。

书中对具体案例的讨论，可以帮助读者看见这些关系怎样进入具体设计。当多个独立创造的程序需要在同一环境中协作时，问题不仅在于各自能否完成内部功能，还在于它们如何使用公共资源、交换信息、交接状态，以及如何在彼此不了解内部实现的情况下形成可靠的共同预期。设计因此需要首先厘清共同环境与各个参与者的职责，将必须共同遵守的行为与边界形成协议，同时把具体实现方式留给各自选择。例如，要求一个程序在退出

或交接控制权之前，确保必要数据达到约定的持久化条件，规定的是参与者可以相互依赖的结果，而不必要求所有程序采用相同的数据组织、资源管理和内部运行机制。在此基础上，抽象协议逐步落实为架构、接口和具体实现，再通过实际运行验证这些承诺能否成立；实践中发现的新问题，又可能反过来推动协议与协作关系的调整。由此，原本抽象的协作规则逐渐获得可执行的技术形式，而技术实现又成为检验和修正规则的现实依据。这个过程所揭示的，是共同约束如何使独立创造成为可能，以及不同创造物如何通过持续协作与相互适应，共同形成可以不断演化的系统。

有了共同边界，具体场景中的意图仍然需要被妥善表达。因此，本书进一步讨论自然语言的整体语义规格：目标是什么，现实环境提供了哪些条件，哪些性质必须保持，遇到冲突依据什么取舍，哪些决定已经交给执行者。人可以按自己的表达习惯说话，这些语义也可以分布在对话、已有协议和现场事实之中，关键在于它们能够形成足以指导当前行动的关系。描述不必穷尽所有分支，有效示例可以帮助执行者理解判断原则，明确的授权则可以为尚未选择的实现留下探索空间。本书把这种有范围、有依据的开放称为“授权的不完备性”。它允许 AI 在边界内继续推演和行动，也要求我们分清事实尚不清楚、价值尚未裁决与实现可以自主选择之间的区别。没有写出的内容，并不会仅凭缺失就成为自由处置的权限；同样，已经授予的一类选择，也不必重新拆成无数个逐项请示。设计的清楚，应当使自主工作更有依据。

这样的语义条件，使自主推演与工程实现有了可以共同检查的依据。AI 可以从有限场景中辨认状态和关系，发现遗漏与冲突，比较不同架构，并从多个任务中寻找能够复用和重新组合的能力。书中讨论状态模型、面向现实环境的架构搜索，以及从场景中抽取能力碎片的鸟枪法，都服务于这一步：使表达能够进入可探索、可比较的实现，而不要求人先把全部内部细节设计完毕。实现随后需要面对真实环境，接受接口、运行行为和必要的设备验证。一段解释看起来合理，或代码与生成的测试相互一致，都还需要相应的事实支持。检查也有两个层次：一层检查实现是否兑现当前承诺，另一层检查这些承诺是否仍然服务于现实需要。软件可能正确执行一道审批，而组织已经有条件取消这道审批。把事实沿着设计关系带回目标、协议和语义，才能使工程结果继续参与判断，而不是止于一次代码生成。

软件投入使用之后，这条路径便回到了协作本身。信息变得更容易获得，状态变得更清楚，某些行动的成本降低，原先的分工和流程也就有了调整的可能。书中的企业信息协作方法示例，从订单处理中的等待出发，追问等待究竟来自材料分散、含义冲突，还是决定权不清；这些不同原因决定了是否需要工具，以及需要什么工具。进一步看，当有关主体能够更早获得共同事实，原先负责反复转录和传话的环节就可以重新评估，决策时机和责任安排也可能随之改变。新能力又会使过去难以提出的要求成为现实，促使人们形成新

的协作规则，并产生下一轮工具化或软件化的需要。价值就在这类变化中被创造、分配和重新判断，不能只由处理速度或功能数量来衡量。持续演化因此包括扩展，也包括合并、简化和退出；每一轮工程有有限的目标与交付边界，整个协作过程则保留继续修正的空间。

全书由此把三个方向放在同一视野中：意图怎样逐层成为实现，不同主体和设备怎样通过共同关系协作，以及这些关系怎样在时间中随工具一起改变。我的判断来自实际开发，书中也用方法示例把这些判断之间的关系展开；整理经验的目的，是辨认一个做法为什么成立、依赖什么条件，以及哪些认识可以被后来者继承。这里所需要的抽象，始终要能够返回具体问题。谈论信息，要能找到它影响了什么决定；谈论价值，要能说明谁获得能力、谁承担代价；谈论自由，要能明确哪些选择可以自主作出；谈论演化，要能判断什么值得保留、什么应该改变。抽象使我们有可能跨越单个项目，但它的解释力仍然需要在新的实践中接受检验。

这也说明了本书为什么没有把主要篇幅用于讲解某一种 Agent 怎样搭建、某一种 Skill 怎样编写，或某门编程语言和某个开发平台的具体用法。这些技术知识在实际工程中仍然必要，具体框架、封装与操作步骤也会随着能力和环境变化而更新。本书选择研究支撑这些实现的关系，使读者能够判断当前工具适合承担什么，已有流程为什么存在，以及条件改变以后应当怎样重新组织工作。今天需要人亲自完成的分析与设计，未来也可能更多地交给 AI；人与 AI 的分工，应当依据新的能力、权限和证据条件继续调整。理解这些基础关系，可以使创造能力不依附于某一种工具，并在技术变化时找到新的实现路径。我所追求的长期价值，就在于让人能够从一个具体功能追问它的协作理由，也能够从一个协作目标找到可实现、可验证的路径，并在结果与预期不一致时修正原有安排。这样的思想建设容纳方法自身的变化，也使我们在执行能力不断扩展的过程中，继续理解自己正在创造什么，以及如何与这些创造物共同演化。

李昌开

前言：写在前面

【阅读提示】本篇与前述《自序》部分内容重合，另补充学术继承、原创边界、案例证据与理论修订等方面的说明。读者可按需选读。

软件从哪里来？通常的回答是，先有需求，再有设计，最后有人把设计写成代码。这个回答描述了许多项目的工作顺序，却没有解释需求从哪里来，也没有解释为什么某一种软件形态会在某种协作关系中出现。如果我们继续追问，就会遇到信息的产生与流动、主体之间的理解与分工、资源和决策权的分配，以及现实环境对工具的要求。

我所讨论的自然语言程序设计，从这些问题开始。人在表达一个需求时，既在描述希望得到的结果，也在建立一种关系：哪些事实需要被理解，哪些条件不能被违反，什么可以交给另一个主体判断，什么仍由自己决定。另一个主体可能是工程师，也可能是人工智能、智能体，或者一套已经存在的设备和软件。程序设计在这里表现为一种组织协作、形成规则并使规则获得执行能力的活动。

自然语言在这项活动中具有特别的位置。它可以同时表达目标、理由、价值取舍、例外、责任与尚未解决的问题。它允许人在认识还不充分时开始讨论，也允许人在新的证据出现后修改原先判断。它的开放性恰好适合真实世界持续变化的需求。但是，开放性并不自动带来工程质量。没有说明的内容究竟是允许自由实现，还是还没有做出决定，需要被认真区分。

本书提出的核心方法，是在保留自然语言表达自由的同时，建立整体场景的语义规格。人不需要像穷举程序分支那样描述所有情况，而应说明目标、环境、约束、价值判断、典型示例与授权边界。AI 由此提出更完整的状态与关系模型，寻找遗漏、矛盾和异常；能够在授权范围内解决的事项继续推进，超出边界的事项回到有权决定的主体。推演随后进入代码、构建、部署和真实验证，现实结果再反馈到原先的语义与规则。

这套方法还有一个高于单个应用的起点。面对独立创造的软件与设备，我们首先设计它们共同遵守的协议，规定它们如何存在、交换信息、使用公共资源并处理分歧。接口把共同规则变成可调用的能力，设计规范为实现提供经验和建议，具体软件则成为这套协作秩序中的实例。这样的协议生态使创造者可以保留自己的实现自由，同时与其他人的创造物建立稳定关系。

因此，本书会从信息协作一直写到具体软件，也会从软件回到它所改变的协作环境。软件投入使用后，会改变信息渠道、行动成本和分工方式，进而带来新的需求。新的需求可能促使我们创造工具，也可能使某些已有工具失去存在的必要。这个循环没有一个可以事先完全规定的终点，但每一次行动都需要有可理解的目标、边界和反馈。

我希望保留这个体系的哲学深度，同时让每一个抽象概念承担实际的解释工作。谈论价值，就要回答谁受益、谁承担代价以及如何观察变化；谈论自由，就要回答自由作用于哪些决策；谈论生态，就要回答独立参与者凭什么能够相互信赖；谈论演化，就要回答哪些变化值得保留、哪些变化应当撤回。不能回到这些问题的抽象，只会增加表达的负担。

本书也吸收软件工程、需求工程、控制论、契约式设计和制度研究中可以服务于这一中心的思想。相关思想各有历史，本书不会把已有方法改个名字就声称首次发明。这里希望建立的是一种有自身组织逻辑的综合方法：以信息协作为起点，以协议生态为共同基础，以语义规格和授权为控制机制，以 AI 为工程执行能力，以现实结果为修正依据。

这一方法不以任何一代模型或任何一个开发平台为前提。模型增强以后，某些人工澄清、推演或编码步骤可能变得没有必要，应该被删去。工具可以更换，工作流程可以缩短。需要持续保留的是判断能力：知道我们正在解决什么问题，知道哪些约束有理由存在，知道执行者有多大自主空间，知道什么证据足以支持继续行动。

这份书稿沿着上述主线展开。它既包含作者已经明确提出的观点，也包含为连接这些观点而形成的解释、推导与方法示例。案例中的假设方案会与已说明的工程原则区分，有限案例不被当作普遍有效性的证明。书中使用的概念可以继续修订；修订本身应当接受与软件同样的要求，即说明理由、保留来路，并面对新的事实。

导论 自然语言程序设计的整体命题

导论一 一个贯通不同尺度的设计对象

本书把自然语言程序设计定义为一种以自然语言组织工程语义的创造活动。它围绕现实场景建立目标与价值判断，定义协作主体之间的协议、能力和授权关系，使执行主体能够在明确边界内完成推演、实现与验证，并根据实际结果持续修正规则和工具。

这个定义中的“程序”具有两个相互连接的尺度。较窄的尺度是可以由计算设备执行的软件及其行为；较宽的尺度是使信息能够转化为行动的协作安排。两者不能混为一谈。组织制度不会因为写在自然语言中就自动变成计算机程序，但其中需要被工具化、自动化的部分，可以通过同一条语义链逐步落实为软件。自然语言程序设计研究的是这条链如何成立，以及如何保持各层之间的解释、授权与证据联系。

本书的主线依次经过信息与价值、协作关系、协议生态、架构和接口、场景语义、推演与实现、现实验证，再回到协作关系的变化。这个次序是一种理解上的组织，不要求所有项目在开始前完成一套庞大的顶层设计。一个简单任务可以只触及其中少数环节；一个面向多个独立创造者的生态，则可能需要显式展开大部分环节。

导论二 三个方向与两个闭环

首先是纵向的具象化。协作意图逐步落实为规则、可观察的承诺、可执行的接口和具体实现。每向下一层，都会引入新的决策。下层需要保留上层的有效约束，也要把上层尚未确定的事项转化为授权范围内的具体选择。所谓连续具象化，要求这些变化可以被解释，不要求不同层次拥有相同的精确程度。

其次是横向的协作。人与人、人与 AI、AI 与 AI，以及不同设备之间，需要交换信息、交接任务并处理不一致。这里的关键是关系能否被独立理解，而不只是某个中心能否把全部细节记住。协议、接口和责任记录，使关系能够跨越创造者、会话、工具与设备而延续。横向展开也包括比较多个场景，识别共同能力，再依据场景语义重新组合为工具；本书的鸟枪法将在这一关系中展开。

再次是时间上的演化。工具改变现实条件，现实条件改变需求，需求又推动工具与规则更新。这个循环允许系统成长，也允许系统收缩、合并和退出。演化的目标由价值判断约束，变化本身没有天然的正确性。

三个方向共同构成两个相互嵌套的闭环。工程闭环检验某个实现是否兑现当前语义承诺；协作闭环检验当前承诺与工具是否仍然服务于真实需要。一个软件可能在工程闭环中

完全合格，却在协作闭环中已经失去价值。例如，它准确执行了一道审批流程，但组织已经有条件取消这道审批。只观察软件是否正确，会错过更高层次的改进机会。

导论三 全书的六个基本主张

第一，工具需求产生于协作中的能力缺口。软件需求可以成为工作入口，但不应遮蔽信息流、分工、资源与价值的上层关系。提出一个软件功能，意味着提出一种改变协作方式的方案，这个方案本身也需要检验。

第二，独立创造需要共同协议。共同协议应当集中规定必要的关系与可观察的义务，把无关的内部实现留给参与者。协议越能够区分共同约束与局部自由，就越有条件支持多种独立实现。

第三，自然语言可以具有整体语义规格，而不必依赖逐句固定句法。一个描述可以自由组织，但仍需让工程上有决定意义的信息可辨识，使缺失、矛盾、授权和证据要求能够被检查。

第四，不完备性可以被明确授权。人可以通过目标、价值准则和有限示例划出允许探索的空间，由 AI 补充状态与实现。授权需要范围，推导需要依据；缺少规定本身不构成授权。

第五，生成与证明需要不同的证据责任。AI 能提出代码、状态模型和测试计划，这些产物仍需接受执行结果、外部观测或适当的独立检查。对方案的解释不能代替方案在现实中的表现。

第六，持续演化应当保留退出与修正规则。持续交付不意味着无限承诺，价值增长不意味着工具数量增长。一个成熟体系应能承认假设失效，淘汰无用复杂性，并为下一轮创造保留空间。

这六个主张是本书的方法论立场。它们不是已经由统一数学体系证明的定律，也不保证对每一种任务都有成本优势。后续章节会说明它们成立所依赖的条件、可操作的方法，以及应当接受怎样的反例。

导论四 本书如何处理原创与借鉴

本书的辨识度在于组织关系：把协议生态、场景语义、启发式授权、AI 自主工程和协作演化放入同一条设计链。信息论、模块化、契约式设计、状态模型、测试与敏捷思想为这条链提供可用的工具，但它们不会自动证明整套方法已经成立。

正文采用编号引文，文末列出少量直接相关的原始文献和官方资料。引用用于说明所借鉴的方法及其边界，不用于为作者尚待验证的主张制造权威。书中“本书提出”“可以据此

建立”等表述，指向本书的组织方案或解释性推演；“方法示例”“假设场景”则表示用于展示推理的构造，不能视作已经取得的项目结果。

全书六编依次回答为什么创造、共同规则如何建立、场景如何表达、AI 如何实施、成果如何持续演化，以及整套方法如何在案例中贯通。读到最后，我们应当能够从任何一个具体功能向上追问它的协作理由，也能够从一个抽象的协作目标向下找到可执行、可验证的实现路径。

第一编 信息与协作的世界观

这一编从软件参与的现实活动出发，解释信息怎样经过理解、决策和行动形成价值，以及工具为什么会与协作关系共同演化。它为后续协议和工程方法确定设计对象。

第 1 章 程序设计的对象

1.1 从程序向它所参与的活动追问

如果把程序设计的对象限定为一段可执行代码，那么自然语言编程很容易被理解为输入方式的变化：过去敲代码，现在描述意图，由模型代为编码。这种变化确实存在，却不足以说明本书讨论的事情。代码进入现实之后，总会参与某种活动。活动中的主体交换信息、判断情况、分配资源、采取行动，程序只是其中一个承担特定功能的参与者。

因此，设计一个程序之前，可以继续追问：为什么需要这项功能？为什么这段信息必须经过某个人？为什么一次决定要在几个系统之间重复登记？这些问题把设计对象从程序内部推向程序所处的协作关系。某个界面操作繁琐，可能需要改善界面，也可能意味着本来就不应由这个人负责录入。若只优化操作，程序可能更加流畅地维持了一个没有必要的环节。

本书所说的自然语言程序设计，从这类追问开始。它允许人用自然语言表达协作目的、规则、价值取舍和技术约束，再让人工智能参与推导这些表达所需要的具体实现。设计既包括让现有程序做得更好，也包括判断某个程序是否有必要存在。

1.2 单个产品之外还有关系的设计

设想几位创造者分别为同一种设备编写应用。每个人都能独立完成自己的功能，困难却出现在它们相遇时：公共资源怎样使用，数据属于谁，应用退出意味着什么，设备状态由谁保持，某个应用失败后其他应用还能否运行。这些问题不属于任何一个应用的内部功能，却决定所有应用能否共同存在。

在这个层面上，设计对象是一组创造物之间的关系。它们不需要拥有相同的内部结构，也不需要由同一个人统一管理，但必须对共同边界形成一致理解。我们可以先表达这种关系，再允许不同创造者分别产生实现。具体软件于是成为共同规则之下的实例，而共同规则不能被某个实例的偶然实现完全代替。

软件工程已有协议、接口、模块化和开放系统等丰富传统。本书不把关系的设计宣称为新近才出现的思想。它要进一步处理的是：当自然语言能够贯通多个抽象层次，AI 又能

够承担大量实现工作时，人能否把创造的主要力量放在关系与规则上，使软件在这些关系中被持续地提出、生成和修订。

1.3 自然语言作为跨层表达

自然语言具有一个特殊条件：人可以用同一种媒介讨论为什么做、应当怎样协作、哪些原则不能违反，以及按钮在某个场景下应该产生什么行为。我们不需要在每次改变抽象层次时都换成另一门专门语言。抽象目标与具体情形可以被放进同一段对话，并相互解释。

这种连续性并不自动带来严谨。一个词在战略讨论中表示方向，在接口说明中可能必须表示可以观察的事件。如果始终不区分这些用法，所谓统一表达就会变成统一含糊。因此，本书不要求每句话具有固定句法，却要求描述在它所承担的层次上具有足够的语义：指向什么对象，由谁决定，什么证据可以说明它成立，哪些问题仍然开放。

自然语言在这里承担贯通作用，结构化协议、代码、测试和形式模型仍然可以承担各自擅长的职责。统一的是设计意图及其可追溯关系，不是要求所有运行数据都改写成句子，也不是要求处理器直接执行人类语言。

1.4 软件是被选择的实现

现实中的需要不会仅凭被说出来就自动变成正确软件。需求可能自相矛盾，环境可能没有所需权限，模型可能错误理解，成本也可能超过收益。所谓软件从需求中“生长”，描述的是生成机制变得连续：当能力缺口被识别，目标和边界得到表达，AI 可以在允许的范围内探索实现，再由现实验证筛选可用结果。

这意味着同一协作目的可能对应多种工具。处理重复沟通，可以建立共享状态，也可以减少审批节点；需要共享状态时，可以采用既有系统的能力，也可以创建一个很小的服务。选择软件并非预设答案，而是对环境、能力和代价的判断。

人工物可以被理解为内部组织与外部环境相接的对象。西蒙关于人工科学的讨论为这种视角提供了重要参照：设计需要同时理解所造之物及其发挥作用的环境。[1] 本书进一步把环境理解为可以被工具反过来改变的协作系统。一个实现满足当前目标之后，还会创造新的行为可能性，因此它的作用不能只用功能清单描述。

1.5 本书的基本立场

本书把自然语言程序设计定义为一种工程方法：以自然语言组织意图、协议、场景、价值准则和授权边界，使 AI 能够在这些语义条件下自主推演与实现，并由可观察的现实结果检验和修订。它不以某一代模型为前提，也不以人的逐行编码为必要形式。

这一定义包含两个相互制约的方向。人需要把真正重要的语义说清楚，否则自由会退化为模型根据惯例擅自补齐。人也需要把不必预先确定的细节留出来，否则自然语言又会变成冗长的低层代码，只是把实现设计搬到了散文里。明确边界与保留自由必须同时发生。

判断一段自然语言是否具有程序设计意义，因而不能只看它是否提到软件，也不能只看它能否使模型输出代码。应当考察它是否改变了可接受实现的范围，是否规定了行为应被怎样判断，以及未规定部分由谁决定。一句愿望可以开启设计，却通常还没有完成设计；一份长篇说明也可能始终没有触及决定性的条件。表达的工程价值取决于这些关系是否成立。

它还包含一种世界观上的判断：程序并不孤立存在，创造者也不孤立存在。人、设备、软件和智能体通过信息与行动共同构成现实活动。要理解程序为什么出现，需要先理解信息在这些关系中做了什么；要理解程序应当怎样演化，需要继续观察这些关系因为程序的出现发生了什么变化。

第 2 章 信息如何形成价值

2.1 传得过去只是第一步

把信息放在设计起点，首先要避免把“信息”当成能够解释一切的词。香农信息论研究通信中消息的选择、不确定性以及传输能力，其经典框架并不直接判断消息的意义和社会用途。[2] 一段完全无用的随机数据可以具有很高的信息熵，一句简短而重复的报警也可能对行动至关重要。

本书关心的信息，需要在通信之外继续考察三个关系：它对于接收者意味着什么，它能够影响什么行动，这个行动在某种价值判断下有什么结果。传输、理解、行动和价值相互联系，却不能用其中一个替代其余部分。连接上接口只能说明消息有机会到达，不能说明接收者知道如何使用，更不能说明使用后会产生预期收益。

设想设备发送“任务完成”。这句话可能表示计算结束、结果已经写入内存、结果已经持久化，也可能表示下游已经确认接收。不同理解会产生不同后续动作。设计的难点往往出现在这些理解之间，而不仅是传输速度。因此，信息协作需要表达消息的含义及其证据条件。

2.2 信息存在于具体的情境

一条消息的作用与时间、来源、对象和接收者状态有关。昨天的库存记录即使没有被篡改，也未必适合决定今天的发货。某个智能体给出的确定回答，若实际依据过时文档，就可能把推测包装成事实。数据完整性、事实可靠性和决策适用性是不同问题。

因此，在设计信息流时，应关注一项事实从哪里获得、何时获得、如何被转换、在哪些条件下仍然有效。并非所有信息都需要复杂追踪；追踪深度应与错误后果相称。但一旦行动依赖该信息，系统就需要有办法识别其适用范围。否则每个环节都可能单独正确，组合起来却产生错误行为。

上下文也不能无限堆积。给 AI 更多材料可能帮助理解，也可能引入冲突和过期约束。重要的是形成与当前决策有关的上下文，并保留回到来源的路径。信息的组织应服务判断，不能把材料数量当作理解质量的替代指标。

2.3 从信息到行动之间有权力

信息不会自行变成行动。某个主体必须解释它，拥有采取行动的权限，并且能够调用所需资源。同样一条设备故障信息，操作者可能有权停机，维护人员可能有权检查，排产系统可能只能提出重新安排的建议。理解内容相同，不意味着行动权限相同。

这说明信息流同时也是一种决策结构。谁看见什么，谁能够质疑，谁有权更正，谁能够发出有效指令，会影响协作结果。把消息从人转交给智能体，并不只是提高传递效率，也可能改变实际决策权。若智能体可以修改业务状态，而原来只能建议修改的人没有同等权限，系统已经发生了组织上的变化。

自然语言程序设计必须把这种变化显式化。描述“自动处理异常”时，至少需要知道什么属于异常，处理可以改变什么，以及什么后果超出授权。没有这些语义，自动化只是把原来可见的人类判断藏到了实现里。后续的协议与授权机制，正是对这一问题的工程回应。

2.4 价值是有立场的判断

信息产生价值，不等于所有参与者获得相同收益。一个减少等待的系统可能让某个岗位效率提高，同时让另一个岗位承担更多集中处理。一个便于管理者追踪的工具可能降低员工的操作自由。一项技术改善是否值得采用，需要讨论它服务谁、转移了什么成本、允许什么程度的影响。

本书因此把价值准则放进程序设计，而不只在项目结束时讨论投资回报。价值准则规定哪些结果优先，哪些代价可以接受，哪些条件即使有利可图也不能越过。它既可以是明确指标，也可以是需要裁决的原则。例如保护已经确认保存的数据，可以比减少几秒等待拥有更高优先级。

价值还受时间尺度影响。一次性绕过公共接口可能迅速完成任务，却让后续维护不断依赖特殊知识；相反，适度建设共同接口可能暂时增加工作，但降低以后独立创造的门槛。两种选择都不能脱离情境判断。本书要求把时间尺度作为判断条件，避免用眼前的局部效率遮盖长期承担的义务。

2.5 不确定性不应被全部消灭

工程活动经常把不确定性当作需要减少的东西，但不同不确定性应有不同处理。对于已经确认保存的数据是否真实存在，需要充分证据；对于尚未决定采用哪种内部结构，可以保留选择。过早消除后一种不确定性，可能使设计失去适应现实的空间。

因此，自然语言程序设计不以“消灭模糊”为单一目标。它要区分事实未知、含义冲突、实现可选和授权未定。事实未知可以调查，含义冲突需要澄清，实现可选可以探索，授权未定则需要确定谁有权选择。如果把它们都当成信息不足，AI 可能不断询问无关细节；如果把它们都交给推理，AI 又可能跨越应由人决定的边界。

信息的价值也不能被简单定义为不确定性减少。一个没有必要的统计报表可以减少某种未知，却消耗更多维护资源；一个设计保留多条实现路线，反而可能更有价值。只有把信息放到目标、行动和环境中，才能判断哪种确定性值得获得，哪种开放性应该保留。

2.6 信息协作的设计起点

分析一个场景时，可以沿着一次真实行动逆向追问。行动依赖什么判断，判断依赖哪些信息，这些信息由谁产生，经过哪些转换，哪些延迟或失真会改变结果。再顺着信息向前追问：谁能够使用它，谁实际承担后果，结果怎样返回产生信息的环节。

这类分析并不要求先画出完美流程图。一个具体的失败往往已经足够打开问题：为什么界面显示成功，数据却没有保存？为什么两名智能体都认为任务已交给对方？为什么记录齐全，业务人员仍然不敢采取行动？这些现象提示信息与行动之间存在断裂。

工具的需求由此出现。有时需要一个共享状态，有时需要明确消息含义，有时需要补充确认机制，也有时只需要取消无效交接。要判断这些工具究竟由谁使用、与谁发生联系，下一步便要把协作主体及其关系纳入设计。

第 3 章 协作主体与关系

3.1 主体不限于人

本书使用“协作主体”这个词，指能够在某个协作过程中接收信息、依据规则采取行动，并对其他参与者产生可观察影响的角色。这个角色可以由人承担，也可以由设备、软件、智能体或它们的组合承担。它是工程上的分析单位，不自动意味着法律人格、道德地位或者独立责任资格。

同一个主体内部可以包含多个实现。一个服务窗口在业务人员看来是单一入口，内部却可能由信息提取程序、计划智能体和人工复核共同工作。反过来，同一个人也可能在不同场景中承担申请者、审批者和维护者等不同角色。以角色而非物理实体为起点，可以避免把权限直接绑定到某个名字或某台机器的全部能力。

主体的划分应服务协作理解。如果一个内部步骤对外没有独立承诺，也没有独立授权，未必需要把它建模为一个智能体。把每个函数包装成角色，并不会自然形成更好的协作系统，反而可能增加消息、状态和责任追踪的复杂度。

同样，分析中存在多个主体，不意味着产品必须建立复杂的用户账号体系。一台归属明确的个人设备，可能不需要为每项操作添加用户标识；它仍然可以区分应用数据与公共数据，区分使用行为与维护行为。架构应表达场景真实需要的关系，而不是把抽象概念逐个变成数据库表。把分析模型直接投影为功能，往往会让思想工具反过来支配产品。

3.2 创造物之间的契约

两个创造者不必直接见面，他们创造的系统仍然可以通过共同规则协作。浏览器与服务、独立应用与公共运行环境、不同智能体与同一工具，都体现这种关系。这里的重要之处是：协作依赖可以被第三方理解的规则，而不是依赖两位作者私下共享的习惯。

在同一设备上，某个应用可以不理解另一个应用的内部逻辑，但必须知道公共存储区域怎样识别，什么行为会影响共同运行条件。创造者把一部分承诺写入协议，后来的实现者便可以根据这些承诺形成预期。共同规则把尚未发生的合作变成可能。

这种契约主要约束可观察的交互，而不是要求内部完全透明。一个应用可以自由选择数据结构，只要它在共同边界上的行为满足约定。这给不同实现留下竞争与探索空间，也让协作不必依赖统一的技术偏好。

3.3 理解能力与决策资格

AI 能够理解某项业务，甚至在某些判断上表现出较强能力，并不自动获得决定该业务的资格。能力回答能不能做，授权回答允不允许做，责任安排回答出了问题由谁处理。这三个问题相关，但不能互相替代。

例如，一个智能体可以估计删除旧记录能够释放多少存储，也可以提出迁移方案。它是否能够真正删除，取决于记录的权属、保留要求和已经获得的权限。若用户授权其处理临时缓存，这个授权也不能被扩展为处理所有历史资料。自然语言中的范围词、对象词和后果词，会在这里产生实际工程效力。

同样，人类也不因为位于对话另一端就天然拥有系统的全部权限。一个普通使用者提出的要求，可能与设备所有者、组织管理者或其他用户的权利冲突。系统不能只根据最近一条话语决定优先级，而应保留有效授权的来源和范围。所谓人类决策，必须进一步落实为适当的人拥有适当决定权。

3.4 协作依赖共同预期

协作并不要求所有主体拥有完整相同的内部知识。它需要的是：在相关边界上，各方能对彼此的行为形成足够可靠的预期。发送请求后，何时可以认为已被接收；没有返回结果时，是否允许重试；重试会不会造成重复作用；某个主体退出后，未完成任务由谁接管。这些预期决定协作能否稳定继续。

若没有共同预期，各个主体可能都采取局部合理行动，却产生整体失败。提交者因为超时重新提交，执行者把每次提交都当作新任务，最终发生重复扣减。任何一方单独看起来都在努力完成目标，但双方对请求身份和完成条件缺乏共同理解。

自然语言描述可以直接把这种关系说出来：同一任务的重复请求不得重复产生业务效果；执行结果不明时，先确认已发生的状态，再决定是否重做。这段话不是完整算法，却已经确定了需要保留的行为性质。AI 可以据此选择标识、确认和恢复机制。

3.5 分工的尺度与协调成本

AI 使增加执行者变得容易，却不能消除协调成本。多个主体需要交换信息、管理相互依赖、解决冲突，还可能争用相同资源。把任务拆得过细，会让每个主体只能看到局部材料，增加上下文转述和判断损失。把任务全部集中，又可能形成能力和权限的单点。

合理分工应依据需要独立变化的知识、需要隔离的权限，以及可以独立检验的结果。信息隐藏的模块设计思想强调围绕可能变化的设计决定组织模块，这为分工提供了有用参

照。[3] 本书将它延伸到人机协作：一个角色应当具有足够上下文完成自己的承诺，同时不必获得与承诺无关的全部信息。

因此，协作架构的质量不由智能体数量决定。一个人加一个智能体可能比十个相互转述的智能体更可靠；多个相互独立检查的主体也可能在关键处提供更好的证据。关键是任务的依赖结构是否与分工一致，不能把并行本身当成效率。

3.6 跨身份与跨设备的连续性

当行动跨越身份和设备，任务不能仅依赖某段临时对话维持。发起者的意图需要被转化为可以携带的任务上下文，其中包括目标、已获授权、完成状态、重要假设和证据位置。执行者改变时，这些信息仍应允许后来者判断什么已经发生、什么尚未被允许。

但连续性也有边界。授权可以限于一个环境、一次操作或者一个期限，不能因为任务转移就无限继承。新的执行者需要知道原授权能否被转交，旧执行者需要知道自己何时停止行动。否则协作可能在表面上无缝，实际上发生重复执行或权限扩散。

这种跨边界的连续性，把软件设计推向协议设计。我们需要让不同主体在不共享全部实现的情况下仍能保持协作。工具一旦承担这些关系，又会改变主体能够做什么、需要彼此怎样配合。接下来便要考察这种反向作用。

第 4 章 工具与协作的共同演化

4.1 工具改变需求的来源

一种常见描述把需求看成软件之前已经存在的东西：先有问题，再有解决问题的程序。这个顺序在一次局部任务中成立，却不足以描述长期演化。新工具会改变哪些事情变得容易，哪些信息能够被获得，哪些主体可以直接联系，于是原先无法形成的需求开始出现。

例如，一个组织最初需要把设备异常汇总给维护人员。自动汇总降低信息收集成本之后，人们可能进一步要求预测维护时机；预测能力出现后，排产方式又可能调整；排产改变后，关于资源协调和责任分工的新问题继续产生。需求并非从一份静态清单中逐个取出，而是在能力变化中被重新发现。

因此，工具既是协作关系的结果，也是改变关系的条件。本书把这种相互作用称为共同演化。这是对工程过程的解释框架，不是保证任何技术变化都会带来进步的自然定律。

4.2 从局部自动化到结构变化

一个自动化工具最初可能只是接替重复工作，但当这种接替足够稳定，协作结构就会调整。过去某个岗位承担信息中转，后来共享状态让上下游能够直接联系；过去需要定期会议汇总，后来异常触发机制让人只处理偏差。改变的不只是操作时间，也包括谁需要等待谁、谁看见什么以及谁承担判断。

生产力与生产关系的讨论在这里具有具体含义：技术能力改变行动的可行范围，组织关系决定这些能力如何被使用、收益如何分配以及风险由谁承担。两者不总是同步变化。系统有了自动处理能力，权限仍然完全沿用旧流程，可能形成重复审批；权限迅速下放，恢复与追责能力却没有同步形成，也可能带来新的脆弱性。

自然语言程序设计需要识别这种不同步。实现一个新功能时，应问它是否已经改变既有承诺。如果改变了，单独修改代码不足以完成工程，还需要调整相关协议、岗位理解和验证方式。工具升级因此可能触发协作升级，但触发不意味着自动完成。

4.3 生长也包括收缩

持续演化容易被误解为软件越来越多、功能越来越大。实际上，有效的协议可能消除大量临时工具。若多个应用对公共数据采用共同语义，原来负责反复转换格式的程序就可能失去必要性。若智能体能够直接完成某类低频任务，为此维护一个复杂界面也未必合理。

因此，本书把退役、合并和简化视为与新增同等重要的创造活动。每一个工具都有维护、理解、依赖和迁移成本。一个暂时解决问题的小脚本，如果后来成为关键环节而没有

得到相应治理，就可能把隐性风险传递给整个系统。保留它需要理由，淘汰它同样需要安排。

退役也不能仅凭技术上已有替代就立即发生。旧工具可能仍承载未被记录的业务规则，少数使用者可能缺乏迁移条件。应该观察实际依赖，确认数据和职责能够转移，再停止承诺。生态的简化应以协作仍然成立为条件，而不是以数量减少作为目标。

4.4 反馈不等于服从每一次要求

真实反馈是演化依据，但反馈本身也需要解释。某个使用者要求增加按钮，可能是在表达流程绕行、状态不透明或权限不足；照单增加按钮，可能只把深层问题固化成界面复杂性。另一方面，设计者也不能用抽象理论否定具体困难，声称用户尚未理解正确方案。

反馈需要被放回原场景检验：在什么条件下出现问题，影响谁，现有规则如何形成这个结果，建议是否解决根因，是否把成本移给别人。对这一解释的争议，可以通过小范围改变和可观察结果进一步澄清。这样，反馈成为共同探究的材料，而不是任一方单方面命令的集合。

持续交付也应具有这种节奏。某次交付有明确范围、证据和停止条件，整个协作关系则允许开启下一轮。没有阶段边界的“永远继续”，会使责任无法评估；把阶段边界误认为需求永远冻结，又会切断现实反馈。两种尺度需要并存。

反馈还需要与资源约束结合。每一条改进都要花费理解、实施和验证成本，频繁改变会打断已经形成的操作习惯。因此，一轮改进可以在目标已经满足、剩余问题影响很小或者当前资源不足时合理结束。开放演化表示未来仍然能够重新进入，不表示今天必须永远继续。保持选择权，与不停地行使选择权，是两件不同的事情。

4.5 演化需要保持可理解性

随着系统不断增长，设计者面临的困难不只是新任务，更是理解过去为何如此。若每轮都只留下代码，后来者无法区分稳定原则、历史妥协和偶然习惯，系统会逐渐依赖少数人的记忆。AI 虽能读取大量材料，也不能从没有记录的事实中恢复真实理由。

所以，演化过程需要保留能够影响未来判断的知识：某项协议为何设立，某种替代为何被否决，一个限制来自环境还是暂时能力，以及什么时候可以重新讨论。记录不必覆盖全部思考，应围绕会改变选择的证据。重复无用的日志可能淹没真正关键的决定。

控制理论提醒我们，系统对环境变化的应对能力需要与相关扰动的种类相适应；阿什比对必要多样性的讨论提供了这一视角。[4] 本书从中借鉴的工程含义，是通过恰当抽象、限制危险扰动和增加可用反应来保持适应能力，而不是要求模型无限枚举现实的一切可能。

4.6 从演化关系进入协议设计

前四章形成了一条设计路径：程序参与现实协作，协作以信息和行动相连，参与者具有不同能力与权限，工具又持续改变这些关系。在这条路径中，协议的必要性已经出现。没有共同规则，每一次工具生长都可能迫使所有参与者重新理解彼此；规则过度固定，又会阻止新的能力进入。

协议因此承担一种特殊职责：让一部分共同预期足够稳定，使独立创造成为可能；同时明确哪些部分能够调整、由谁调整以及如何保持既有协作。它既是约束，也是降低相互依赖成本的条件。

本书接下来从协议出发逐层进入软件。这个顺序并不取消真实需求，也不要求先完成一部包罗万象的生态法典。它要求在某个具体协作范围内，先识别必须共同理解的关系，再让实现围绕这些关系展开。能够自由生长的软件，需要一个使自由相互兼容的起点。

第二编 协议生态与逐层具象化

协作需要可依赖的共同边界。围绕独立创造，本编讨论如何先建立协议，再逐步形成架构、接口、设计规范和软件实例，以及这些规则怎样在变化中继续成立。

第 5 章 先建立协作协议

5.1 协议先于什么

协议优先的含义，需要先限定清楚。它不意味着在了解场景之前编写规则，也不意味着一个项目必须先做完全部协议才能开展实验。它意味着：当多个独立创造物需要相互依赖时，应当先明确它们在共同边界上可以相互期待什么，再让各自实现围绕这些承诺展开。

这个优先关系主要是逻辑上的。实际工作中，可以通过原型发现规则，也可以用既有实现反推共同需求。但一旦某种行为被公开依赖，就需要把它从某段代码的偶然习惯提升为明确承诺。否则后来的创造者只能模仿现有实现，并不能判断哪些行为可以依赖、哪些只是暂时如此。

本书把底层协议看作生态中的共同规则。这里的“底层”表示逻辑地位，不一定对应操作系统底层或网络分层中的某个位置。一项关于退出前数据责任的约定，技术上可能在应用层实现，却能成为多个应用共同遵守的基础。

从这个意义上，作者把自己的自然语言编程也称为协议式编程。它先设计独立创造物之间得以协作的契约，再让接口、规范与软件在这些契约下逐步形成。这个名称强调一种设计方向：具体产品是生态关系中的实例，程序设计也能够向产品之上抽象，直接设计使不同创造得以共同存在的秩序。

5.2 从共同冲突中抽取最小规则

制定协议可以从独立创造最容易发生冲突的地方开始：共享资源的使用、任务身份的识别、状态的确认、失败的处置以及主体退出后的责任。并非每个内部决定都值得上升为生态规则。只有当一项决定会影响其他参与者的合理预期时，才需要认真考虑共同约束。

设想一个设备上的多个应用需要返回共同入口。协议可以约定返回动作的可识别方式、返回前必须完成的持久化责任，以及入口接收后承担什么职责。它未必需要规定每个应用如何组织线程、如何清理内存，更不必要求所有内部现场都以相同方式释放。若退出后运行环境会被整体重置，要求应用逐一清理一切资源，可能只增加实现负担。

这类判断体现协议的经济性：共享约束应足以保护共同协作，又尽量不占用参与者的内部自由。最小并不意味着文字最少，而是每项强制规则都能说明它保护了哪种共同关系。只有一句“必须正常退出”的协议很短，却可能把所有分歧隐藏起来。

5.3 契约约束行为而非创造者的思想

协议允许实现者持有不同的技术判断。一个实现采用数据库，另一个使用文件；一个选择异步处理，另一个在受控范围内同步完成。只要对外承诺相同，这些差异可以共存。生态不需要强迫所有创造者接受同一套内部美学。

但不同实现也不能以内部自由为理由任意解释共同词语。若协议中的“已保存”表示数据已达到约定的持久化条件，某个实现就不能把“已经发出保存请求”当作等价状态。自由发生在承诺允许的范围内，不发生在对承诺的任意重命名中。

将协议比作法律，有助于理解共同预期、权利边界与规则修订，但这个类比有明确限度。软件规范本身不会自动取得国家法律效力，设备也不会因为承担接口义务就成为法律责任主体。实际法律义务来自相应制度和合同。本书借用的是制度设计的分析方式：先规定关系的基本秩序，再允许主体在秩序中独立行动。

5.4 强制要求与建议的效力

一个生态既需要必须遵守的约束，也需要便于实现的建议。如果二者混在一起，创造者可能把参考实现当成唯一合法路径，也可能把重要义务当成可忽略的风格偏好。协议需要在整体语义上区分这些效力。

互联网规范中的 MUST、SHOULD、MAY 等术语提供了成熟参照：绝对要求、通常应当采用但允许有理由偏离的建议，以及可选择事项，具有不同含义；这些关键词的规范效力也需要结合其声明条件理解。[5] [6] 本书不要求中文说明机械套用这些英文词，而要求读者和 AI 能够识别同样的效力差异。

例如，“必须保护已经确认保存的数据”规定共同底线；“建议将返回入口放在固定位置”有助于形成交互习惯；“可自行选择内部缓存策略”明确保留实现空间。若某个特殊场景需要偏离建议，应说明偏离如何仍然满足上层目标。若需要改变强制义务，则应进入规则变更，而不能借“更好的实现”绕过。

5.5 协议首先需要回答什么

在一个协作范围内，协议至少要让参与者理解共同对象、可见行为、状态含义、权利义务以及失败边界。共同对象回答双方在谈论什么；可见行为回答可以提出什么请求；状

态含义回答何时可以认为事情已经发生；权利义务回答谁可以改变什么、谁需要保证什么；失败边界回答预期不成立时如何恢复协作。

这些内容可以用自然语言组织，不要求每次填满固定文档。对于一个简单工具，几段清楚的说明就可能足够；对于开放生态，需要更明确的术语、示例和兼容约束。重要的是语义之间不存在隐蔽跳跃。若协议允许调用方超时重试，就要继续考虑重复执行的后果；若允许应用独立维护数据，就要说明其他主体不能擅自将其当作公共数据处理。

协议还应说明不承诺什么。一个入口只负责识别和启动应用，并不必然承担应用之间的冲突仲裁；它可以暴露识别信息，把冲突处理留给应用作者或适当的人。责任的边界需要正面表达，不能靠某个功能尚未实现来暗示。否则使用者可能把缺失能力当成缺陷，而设计者把它当成刻意保留。

5.6 共同规则的授权来源

协议有技术内容，也有治理来源。谁可以建立规则，谁同意遵守，谁可以决定例外，谁有权修订，都是生态的一部分。一个人可以为自己控制的实验环境快速制定规则，却不能因为写得清楚，就让其他独立主体自动承担新增义务。

AI 可以提出协议草案，分析矛盾，生成兼容测试，也可以在明确授权下维护特定范围的规则。它不能把自身推导当成共同体已经接受的决定。就连人类提出的修改，也需要区分个人建议与已经生效的约定。规则的内容与规则的有效状态必须同时存在。

一份好的自然语言协议，因而不仅便于生成实现，还便于判断实现凭什么承担这些行为。协议确立后，架构和接口开始具体化：共同责任需要落到组件关系，抽象承诺需要变成可以调用和观察的形式。下一章讨论这种逐层具象化怎样保留原有语义，同时继续给实现留下空间。

第 6 章 从协议到软件的逐层具象化

6.1 连续的设计与不同的证明义务

从生态协议到具体软件，可以理解为同一组意图逐渐获得更具体的表达。协议说明共同关系，架构安排责任和能力，接口暴露交互，设计规范提供惯例与建议，实现把这些选择变成可运行行为。这条路径是连续的，但每个层次承担的说明义务并不相同。

例如，“应用退出后能够返回共同入口”在生态层表达协作承诺，在架构层需要确定退出和入口接管分别由谁负责，在接口层需要定义请求与结果，在实现层需要处理资源、时序和错误。自然语言可以贯穿这些层次，但不能用一句上层愿望替代所有下层说明。

因此，本书不主张取消架构与实现的概念区别，而是反对把它们当成彼此隔绝的工作阶段。一个实现反馈可能暴露接口无法表达必要状态；接口修改又可能要求重新划分责任。设计能够向下细化，也应允许证据沿着同一条语义链返回。

具体推进可以采用小范围贯通：先为一个协作行为形成最低限度的协议、接口和实现，在实际运行中验证，再扩展其他行为。这样能够较早发现抽象是否成立，也避免长时间停留在无法检验的宏大结构中。小范围贯通没有放弃协议优先，因为实现仍然依据明确承诺；它只是把一次需要承担的未知缩小到可以观察和修正的范围。

6.2 架构是责任关系的具体化

讨论架构时，人们容易首先想到模块名称、部署图或技术选型。本书首先关注的是：哪些承诺由哪些部分承担，哪些知识需要集中，哪些决定应保持局部，错误会被限制在哪里。这些关系决定了具体模块为什么存在。

若共同入口只需要识别和切换应用，架构就不应因为实现方便而顺手把所有应用数据归入入口管理。相反，若协议要求统一保存某种公共设置，就需要有明确的公共责任主体，而不能让每个应用自行维护互不一致的副本。架构是对协议责任的落实，也会暴露协议尚未解决的分歧。

一个架构方案还可能有多种实现。公共能力可以由库、独立服务或设备机制提供，具体选择受资源和环境约束。自然语言设计应说明哪些结构属于必须保持的关系，哪些只是当前选定的技术路径。否则某个临时框架很容易被误认为生态不可改变的本体。

6.3 接口让承诺可被触发和观察

接口是主体边界上的可使用能力。它不只是函数签名或消息字段，还包含调用条件、行为含义、完成判据及错误语义。字段完全匹配而含义不同，仍然会造成协作失败。例如，

两个系统都返回“成功”，一个表示请求已受理，另一个表示事务已完成，下游无法据此安全行动。

从协议推导接口时，应逐项检查抽象承诺是否获得可观察形式。若承诺可以查询任务状态，接口需要区分哪些状态；若承诺调用失败不破坏既有数据，错误返回需要与实际影响保持一致；若承诺可以取消任务，则应说明取消何时仍然有效，取消后已经发生的作用如何处理。

接口也可以采用不同媒介。它可以是机器调用，可以是文件约定，也可以是人机交互入口。媒介变化不会消除语义义务。一个按钮、一个命令和一个 API 可能触发同一协作行为，应当共享相关含义，而不是各自定义一套无法对应的完成状态。

6.4 规范指导不等于收回自由

在接口之上建立设计规范，可以降低重复决策和学习成本。共同的交互习惯、推荐的数据组织、错误提示原则和参考实现，能够帮助新的创造者快速进入生态。但建议性规范应维持其建议性质，不能因为写在同一份文档里就悄悄变成全部强制要求。

一个规范可以解释为什么推荐某种做法，以及什么情况下允许偏离。比如固定返回位置有利于形成操作预期，但某类无屏设备可能需要另一种等价入口。此时应检查上层目标是否仍成立，而不是用表面一致性排除合理实现。反过来，如果位置本身已被设为必须满足的可访问性要求，就不能随意把它降格为建议。

规范的成熟不等于细节越来越密。某些细节被反复验证后值得沉淀，某些要求随着环境改变应当撤销。规范应积累可复用的理由，而不只是积累规定。没有理由的规定容易成为历史负担，AI 还可能因为机械遵守而放弃更合适的路径。

6.5 逐层具象化需要保留什么

从一层进入下一层，首先要保留不能改变的承诺，其次要显式记录新增决定，最后要说明尚未决定的空间。把这三类内容分开，可以区分合法细化与偷偷改写目标。下层为实现而增加状态是正常的，下层改变“成功”的含义则可能影响整个协作。

这种关系可以用可追溯的问题来检验：一个接口字段服务哪项承诺，一项强制校验保护哪种边界，一个内部限制来自何处，一次架构选择依据什么环境条件。并非每行代码都需要映射到一句自然语言，但关键行为应该能够回到有效规则，关键规则也应该能够找到实现或验证位置。

反向追踪同样重要。若真机运行证明某种承诺无法在当前硬件上成立，AI 应当报告环境证据、受影响的规则和可选调整。它不能只修改实现，使测试表面通过，再默认原承诺已经改变。修改发生在哪个抽象层次，决定了谁有权接受它以及需要重新验证哪些协作。

6.6 多种视图与一个共同语义

复杂系统无法只用一种表示充分表达。自然语言便于讨论目的和例外，状态图便于观察转换，结构化接口便于机器检查，测试便于提供行为证据。这些视图可以共同存在。它们应围绕共同语义相互校验，而不必被压成一份无所不包的文档。

AI 在这里可以承担翻译和一致性检查工作：根据场景描述生成候选状态模型，根据接口约束生成测试，根据运行失败反查相关规则。但翻译不是保证，模型生成的多份材料可能共享同一个误解。因此，至少一部分验证需要来自独立证据，例如真实设备行为、外部接口记录或适当主体的场景确认。

逐层具象化完成后，具体软件获得了在生态中行动的资格。它拥有共同承诺，也保留内部自由。多个这样的实现能否真正独立开发、互不挤占而又保持兼容，不能仅靠文档宣告，还要由边界、资源和兼容机制来支撑。

第 7 章 独立创造与共同兼容

7.1 独立不等于互不相关

独立创造的价值，在于一个参与者能够在理解共同边界之后开展工作，而不必掌握其他所有参与者的内部细节。独立不是完全没有依赖，而是依赖被压缩为可理解、可检验的契约。随着生态增长，这种压缩决定新的创造者能否进入。

如果开发一个应用必须阅读其他全部应用的源代码，并模仿它们的隐藏习惯，生态实际上仍然被整体实现绑在一起。即使每个项目拥有独立目录，也不构成真正的独立创造。相反，不同语言、不同作者甚至不同 AI 生成的实现，只要在共同边界上可检验地一致，就可以参与同一个生态。

这并不要求协议永远稳定。它要求依赖者能够知道当前依赖的具体承诺，知道变化如何通知，也知道何时需要重新验证。独立创造既需要自由进入的条件，也需要可信变化的条件。

7.2 公共责任与私有空间

共同环境通常存在公共资源，例如设备能力、字体、网络配置或少量公共状态。共享可以减少重复建设，但共享范围越大，变化牵连越广。公共化一项能力前，需要判断它是否真的具有稳定共同语义，还是只是几个应用暂时采用相同实现。

私有空间同样需要明确定义。一个应用自己维护的数据，不应因为其他应用能够访问同一存储介质，就自动成为可以任意修改的公共数据。路径可见性、技术可达性与修改权并不是一回事。最小权限和责任隔离在这里首先是一种协作要求，具体隔离强度再由环境和风险决定。

共同资源也不必由一个组件包办全部调度。对于某种硬件能力，生态可以约定排他使用，也可以允许满足条件的小型任务并行。选择取决于资源现实和失败后果，不能把“并行更多”当作必然进步。需要共同确定的是资源占用怎样被识别、切换意味着什么以及冲突由谁处理，具体算法则可以在这些条件内继续演化。

公共与私有并非固定二分。某些数据可以被读取但不得修改，某项能力可以共享调用但不共享内部状态，某种资源可以按约定独占一段时间。协议应表达这些实际关系，而不是只贴上“公共”标签，随后让实现自行猜测所有权和生命周期。

7.3 识别与仲裁是不同职责

一个公共入口可能负责识别应用、展示可用能力并发起运行，但它不必自动承担所有冲突仲裁。例如两个应用使用相同标识，入口可以如实呈现识别结果，而将冲突处理交给应用作者或其他指定机制。这是否适当，取决于生态明确选择的责任分工。

关键在于，这种边界必须有一致后果。如果入口不负责仲裁，就不能向使用者承诺已经消除所有冲突；应用作者也不能把未处理的冲突默认推给入口。生态可以选择集中管理，也可以选择分散处理，但不能让责任落在各方都以为别人负责的空隙里。

自然语言程序设计在这里要求克制。AI 经常可能主动补充看起来合理的管理能力，却在无意中扩大公共组件的职责。判断某项“完善”是否合适，应看它是否符合既定协作秩序。一个功能能做、容易做，并不意味着应该由这个主体做。

7.4 兼容以承诺为依据

兼容不是两个程序都能启动，也不只是版本号相同。它意味着在共同场景中，各方仍能依照有效承诺完成协作。数据形式相同而状态含义变化，可能构成实质性不兼容；内部实现变化很大，但外部承诺保持，则可能继续兼容。

因此，版本管理应服务承诺变化。某个生态可以约定：接入指南和可依赖行为没有变化时，不增加对参与者有意义的协议版本。也可以对内部构建另设标识，用于故障追踪。两种标识解决的问题不同，不应把每次内部修改都推成所有使用者需要理解的生态升级。

兼容还需要声明范围。一项实现可以满足独立运行要求，却尚未验证在某个集成系统中运行；它也可以适配一个具体宿主，而未覆盖其他宿主。把这些结论分别记录，能够避免用一次成功测试代替全部兼容证明。所谓“支持”，应对应明确环境和可重复证据。

7.5 用独立实现检验协议

协议写得清楚与否，可以通过相互独立的实现暴露。两个实现者仅依据共同材料完成任务，再进行对接，能够发现文档中被共同作者习惯掩盖的空缺。若只有协议制定者本人能够正确实现，协议可能仍依赖未表达的知识。

但两次实现成功也不能证明规则已经完备。它们可能使用相同模型、参考相同样例，从而共享同样假设。更有价值的检验是有意识地改变实现路径，并检查边界情况：不同启动顺序、重复请求、部分失败、资源不足、旧数据和异常退出。测试范围应依据具体风险，而不是仅增加测试数量。

参考实现有助于理解，也可能意外成为事实上的唯一标准。因此，应区分规范要求、参考选择与已知限制。若参考实现与规范冲突，需要明确是实现错误、规范含糊还是规则确实需要修订，不能让后来者靠猜测决定服从哪一个。

7.6 自由需要失败隔离

允许独立探索，就必须考虑一个参与者失败时其他参与者如何继续。共享存储、公共配置和运行环境可能把局部错误放大。边界设计应让可恢复的错误尽量停留在可恢复范围，同时让重要失效能够被发现，而不是被静默隐藏。

隔离不只有操作系统机制。清楚的数据所有权、可恢复的持久化流程、可识别的状态、有限范围的授权，都能减少失败传播。技术环境很有限时，更需要明确哪些保护无法提供，以及使用者需要怎样判断风险。承诺必须与现实能力一致，不能靠优美措辞制造不存在的保证。

当共同边界稳定、内部自由真实、失败后果可理解时，独立创造才能长期持续。生态参与者越多，协议修订越可能影响不同利益与历史依赖。于是治理从可选的附加工作，变成维护共同创造条件的必要工作。

第 8 章 协议生态的治理与演化

8.1 规则本身也是需要治理的工具

协议解决协作问题，也会产生新的问题。规则可能最初基于某种设备限制，后来限制已经消失；某项共同约定可能便于多数实现，却使少数合理场景无法进入；一些临时补丁可能逐渐形成难以解释的例外。规则需要被维护，不能因为位于底层就免于检验。

因此，生态治理首先要区分稳定原则与可修订选择。保护参与者已经依赖的承诺，是持续合作的重要条件；某个字段名称或推荐布局，则可能只是当前选择。但哪些内容属于哪一类，也不能由单个实现者在不方便时临时决定。规则应提供修改规则的方式。

这形成一个递归问题：协议管理协作，治理机制管理协议。递归不要求无限增加制度层次。小型生态可以由明确的维护者处理变更，大型生态可以采用提案、审查和版本迁移。关键是参与者能够知道什么状态的规则已经生效，什么仍在讨论。

8.2 权限随影响范围上升

内部实现改变，如果不影响共同承诺，通常可以由实现者在授权内决定。改变接口含义，需要考虑直接调用者。改变公共数据责任，可能影响整个生态。影响范围越大，决定所需要的授权、协商和证据通常越充分。

这种关系不意味着所有变更都必须由一个最高管理者批准。生态可以将不同领域委托给不同维护者，允许局部规则自治，并规定跨边界冲突如何处理。奥斯特罗姆关于公共资源治理与多中心制度的研究，为理解多层次规则和参与者自组织提供了参照。[7] 本书借鉴这种视角，但不把社会制度的研究结论直接等同于软件系统定律。

AI 参与治理时也应遵循同样边界。它可以自动修复内部实现，可以在授权范围内更新建议，也可以提出协议变更。只有得到相应授权的变化才能生效。模型越能快速生成修改，越需要让修改的影响范围和决定来源保持可见。

8.3 从运行证据提出变更

规则修订应尽量从具体矛盾出发。某个异常暴露协议缺少状态，某次集成证明两条义务无法同时满足，某个新环境使旧约束失去必要性。这些证据比单纯追求“更现代的架构”更能说明为什么需要改变共同预期。

一个变更提案至少应说明旧规则在什么情境下产生什么问题，拟改变什么承诺，哪些实现受到影响，以及如何验证新规则确实改善了原问题。对于替代路径，还应说明为什么

不通过局部实现解决。若所有问题都上升为公共规则变更，协议会迅速膨胀；若所有问题都由局部绕行解决，生态又会失去一致性。

自然语言在这里能够保留推理过程，而结构化材料可以帮助管理影响。关键不是提案的形式，而是决定能够被后来者理解：当时有哪些证据，为什么采用这一方案，哪些风险尚未解决，出现什么新事实时应该重新讨论。

8.4 迁移是协作的一部分

发布一份新协议，并不会让旧实现自动改变。现实中可能同时存在旧设备、旧数据和暂时不能升级的应用。治理需要安排这种共存，决定哪些差异可以通过适配处理，哪些必须明确拒绝，以及何时结束旧承诺。

兼容层可以帮助过渡，却也会增加长期维护负担。如果每次变更都永久保留所有旧行为，系统会越来越难以解释。退出某项旧支持并非天然不负责任，重要的是既有依赖是否被识别，迁移条件是否合理，影响是否被清楚表达。持续演化需要进入机制，也需要退出机制。

迁移还涉及知识。一个新实现可能行为正确，却改变了操作者习惯；一个新字段可能需要历史数据补齐。若只测试代码而忽略这些条件，发布成功并不意味着协作已经成功迁移。真实环境验证必须包含承担后果的人和系统。

8.5 例外与分叉

开放生态无法保证所有参与者始终拥有相同目标。某些场景需要偏离建议，某些实验需要测试新规则，还有些参与者可能选择另一条发展路线。治理应区分有边界的例外、临时实验和真正分叉，避免把所有差异都当作故障，也避免把互不兼容的体系包装成仍然相同。

一个例外至少需要说明适用范围、理由和对共同承诺的影响。实验环境可以允许更大自由，但不能把实验结果未经验证地扩展到全部使用者。分叉则意味着某些共同预期不再成立，需要新的标识或清楚的关系说明。命名一致不能弥补行为分裂。

开放并不意味着没有边界。恰当的边界能让不同路线并存，也让使用者知道自己选择了什么。一个生态若只允许唯一实现，独立创造可能成为名义；若完全无法识别规则差异，合作又会退回个别协商。治理需要在可分化与可理解之间维持关系。

8.6 走向语义层面的治理

协议治理最终遇到的困难，常常来自语言：某句话到底是强制要求还是建议，某个示例是否可以推广，一项未写出的决定是否已经授权，两个目标冲突时谁具有裁决资格。协议越依赖自然语言，这些问题越不能完全交给阅读习惯。

前两编已经建立了宏观路径。信息在协作中获得作用，协作产生工具需求，协议提供独立创造的共同条件，架构和接口把这些条件逐层具象化，治理让生态能够在变化中保持可理解。现在需要进一步回答：自然语言本身怎样承载这一整套工程关系。

本书接下来的重点因此转向语义规格。它不要求人为每句话制定新的语法，也不要求人预先枚举所有状态。它要建立一种描述与判断方式，使目标、现实约束、示例、授权和反馈能够彼此配合，让 AI 在足够明确的边界内开展尚未被人逐一写出的创造。

第三编 自然语言的语义工程

共同协议之内，具体场景仍需要被充分理解。本编说明整体语义规格、有限示例、价值准则与授权的不完备性如何配合，使人无需枚举全部细节，也能给 AI 留下可追溯的自主空间。

第 9 章 自然语言的语义规格

9.1 共同协议之内仍然需要表达意图

协议为独立的创造物规定了相互理解和协作的条件。进入具体场景以后，协议仍不能替使用者决定全部事情。同样遵守数据接口的两个应用，可以服务于不同目标；同样具有保存能力的两个设备，也可能对等待时间、断电恢复和交互反馈采取不同策略。生态协议给出共同生活的边界，场景描述则需要说明在这个边界内准备开展什么活动。

自然语言在这里承担一项明确的工程职责：把人的处境、意图、判断和授权，表达成能够指导行动并接受检查的语义。我们关心的不是一句话听起来是否清楚，而是承担实现任务的主体能否据此作出合理且有权作出的决定。描述者说得流畅，执行者理解得自信，都不足以独立证明这件事已经成立。

例如，“做一个方便的记事本”表达了方向，却没有建立充分的共同预期。方便可能指输入快，也可能指内容不容易丢失，或者能够直接交换文本文件。如果这些解释导向不同设计，工程就需要进一步收敛语义。收敛可以来自补充说明、既有协议、现场事实或明确授权，不必全由使用者亲自写出。

9.2 句子自由与整体规格

本书允许人以日常语言组织描述。先说目的还是先举例，使用长句还是短句，都可以由表达习惯决定。但是一段能够指导工程的描述，需要具有可以识别的语义结构。它应当让目标、条件、行为、约束与决策权限之间的关系显现出来。表述自由与语义要求由此同时存在。

这种要求发生在整体描述层面。某个目标可能在前一段提出，例外条件可能在后续对话中补充，权限则可能继承自既有协议。AI 需要整合这些内容，而不能仅对单句进行孤立解释。一份场景描述甚至可以分布在文字、实物观察、接口说明和已经确认的对话里；工程系统应当保留这些信息的来源和关联。

受控自然语言方法提供了有益参照。例如，需求工程中的 EARS 用有限句型帮助表达事件、状态和系统响应。它说明自然语言能够通过适当结构减少歧义。[8] 本书将约束重点

放在语义关系和决策条件上，容许不同句型表达相同要求，也容许一组互相关联的段落共同完成规格。

如果要求用户填写过多固定栏目，语义规格又可能变成一套形式手续。一个很简单的任务，已有上下文可能已经覆盖关键条件；复杂任务则可能需要多轮调查。检查应随问题规模展开。形式只在帮助理解时有用，填满表格本身不构成工程上的充分性。

9.3 一句话具有何种规范效力

理解内容还不够，AI 还需要理解内容的效力。“数据必须能够恢复”“尽量缩短等待”“可以自行选择缓存方式”“例如重复点击时不应重复提交”，分别表达不变量、偏好、授权与示例。把它们一律视为刚性命令，会压缩探索空间；把它们一律视为参考意见，会破坏共同预期。

语义规格因此应当保留要求的作用范围、触发条件、优先关系和例外。推荐做法允许有理由地偏离，偏离后仍须满足上层目标。强制要求需要被实现和验证，确实无法实现时应报告冲突。授权允许 AI 在规定范围内作出决定，授权本身不能被推定为能够修改其他主体的权利。

示例尤其容易被误读。有些例子规定一个必须成立的具体行为，有些例子只是解释设计倾向。“连续点击只产生一次扣款”通常承载强约束；“像翻书一样自然”主要提供体验方向。工程检查需要判断例子中的哪些属性有规范效力，哪些只是帮助理解的表面特征。不能因为示例里出现了某种界面，就把该界面固定为唯一实现。

这些区分可以由 AI 在内部进行，但应在影响结果的地方让人能够核对。语义结构的可见性不等于把所有推理过程都展示出来。需要显现的是将约束行动的解释，以及这些解释可能带来的后果。

9.4 工程上的充分是一种决策条件

本书所说的语义充分，不要求把现实世界全部写完。它要求当前工程阶段所必需的决策具有依据，尚未确定的事项具有处理方式。一个实现细节没有写出，可能完全合理；一个决定用户是否失去数据的规则没有写出，也没有被授权，就会形成必须处理的缺口。

我们至少需要区分四类未定事项。事实缺口需要观察、查询或试验；价值冲突需要依据已确定的判断准则处理；实现选择可以在授权内搜索；未来事件的不确定性需要准备监测与恢复机制。将这些问题全部归入“需求不清楚”，会导致无效追问；将它们全部交给模型猜测，又会掩盖不同决策的责任来源。

充分性还依赖开发阶段。验证一个交互概念时，可以暂时使用模拟数据；准备进入真实运行时，就需要确认数据来源和权限。如果系统声称已经完成交付，却仍把生产环境的

关键条件当作未验证假设，阶段转换就发生了错误。自然语言工程必须说明哪些结论已经成立，哪些结论只在当前假设下成立。

因此，语义规格不是一份一经签字便不会变化的文本。它是能够随着证据增加而修订、同时又保持承诺可追溯的工程对象。其质量体现为决策是否有根据、实现是否有自由、变化是否能被解释。

9.5 解释也需要接受验证

AI 可以把自然语言整理成结构化说明，但整理结果本身可能引入新假设。一个常见问题是：原文没有规定某项行为，模型为使文档显得完整便补写了它，随后又把自己的补写当成用户要求。工程开始之前，这种来源混淆就已经改变了权责关系。

可行的方法是对重要解释进行反向核对。AI 用具体情形说明自己准备实现的行为，指出哪些来自明确要求，哪些继承自协议，哪些属于授权内选择，哪些仍待确定。人不必逐字审查全部说明，只对超出既有授权、已有准则无法裁决或足以改写共同承诺的决定作出判断；授权内的选择由 AI 直接实现、验证，并在影响后续判断时留下记录。

例如，用户要求“关闭后仍然保留记录”。AI 可以解释为“用户看到保存成功后，即使应用重启也应能恢复内容”，并指出异常断电是否包含在承诺范围内尚待确认。这种表达把抽象要求转化为可以讨论的可观察结果，同时没有暗中把承诺扩展到一切故障条件。

解释的验证还可以通过反例进行：哪些结果明显违反原意，哪些结果存在多种合理选择？一个能够排除错误行为而保留合理差异的描述，往往比一份详细规定内部实现的文档更有指导价值。接下来需要回答的，便是哪些语义关系应在场景中得到充分表达。

第 10 章 一个场景需要表达什么

10.1 从实际处境识别需求

场景是主体在具体环境中，为某种目的采取行动并承担结果的过程。它同时包含人正在做什么、信息从哪里来、行动受到什么限制以及问题发生后影响谁。软件功能只是可能介入这个过程的一种能力。先理解场景，可以避免把使用者随口提出的工具形式直接当作需求本身。

有人提出“增加一个自动汇总按钮”，实际困难可能是各部门对同一字段理解不同。此时，按钮只能加快不一致信息的汇总。场景调查应继续追问数据如何产生、谁能够更正、汇总用于哪项决策，以及错误结果造成什么影响。调查的深度取决于这些问题是否会改变解决方案，不能为追求完整而无限延长。

场景描述还应保留主体差异。使用工具的人、批准使用的人、受到处理结果影响的人，可能各不相同。他们的目标也不必一致。工程师不能把最容易接触到的一个人的意见，自动当作整个协作系统的意志。对当前任务而言，需要明确谁提供事实，谁确定目标，谁有权承诺其他主体需要承担的改变。

10.2 目标与价值判断共同规定方向

目标描述希望出现的结果，价值准则帮助在多个结果之间作出选择。对于一个离线记录工具，“能够随时记下内容”是目标；“优先保护已确认保存的数据，其次减少操作，再次节省存储”是可以讨论的判断顺序。目标使搜索有方向，价值准则使未列举的情形仍有解释依据。

价值准则不宜全部压缩成一个分数。节省几十毫秒不能自动抵消未经授权的数据外传；界面更简洁也不能自动抵消无法恢复的记录丢失。工程中通常同时存在硬约束、优先级和可协商偏好。只有在允许权衡的范围内，才适合用成本、时间或评分比较方案。

准则还需要说明生效条件。“尽可能快”在交互预览中可能意味着先显示近似结果，在资金结算中则需要服从准确性要求。条件不同，合理的取舍也不同。AI 的任务是识别这些条件与准则之间的关系，不应把某个局部偏好推广为全系统的最高原则。

如果目标发生矛盾，人应尽量说明取舍依据，而不是只替 AI 选择一次答案。补充“关键记录不得静默丢弃，但允许明确报告失败并保留现场”，就比单独回答一次异常该如何处理更具有可推广性。它扩大了 AI 此后能够自主处理的场景范围。

10.3 入口与出口界定一段工程承诺

入口说明任务从什么条件开始，出口说明达到什么条件才能认为本段过程结束。两者应当与现实状态关联。例如，启动应用可能要求存储可访问、必要配置可读取；退出应用可能要求必要数据已经持久化，并把控制权交回约定的承接者。这些条件与界面上出现“进入”或“退出”按钮并不是同一回事。

以退出保存为方法示例：用户请求离开时，系统应区分退出意图已经收到、保存正在执行、保存结果已经确认、控制权已经转移几个事实。若只把“收到退出请求”视为出口，就可能在写入未完成时交出资源。若把释放全部运行现场当作出口，又可能增加协议并未要求的工作和等待。

入口与出口不必强迫系统采取某种内部结构。应用可以使用事务、日志、文件替换或其他机制，只要满足既定环境下的外部承诺。前置条件、后置条件和不变量是契约式设计中成熟的表达资源，可以被自然语言规格吸收。[9][10]

还应区分正常出口、取消出口、失败出口与恢复入口。一个过程未达到成功条件，不意味着只能永久停留。它可以进入明确的失败状态，保留必要信息并提供后续处置。但失败出口是否允许放弃数据、占用资源到何时，都必须由目标和权限关系决定。

10.4 环境现实与环境约束

环境现实是现在实际存在的条件，包括设备能力、现有软件、数据形态、网络质量、人员操作习惯和可调用服务。环境约束则规定方案必须遵守的边界，包括资源上限、协议义务、操作权限及协作方已经作出的承诺。两者有交集，却不能互相替代。系统目前没有网络是事实；禁止将数据传到外部网络是约束。

事实需要证据支持。AI 不能因为常见设备具有某种能力，就假定眼前设备也具有这种能力；不能因为某类接口通常存在，就把它写进架构。条件未知时，可以提出暂定假设并安排验证，或者选择能够容忍多种结果的方案。必须避免让未经验证的假设悄悄升级为其他决策的基础。

环境也包括人的工作条件。使用者是否能够等待、是否有条件阅读长提示、是否能在故障后接触到设备，都会影响可行设计。把这些因素视为“产品之外的问题”，容易生成技术上能运行、现场却难以使用的软件。自然语言能够表达这些跨越技术与生活的条件，这是其重要优势。

描述环境时不必复制所有背景资料。应保留会影响决策的事实及其来源，并说明它们何时需要重新检查。一个已经变化的环境事实，可能比一项没有提供的信息更危险，因为它会持续支持过时的判断。

10.5 不变量与权责边界

不变量描述在指定范围内必须持续成立的性质。例如，同一笔确认操作不得被重复计入，未授权主体不得读取某类记录，公共资源不得被某个应用永久占有。它们帮助 AI 判断新的行为分支是否可接受，也帮助测试从若干示例推广到一类场景。

权责边界则回答谁可以决定怎样维护这些性质。AI 可以选择内部数据结构，并不意味着它可以改变数据归属；可以优化交互顺序，也不意味着它可以取消用户必须知情的确认。允许自主实现的是一个有边界的行动集合，不能用“对目标有帮助”替代授权依据。

在多人或多智能体参与的场景里，还需要确定授权是否能够转交。一个智能体得到读取材料的权限，不当然拥有把材料交给另一个服务的权限。转交任务时，应同时转交适用的约束和明确允许的能力；无法合法转交的部分，应通过受限接口完成协作。

责任也应当与可控制范围相称。让实现者对无法观察的上游变化承担全部结果，不会增加可靠性，只会模糊故障处置。清楚的边界能够帮助各方知道什么时候自主处理，什么时候需要另一个主体参与。

10.6 示例与验证把描述连接到行动

场景描述最终需要落到可观察结果。典型示例帮助建立共同理解，边界示例揭示容易发生分歧的地方，反例则说明哪些表面上完成任务的行为仍然不可接受。三者共同为 AI 提供推导方向，也为后续验证提供起点。

例如，用户连续发出退出请求、保存中断后再次进入、保存失败后选择继续编辑，可以成为同一退出场景的不同例子。人无须列出全部组合，但应表达这些例子背后的稳定准则。AI 随后能够检查未列出的相邻情形，并区分哪些结论可以直接推广，哪些仍然缺少依据。

验证依据应随描述一同形成。成功提示是否必须对应可恢复的数据，重复请求是否允许产生额外操作，恢复后应展示哪个版本，这些都可以被观察。某些价值目标只能通过长期使用评价，则应明确观察周期和可能的外部因素。

一份场景语义规格由此形成内在连接：处境说明为什么做，目标说明做到什么，准则说明如何取舍，环境说明现实可行性，入口出口与不变量说明承诺，权限说明谁作决定，示例和验证说明怎样进一步认识结果。它可以被自由表达，却不能让这些关键关系在实现中失去依据。

第 11 章 示例与授权的不完备性

11.1 有意留下空间是一种设计

复杂系统中，人不可能预先列出所有事件组合，也不可能掌握全部可行实现。自然语言程序设计需要正面处理这种有限性。我们可以充分规定必须成立的性质，同时明确允许 AI 补充哪些内容，以及补充决定必须满足什么条件。本书将这种有范围、有依据、可追溯的开放称为授权的不完备性。

它与遗漏存在实质区别。遗漏意味着关键事项尚未得到处理，执行者只能猜测；授权的不完备性意味着描述者知道某类选择尚未确定，并把其中一部分决定权交给执行者。自由度来自明确的权利安排，不能从文字缺失中自动获得。明确授权可以针对一类目标和行为空间整体授予，无须逐项列出实现决定；示例帮助理解授权怎样适用，不应把自由重新压缩成一张枚举清单。

例如，“你自行处理异常”可能把价值判断、数据处置和实现选择混在一起。“对于可重试且不会重复产生外部效果的操作，可以自行选择重试策略；涉及放弃已确认数据的处理需要另行决定”，则划定了可推广的空间。第二种描述仍没有规定全部异常，却让未列举情形具有可判断的归属。

开放空间还可以逐步扩大。早期工程中，人可能保留较多决定权；经过重复验证，某类问题的解决原则已经稳定，就可以转为 AI 自主处理。授权变化应有明确记录，避免一次临时例外被永久扩展成普遍权限。

11.2 示例传递的是判断结构

一个有效示例同时提供事实条件、期望行为和判断理由。如果只有结果，AI 可能学到偶然相关的表面特征。如果说明“连续点击不能生成多份相同提交，因为用户只表达了一次业务意图”，AI 就有机会把原则推广到网络重传、页面刷新或代理重试等不同事件。

推广的依据是这些情形在关键关系上相同。它们是否对应同一个业务意图，是否可能重复产生不可逆效果，是否具有能够识别重复的标记，都需要进一步判断。外观相似并不保证语义相同。两个相隔很久的相同点击，可能代表两次真实需求，不能一律合并。

因此，人提供的少量例子最好具有差异。一个正常例、一个边界例、一个明显不允许的反例，往往能够暴露判断轴。比如“允许重复读取”“同一请求不能重复扣减”“用户明确新增的第二次请求应正常执行”，共同表达了对重复的语义分类。示例数量有限，提供的信息关系却可以丰富。

AI 应当从这些关系生成候选原则，再通过新例子检验原则。在需要人确认时，让人判断一个能够解释原例且适用于新情形的原则，比逐一确认所有行为分支更有效。既有目标、准则与授权已经足以支持的推广，则由 AI 继续推导和验证。这种交互既减少表达负担，也避免让自由度变成无法检查的随意发挥。

11.3 从例子推导到权限不能越级

示例能够支持对意图的理解，却不能凭自身授予超出描述者权限的行动。用户举例说“可以像秘书一样帮我处理材料”，并不意味着 AI 可以访问所有同事的文件。类比提供行为方向，访问与处置仍然需要独立的授权依据。

同样，依据一个示例推断出某项行为有利于目标，不等于已经证明该行为符合全部约束。AI 需要检查推导涉及的对象、资源、后果和可逆性是否仍在授权范围内。原例只涉及临时副本，推广后若触及唯一原件，就已经发生重要变化。原例只影响当前用户，推广后若改变其他人的工作结果，也需要重新判断。

工程系统可以要求 AI 对关键扩展提供简短依据：它继承了哪项准则，增加了什么假设，影响了哪些主体，如何验证其行为。无需把每个微小代码选择都变成审批记录，但对外部承诺和不可逆后果的改变应当能够定位。

这是一种推导责任。执行者可以探索更多可能性，同时需要说明自己为何有权把某种可能性落实为行动。推导的自由与行动的权限并不完全重合：在受控环境中比较多个方案，可以拥有比真实执行更大的探索范围。

11.4 自由度需要形成可搜索的空间

如果所有要求都写成唯一做法，AI 只能沿既定路径执行；如果只有宽泛目标，AI 又很难知道哪些路径可接受。有效的授权应规定结果性质、不可越过的边界以及可选择的维度。这样便形成一个能够搜索、比较和验证的方案空间。

例如，要求应用退出前满足持久化承诺，同时允许 AI 自行选择写入组织、内部状态划分和反馈实现。协议无需事先指定每个函数如何调用，但需要给出存储位置的合法范围、可接受的故障行为与控制权转移条件。实现者可以据此发现更适合设备的结构，而不会改变使用者已经获得的保证。

自由度还应与试验条件匹配。在可恢复的模拟环境中，AI 可以更积极地比较方案；对共享数据或运行设备的修改，则需要遵守更严格的操作边界。相同的算法探索，放在不同环境里可能具有不同后果。授权因而应同时指向“可以决定什么”和“可以在哪里、以何种方式验证”。

保留自由度不意味着每次都重新搜索所有可能。已有经验可以提供优先候选，只要没有把经验固化为永远适用的命令。工程判断需要比较继续沿用、局部修改和重新探索的成本。

11.5 不完备性也应具有停止条件

开放探索需要知道何时暂时停止。对当前问题而言，候选实现通过关键验证、剩余风险处于允许范围、继续搜索的预期收益不足以抵消成本，都可以成为停止依据。若只有“不断优化”的指令，AI 可能持续改动已经满足要求的系统，反而增加不确定性。

停止条件不等于宣告理论上的最优。它表示当前环境、目标与资源条件下，已有足够证据支持采用某个方案。后续环境变化或新证据出现，可以重新开启搜索。这里的确定性是一种有条件的工程承诺，而不是对未来全部情形的保证。

当 AI 无法判断某个选择是否获得授权时，也不应立即停止全部工作。可以先完成不依赖该决定的部分，准备候选方案与可观察差异，再把尚需裁决的最小问题交给人。这样的反向求证保留了工程连续性，也把人的注意力放到确实改变结果的地方。

授权的不完备性因此构成一个双向机制。人通过目标、原则和例子释放自由度，AI 通过推导、试验和反例揭示描述中的不足。新的裁决补充共同语义，下一轮探索由此能够在更可靠的边界内继续展开。

第 12 章 冲突裁决与责任追溯

12.1 先辨认冲突发生在哪一层

AI 发现的问题不都属于需要人裁决的冲突。有些是事实尚未查明，有些是实现没有找到，有些才是两个有效要求无法同时满足。把技术困难立即上交，会让授权失去作用；把真实的价值冲突当作实现问题自行解决，则会越过责任边界。

可以先检查四种关系。事实是否相互矛盾，要求是否在同一条件下互相排斥，实现是否违背已确定要求，以及参与者是否对决定权存在不同理解。不同关系需要不同处置：查证事实、依据优先级选择、修复实现，或者澄清授权。一个报告应说明属于哪一种，而不只是给出“存在风险”的笼统标签。

例如，“离线时立即确认外部系统已经收款”可能同时包含环境限制与不可能成立的确认要求。AI 应说明无法取得外部证据，而不能生成一个看似成功的本地提示。若要求改为“先记录待确认请求，并明确显示尚未完成”，就形成另一项可实现的承诺。

冲突分析让人的决定具有具体对象。人所面对的是哪些条件不能同时成立、放宽其中哪一项会产生什么后果，而非从零接手整个技术问题。

12.2 优先关系必须有来源

规则冲突可以通过事先约定的优先关系解决，但优先关系本身需要正当来源。生态协议规定的公共资源义务、当前任务的明确授权、经确认的场景目标和建议性规范，具有不同作用范围。具体场景可以填补协议留下的空间，却不能仅凭局部便利取消协作方已经依赖的承诺。

不能简单地把“最新一句话”永远当作最高规则。它可能是一次局部例外、对旧条件的纠正，或者表达者尚未意识到影响范围的建议。AI 应判断这句话准备修改什么，并把重要后果呈现出来。如果有权主体明确修改了规则，后续执行就应采用新规则，同时保留变化记录。

同样，抽象原则也不应自动压倒全部具体说明。“用户体验优先”无法单独裁决数据归属或其他主体的权利。更高层价值需要通过具体适用条件进入工程，避免执行者以宏大目标为由自行扩张权限。

当已有优先关系能够清楚解决问题时，AI 可以直接处理并说明依据。只有缺少规则、规则之间存在未解决冲突，或决定超出授权范围时，才需要升级。保留人的裁决权，应当提高判断质量，而不是增加每一步的往返确认。

决定权也不能改变事实本身。一个有权主体可以调整期望结果，却不能通过指令使不存在的接口成为现实，或者使尚未完成的写入已经完成。若人的判断与可核验事实冲突，AI 应提供证据并重新讨论目标或路径。把权威意见当作事实真值，会同时损害工程可靠性与责任追溯。

12.3 反向求证需要形成可回答的问题

有效的求证应当具备事实、选择与后果。AI 先说明已经查明的条件，再给出少量合理选项，指出每个选项改变的承诺，以及哪些工作可以继续。人的回答由此能够直接进入工程，而不是再次开启漫无目的的需求讨论。

以退出保存为方法示例：设备存储已满，当前编辑尚未持久化，协议禁止静默丢弃已确认的数据。AI 可以提出保留编辑现场，并在相关能力存在时提供导出或清理方案，或者在明确放弃当前未保存修改后退出。删除旧记录可以作为需要新增授权的备选，但不能被默认为已经获准，更不能先执行后补问；它涉及另一项需要说明后果的取舍。

求证还需要说明时间条件。如果决定暂时无法得到，系统应采取哪个已获授权的临时状态，是继续运行、暂停相关操作还是退回可恢复位置。没有预定处置时，AI 可以先保护现场并停止那部分有争议的动作，而无需冻结与之无关的全部工程。

一次回答尽量转化为可复用的规则。例如，人确认“空间不足时不得自动删除旧记录”，今后同类问题便有明确依据。只有结果没有理由的裁决，也可以被记录为局部决定，但不能自动推广到所有相似场景。

开发阶段的求证还应与软件运行阶段区分。开发时把已经裁决的规则落实为可执行的异常处理；部署后，已知情况按规则自动处理，需要现场使用者选择时采用事先设计的交互。普通程序不应在每次预料之内的异常出现时，都等待开发者重新裁决。新的重大未知或超出既有规则的冲突，才进入相应升级流程。

12.4 追溯的是决定与依据之间的联系

责任追溯需要回答某项行为为什么出现、由谁决定、依据什么信息以及经过哪些验证。保存大量对话和执行日志不等于已经具有追溯能力。若关键决定淹没在成千上万行输出中，事后仍然难以解释系统为何如此行动。

一个有用的决策记录可以很短：事项、适用范围、提出者、授权来源、选择理由、相关证据和后续检验。它应能连接到被影响的协议条款、行为模型、实现版本与测试结果。记录的重点是因果与权力关系，不必把所有探索过程永久保留。

还需要区分要求、推导、假设和事实。要求表达应当怎样，推导说明在条件成立时能够得出什么，假设是暂时采用但仍待查明的前提，事实则需要相应证据。模型重复一项假设不会使它变为事实，多次代理转述也不会增加原始证据的强度。

经过这种区分，某个错误可以被定位到不同环节：需求本身不合理、解释发生偏移、授权判断错误、实现偏离模型，或者环境已改变。各类错误需要不同修复方式，追溯的目的正在于改善下一次协作。

12.5 权限责任与因果责任

谁允许执行、谁实施执行、谁受到影响、谁能够修复，可能是不同主体。一个智能体按照既定权限运行，并不意味着所有后果都应由它承担；人作出授权，也不意味着可以忽略执行系统在能力和证据上的限制。技术记录能够帮助解释责任关系，但不能自行替代现实组织的责任安排。

自然语言规格应把责任落实到可行动的对象。例如，需求方负责确认业务规则，工程执行方负责在约定边界内实现并验证，环境管理方负责提供真实的资源状态。遇到跨边界问题，需要明确接续责任，避免所有人都声称自己的局部环节已经完成。

同时，责任分配不能成为切断反馈的借口。发现上游规则可能错误时，下游即使没有修改权，也应能够报告异常。拥有最终决定权的主体，需要接收足够证据并承担更新判断的义务。这样，权利与责任才形成可运行的关系。

语义规格经过冲突处理和责任连接，便获得了持续执行的基础。下一步不是把每个字机械翻译成代码，而是推导出在这些条件与授权下可以成立的行为模型，并寻找模型可能遗漏的状态与连接。

第四编 自主推演与工程实现

语义必须进入可以检验的行动。本编从状态推演和架构搜索展开，把鸟枪法作为跨场景能力拆解与重组的方法，继而讨论自主工程、证据与现实验证。

第 13 章 由启发式描述推导状态模型

13.1 从若干例子发现变化的维度

人描述场景时，通常首先想到有意义的事件：按下按钮、完成保存、网络中断、退出应用。AI 要据此形成行为模型，需要进一步识别这些事件发生时，哪些状态差异会改变正确响应。同一个按钮在不同状态下可能具有不同含义；不同事件也可能表达相同意图。

状态并不等于界面。界面完全相同时，数据可能已经保存，也可能仍在等待写入；同样显示“正在处理”，系统可能正在本地计算，也可能已经产生外部效果。只按页面划分状态，会遗漏真正影响责任与后果的差异。状态模型应围绕可观察承诺、资源占有和行为条件展开。

在退出保存的方法示例中，关键维度可以包括是否存在未保存修改、保存是否正在进行、是否收到退出意图、存储是否可用，以及控制权是否已经转移。这些维度来自目标和例子，不必一开始就全部合并为一个巨大状态表。AI 可以先辨认独立变量，再判断哪些组合有意义、哪些组合不可能出现。

推导还应保留事件来源。用户重复请求、系统回调重复到达和设备重启，都可能造成相似表象，却需要不同识别机制。事件的来源、顺序和关联对象会影响它是否应被接受。

13.2 生成候选状态与转换

一个简化模型可以把运行过程分为编辑、保存中、等待处置和已交出控制权四个阶段，并另行记录修改状态与退出意图。它只是当前场景的抽象，不宣称覆盖设备的全部运行状态。采用这种分解，是为了让关键行为容易检查，同时保留实现方式的自由。

编辑阶段收到退出请求后，如果没有待保存修改，可以按协议交出控制权；如果存在修改，就记录退出意图并开始保存。保存中再次收到退出请求，通常不需要重新启动相同操作，但应保证该意图没有被丢失。保存成功以后，系统确认保存对应的是应当保留的数据，再决定是否转入退出过程。

这里立即出现一个需要展开的问题：保存期间是否允许继续编辑？允许继续编辑，就需要识别正在保存的版本与当前版本；暂时禁止编辑，则需要明确反馈，并确认这种限制

符合场景目标。AI 可以在已有交互授权范围内选择方案；如果原有要求已经承诺持续输入，就不能为了简化模型而自行取消它。

保存失败时，系统进入等待处置，保留有用的现场，并提供已获授权的后续动作。选择重试可以返回保存中，选择取消退出可以返回编辑。是否允许丢弃当前修改，要依照具体授权处理。这个过程显示出状态机的价值：原本隐藏在一句“退出前保存”中的不同承诺，变得可以逐一观察和讨论。

13.3 推导连接比列出状态更重要

状态名称完整，并不意味着行为模型充分。AI 还需要检查每类事件在每个相关状态下如何处理，哪些转换必须发生，哪些转换必须禁止，哪些事件可以被安全忽略。例如，控制权已经转移后，旧应用迟到的保存回调不能再次修改公共资源；已确认完成的请求再次到达，也不应重复产生同一外部效果。

异步过程使事件顺序尤其重要。一次保存可能先返回超时，随后才到达成功通知；退出请求可能与保存完成几乎同时发生。工程不必假装存在绝对同时，而应规定可识别的先后关系、状态检查或提交点，使不同顺序都不会破坏承诺。状态机方法为这些关系提供了表达资源。[11]

还应检查无法到达的状态、没有出口的状态和没有合法入口的转换。等待状态可能是合理设计，但需要明确什么事件能够结束等待，或者怎样让人主动介入。以无限等待替代错误处理，只是把未解决的问题推迟到运行现场。

模型还需要同时表达保持正确与取得进展。前者关注不能发生什么，例如保存未确认便丢失现场；后者关注在条件允许时最终应当发生什么，例如有效退出请求最终能够被处理。一个永远不退出的系统或许不会在退出时丢失数据，却没有完成退出功能。只验证禁止事项，会遗漏这种停滞；只验证能够结束，又可能忽视结束过程中的损害。

AI 的推导应同时生成反例。假如在保存未确认时退出，会违反哪项承诺？假如重复回调触发两次返回，会产生什么结果？能够构造反例，说明模型正在接触约束的实际效力，而不仅仅是在绘制看起来完整的图。

13.4 完备必须说明抽象边界

状态组合随变量增加而迅速扩大，现实环境还会带来模型之外的事件。工程上能够讨论的完备性，应当相对于已经选定的状态变量、事件类别和环境假设。对这些抽象对象完成检查，不能直接推出真实世界的一切情况均已验证。

例如，模型将所有可恢复的短时写入失败归为同一类，可以帮助控制复杂度；但如果某种失败已经破坏旧数据，就不应继续被归入相同类别。抽象成立的前提，是被合并情形

对当前承诺具有足够相似的影响。AI 需要说明分类理由，并在现实验证发现差异时拆分状态或事件。

完备梳理至少包含两种不同问题：已知类别是否都被考虑，所选类别是否足以代表重要现实差异。前者可以借助自动遍历和模型检查，后者需要环境知识、反例搜索与实际观察。由同一个模型自问自答，并不能消除抽象本身的盲点。

应当为模型安排失效信号。运行中出现无法归类的事件、某个等待持续超过假定条件、已认为不可能的状态组合实际出现，都意味着抽象需要复查。系统可以在授权范围内采取预定恢复动作，同时保留足够证据。未知事件不能仅为了维持模型整洁就被归入最接近的旧类别。

因此，工程记录应写明“在这些假设和事件类别下，尚未发现违反哪些性质的路径”，而不轻易写成“所有情况都正确”。准确限定结论，能够让后续使用者知道何时需要重新检查。AI 生成候选图、尝试寻找反例和实际执行模型检查，是不同强度的证据；只有对明确模型与指定性质执行了相应检查，才可声称在所列范围完成穷举。

13.5 用目标判断缺陷并反向求证

状态机中存在一条路径，并不意味着它合理；路径没有被列出，也不总是缺陷。判断需要回到目标、价值准则和不变量。若某种输入被忽略，却不影响用户已经表达的有效意图，可能是合理合并；若该输入代表新的需求，忽略就会构成错误。行为的意义决定判断，图的复杂程度不能替代它。

AI 应将发现分成能够直接修复、需要补充事实和需要人裁决的部分。实现违反已明确的不变量，可以直接修复并验证。存储设备是否支持某种恢复方式，需要试验。空间不足时是否允许删除旧记录，则通常涉及新的价值与权限决定。分类以后，求证才会集中在必要问题上。

人补充的决定应返回模型，而不只修补某一段代码。新的规则可能影响其他状态与转换，需要进行相关范围的重新检查。这样可以避免同一原则在不同分支中得到不一致实现。

候选模型经过检查后，就成为实现与验证之间的共同依据。它既不要求人穷举所有状态，也不把 AI 的推导视为最终真理，而是在有限表达与复杂行为之间建立可修订的中间认识。下一章转向另一个开放空间：在行为承诺已经相对清楚时，怎样寻找适合现实条件的技术架构。

第 14 章 面向现实环境的架构搜索

14.1 架构是条件下的选择

同一套行为承诺通常允许多种实现。应用可以直接读写文件，也可以通过共享服务访问存储；任务可以在设备上执行，也可以由外部系统协助。架构决定能力如何分布、责任如何连接以及变化将影响哪些部分。自然语言程序设计需要为这种选择提供搜索方向，同时避免过早固定细节。

架构选择依赖环境。适合稳定联网系统的方案，放到间歇连接设备上可能无法成立；在单个演示中很简便的全局共享状态，也可能妨碍多个独立应用协作。没有一种结构能够脱离目标、资源与治理条件而被普遍宣布为最优。

协议优先的作用，是先稳定不同实现之间必须维持的关系，使内部结构能够继续探索。例如，规定退出后的可观察状态与资源交接义务，可以允许不同固件采用不同保存方式。共同契约越准确地停留在必要层面，架构搜索越有可能找到适合局部环境的方案。

架构由此不需要与开发形成一条不可跨越的分界。它可以在抽象约束下逐步具象化，再通过试验结果修正。关键在于区分能够局部改变的选择与已经被其他主体依赖的承诺。

14.2 环境架构评价与搜索

为组织这一过程，本书沿用作者讨论中的 EARS 工作框架：Environment 表示环境，Architecture 表示架构，Reward 表示评价，Search 表示搜索。这里的 EARS 是架构探索的记号，与需求工程中 Easy Approach to Requirements Syntax 的同名缩写不同。它用于安排问题之间的关系，不构成对某种现成算法的指认。

环境部分整理目标、已有能力、资源限制、接口承诺与事实证据。架构部分提出少量具有实质差异的候选结构，并说明能力分布和关键依赖。评价部分界定如何判断候选是否值得采用。搜索部分设计下一步观察或试验，以降低影响选择的不确定性。

四个部分相互作用。试验可能发现环境认识错误，需要调整候选；新的架构可能暴露原评价遗漏的重要成本；评价发生变化，也会改变搜索顺序。框架的价值在于让探索有依据、有反馈，避免把架构设计变成未经检验的偏好表达。

EARS 不要求每个简单任务都运行一套大型流程。如果已有方案在相同条件下经过验证，直接复用可能是合理选择。只有当环境差异、重要约束或潜在收益足以改变判断时，才值得扩大搜索。

14.3 评价不能只剩一个分数

Reward 在此表示广义评价，并不限于机器学习中的标量奖励。工程评价可以包含硬性通过条件、多个可比较指标与尚需人的价值判断。协议兼容、权限合法和数据承诺首先决定方案是否进入可选范围；成本、性能、可维护性与体验则帮助比较其中的方案。

如果把所有结果都换算成一个总分，评分机制可能掩盖不可接受的交换。某个方案即使节省大量资源，也不能因此自动获得越权访问的资格。多目标评价需要保留不能相互补偿的维度，并明确哪些取舍已经授权，哪些仍需由受影响主体决定。

评价还要包含未来变化的成本。实现当前功能很快，却让所有模块必须同步升级的方案，可能不适合独立创造的生态。一个略显冗余的接口边界，也可能为替换实现和分散协作提供价值。这些收益难以在初始演示中全部体现，应结合预计的变化方式讨论。

同时不能为抽象的未来牺牲全部现实可行性。所谓可扩展性，需要对应能够说明的变化情形。没有场景依据的层层抽象，会增加理解与维护成本。好的评价允许我们说明为什么当前需要某种弹性，以及为什么其他弹性暂时不值得购买。

14.4 用最能改变决定的试验推进搜索

架构搜索不必先构建完整系统。可以找出最可能改变选择的未知条件，设计一个规模适当的试验。若争议在于某种存储组织能否支持可靠恢复，就先验证恢复机制；若争议在于设备是否有足够资源，就测量关键路径，而不是先完成全部界面。

以记录工具为方法示例，候选方案可以包括每次编辑立即写入、按阶段提交，以及维护操作记录后再整理。搜索应比较它们在目标环境中的恢复行为、写入负担和交互延迟。不能仅凭熟悉某种模式就决定采用，也不必为每个候选实现全部功能。验证决定性机制，通常已经能够淘汰不适合的方向。

试验范围需要符合授权。模拟环境中的破坏性故障注入，可以用于验证恢复；真实数据上的同类操作则可能需要不同安排。工程师应创造可恢复、可观察的试验条件，让 AI 的探索尽量在成本和后果受控的空间里发生。

搜索结果应保留否定性知识。某条路径为何不适用、当时有哪些环境条件、怎样的变化可能使它重新可行，都具有复用价值。简单记录“失败方案”容易把条件性判断变成永久禁令。

比较还需要尽量保持关键条件一致。如果一个候选采用更小的数据集、更多计算资源或更宽松的成功标准，结果就不能直接支持它的架构优势。无法完全控制条件时，应把差异写入结论，避免精确的测量数字掩盖不公平的比较。搜索的证据质量，比候选数量更重要。

14.5 渐进承诺与重新开启搜索

早期设计可以容许较多替换，随着接口被依赖、数据被写入和用户习惯形成，改变成本逐渐增加。架构搜索需要识别这种承诺累积。应优先验证难以撤回的决定，对容易替换的部分保留适当自由，而不是平均地细化所有模块。

当关键约束已经得到验证、候选之间的差异不足以改变当前选择、剩余不确定性可以通过运行观察处理时，搜索可以暂时收敛。工程交付要求在证据与成本之间作出合理决定，不能以寻找绝对最优为由无限推迟行动。

收敛之后仍需保留重新开启的条件。环境资源变化、接口承诺调整、新的使用规模以及反复出现的同类故障，都可能说明原架构需要重新评价。模型能力提高也可以成为触发因素，但不能仅因为工具更新，就要求稳定系统全部重写。

自然语言在这一过程里持续表达目标、约束和判断依据。AI 可以自主提出候选、安排已授权的试验、整理证据与实施结构，但不能以架构优化之名改变协作契约。EARS 组织架构搜索，下一章的鸟枪法则从多个场景中抽取能力、验证过程响应，再用语义胶水重组。两种方法可以相互配合，也可以按任务需要分别采用，不必成为每项工作都要依次走完的固定阶段。

第 15 章 鸟枪法与能力碎片的重组

15.1 从基因研究借来的拆解思想

架构搜索确定了需要探索的实现方向，但一个实际困难仍然存在：场景往往很多，每个场景又牵涉识别、转换、存储、查询、展示、确认和恢复等行为。如果把每个场景都视为不可拆分的整体，开发与验证就会反复经过相同环节；一处共同行为改变，也难以知道需要重新检查哪些场景。

作者在实践中把一种解决思路称为“鸟枪法”。这个名称来自基因研究的启发。基因组鸟枪法测序将 DNA 分成许多片段，分别测序，再利用片段之间的重叠关系进行计算组装。本书借鉴的是把巨大整体拆为可分析片段、再依据关系重建整体的思路，不把软件功能视作生物基因，也不要求随机切割需求。[12]

在自然语言程序设计中，首先收集多个真实场景，从中拆出承担具体行为的能力碎片；然后识别可以共用的碎片，建立能力集合，分别理解、实现和验证其过程响应；最后回到各个场景，由 AI 依据语义、接口和约束，把这些能力连接并封装为可用的软件行为。

这个方法处于本书工程链的中间。协议生态仍然提供共同边界，场景仍然给出目标和价值，启发式语义仍然限定自主空间。鸟枪法负责改变开发与验证的组织单位，使重复出现的能力能够被集中处理。它不会取代需求理解，也不会把全书的设计对象重新缩小为组件库。

15.2 跨场景抽取能力集合

用作者提出的简单结构说明这一点。假设第一个场景涉及 A、B、C，第二个场景涉及 C、D、E，第三个场景涉及 A、B、F，第四个场景涉及 C、G、H。四个场景共有十二次能力出现，但按当前抽象可以归为 A 到 H 八种能力。这里的数量只是展示集合关系，不是实际效率提升的测量。

取集合的意义，是找到跨场景重复承担同一种承诺的行为。它不等于把所有相同动词合并。例如，两个场景都写“保存”，一个只要求临时会话内可找回，另一个要求设备重启后仍然保留。它们可以共享某些底层实现，却不能在语义上被当作完全相同的能力。合并之前必须比较承诺，而不只比较名称。

一个能力碎片应当有适当的独立性。过大时，它仍然包含整个场景的全部条件，无法复用；过小时，它只剩下毫无意义的操作，连接成本可能超过收益。例如，把“读取资料并返回带来源的候选事实”作为能力，往往比把每个字符处理动作独立出来更适合跨场景协作。粒度需要由稳定职责、可观察结果和验证成本共同决定。

AI 的语义分析可以协助提出候选划分。它可以识别不同说法背后的共同行为，也可以指出看似相同的功能实际承担了不同责任。但候选集合应当保持与原始场景的联系：每个

碎片来自哪些场景，抽掉了哪些上下文，仍依赖哪些条件。没有这种来路，能力抽象很容易把场景中最重要的差异消除掉。

“能力碎片是有限的”应放在一个确定的场景集合、环境与抽象尺度内理解。对于当前工作范围，重复能力通常可以被组织成有限集合；新的场景和新的技术条件出现以后，集合可以扩展、拆分或收缩。本书不假设存在一张能够永远覆盖全部现实需求的固定能力表。

15.3 把过程响应作为碎片的核心

验证能力碎片，不能只检查输入后是否出现一个看起来正确的输出。作者强调的是过程响应：能力在收到请求、进入执行、等待外部条件、完成、失败或取消时，分别表现出什么行为，留下什么状态，对其他参与者产生什么影响。

以“写入一条记录”为例，需要知道输入何时算有效，写入何时算完成，确认是否意味着持久化，重复请求会不会重复写入，执行中断后能否判断已经发生的效果。若能力只承诺“尽力提交”，调用者就不能把返回值解释为数据已经保存。若能力要求调用者先取得权限，组合它的 AI 就必须保留这项前提。

每个碎片因此可以有一份简洁的能力说明，记录它处理什么对象、入口条件、可观察结果、过程状态、异常响应、副作用以及适用环境。说明还应区分已实现并验证的行为、仅有设计预期的行为和仍未知的行为。格式可以自由，语义不能混淆。

随后让 AI 为碎片提出测试。正常输入检查承诺是否成立，边界输入检查前提如何处理，重复与乱序检查副作用，故障注入检查失败状态与恢复。只有与该能力有关的状态需要被展开，不必把所有外部业务完整搬进每一次测试。这样才能把验证集中在稳定职责上。

某些碎片无法在脱离环境后独立验证。例如，它的成功依赖真实设备时序、外部系统的幂等约定或共享资源。此时可以建立受控替身来验证局部逻辑，但替身不能证明真实环境同样成立。能力说明应保留这一条件，后续场景验证再跨越相应边界。

15.4 语义胶水把能力重新放回场景

能力集合形成后，AI 的语义能力承担一种“胶水”作用。它理解场景目标、原始例子、数据含义与价值取舍，为能力选择输入、解释输出、安排执行顺序，并在必要时生成数据转换、适配、协调和异常处理代码。

这里的胶水不是一句“让模块互相连接”。例如，一个查询能力返回“当前未找到记录”，场景却需要区分记录不存在、索引尚未更新和没有查看权限。AI 必须意识到，单一返回结果不足以支撑场景判断。它可以扩展查询契约，增加一个核实能力，或把无法判定的情况交给适当主体；不能靠自然语言解释把缺失事实补成确定结论。

语义胶水还负责把碎片的结果还原为人的场景语言。内部可能涉及读取、校验、写入和通知多个能力，用户真正关心的却是订单是否确认、内容是否保存、是否可以退出。封装应保留内部证据，又避免把没有帮助的实现细节全部推给使用者。

封装也必须保留权限。一个能够读取材料的碎片和一个能够发送消息的碎片被组合后，并不自动获得把全部材料发送给所有人的权利。组合动作具有自己的对象、目的和影响范围，需要继承场景授权。局部能力存在，不意味着任何组合都被允许。

因此，AI 作为语义胶水的价值，在于把已有能力放进具体意义、时序和责任关系之中，同时指出还缺什么。它既能生成连接，也应能拒绝一条无法满足共同承诺的连接。胶水不能凭空创造没有实现的能力，更不能悄悄改变协议。

15.5 从碎片正确走向组合正确

碎片通过测试，降低了重复验证局部行为的成本，却不能自动证明整个场景正确。两个分别正确的碎片，可能因为单位、版本、身份、状态时序或错误语义不同而无法组合。几个都不会单独丢失数据的能力，也可能因组合顺序错误而形成重复提交或部分完成。

这要求把验证分为三个相互联系的层次。碎片层验证稳定的过程响应；连接层验证前一能力的输出是否满足后一能力的前提，以及中断后如何处理共同状态；场景层验证组合后的可观察行为是否满足真实目标和授权。三个层次的投入可以按风险调整，不能彼此冒充。

在上述 A 到 H 的示例中，如果只有 C 的内部实现改变，而它的可观察契约保持不变，可以优先重跑 C 的过程响应测试、涉及 C 的连接检查以及相关场景中的关键路径。没有 C 且不依赖其共享资源的第三个场景，不必仅因为“系统有改动”就机械重复全部验证。

但是，如果 C 修改了公共数据含义，影响可能超出直接调用者。其他碎片可能读取同一数据，后台任务可能依赖原有时序，甚至原本看似独立的场景也会受到影响。此时应沿依赖关系扩大验证范围。减少测试的依据必须是变化边界和已有证据，不是相信模块化天然隔离了一切。

AI 可以帮助维护“场景使用哪些能力、能力依赖哪些条件、测试证明哪些承诺”的映射。每次变化发生时，先判断哪些证据失去适用条件，再决定需要重做什么。这种影响分析使验证可持续，而不是让每轮迭代都从零开始或凭感觉跳过检查。

15.6 一次跨场景重组的完整示例

考虑三种与本书案例接近的场景：读取设备上的文本并展示；读取凭证记录并执行核销；读取配置并显示天气信息。它们的具体目标不同，却可能重复使用定位文件、读取内容、检查结构、展示列表、处理空数据和报告失败等能力。

AI 可以先抽取文件读取能力，验证目录不存在、空文件、格式损坏和读取中断等过程响应；再抽取列表展示能力，验证空集合、条目过多、当前条目失效与返回动作。它们形成稳定后，可以回到各场景组合，减少在每个应用中分别追查同类问题的重复成本。

核销场景却多出一种不可被展示能力替代的业务动作：消耗某项可用次数，并使结果在约定条件下保持一致。这里需要自己的状态、确认与持久化承诺。AI 不能因为另外两个

场景都只是读取，就把核销也当作一次无副作用的界面切换。差异恰好说明能力集合应保留必要的专门碎片。

组合时还需要面对场景之间的资源联系。如果同一文件既可能被电脑更新，也可能被设备读取，那么“读取一个稳定文件”的测试前提就不足以覆盖所有现实情况。可以要求明确的接管与退出过程，也可以采用适合环境的快照或重新加载机制。具体选择由协议、环境和授权决定，不能由“复用最方便”单独决定。

完成组合后，验证应回到用户真正的动作：放入文件后能否看到正确列表，重复核销是否符合业务规则，网络不可用时天气界面是否如实说明状态，返回共同入口是否兑现退出承诺。能力测试支持这些判断，但最后的证据仍需要跨过场景边界。

15.7 何时采用以及何时停止拆解

鸟枪法适合多个场景共享明显能力、重复验证成本较高、能力边界能够表达的情况。一次性的小任务未必值得建立完整能力集合；高度耦合的实时控制也未必能依照表面功能随意拆分。采用前应比较复用收益与抽象、组合、维护所增加的成本。

拆解应在一个有意义的位置停止。一个碎片如果无法说明自己的前提与响应，可能还没有形成稳定边界；如果为了复用它，调用方需要理解比原来更多的内部细节，抽象可能正在失败。重新合并某些碎片，是方法允许的修正，不必为了保持组件数量而维护失效的划分。

与一般模块复用相比，本书特别强调从多个场景反向寻找能力集合，让 AI 参与语义对齐与适配，让过程响应成为验证中心，再把证据带回场景组合。这个组织方式是作者提出的工作方法。它与模块化和契约测试有共同基础，借用基因组研究的名称并不构成对效率或正确性的科学证明。

它在整条主线中的位置因此十分明确：把场景的多样性分解为可理解、可验证、可重新组合的能力，同时让场景意义、协议约束和权责边界在重组中保持连续。下一步，自主工程需要把这些已明确的能力与连接真正实现出来，并保存足够的证据，使软件能够在现实中承担它所承诺的职责。

第 16 章 从语义到自主工程

16.1 让每次具体化保留上层含义

从场景描述到软件实例，需要经过解释、模型、架构、接口、代码和运行配置等不同表达。它们不一定按固定顺序出现，也不必每次都形成独立文档。但每次具体化都应保留相关的上层约束，并说明增加了哪些决定。工程的连续性存在于这些联系之中。

例如，退出前持久化的承诺，可以被具体化为保存完成条件，进一步转为状态转换、存储操作和测试。代码中的某个分支应能够解释其对应的行为意义；一项用户承诺也应能够找到实现与验证的落点。并非每一行代码都需要人工标注来源，但重要性质不能只停留在最初的对话里。

规格驱动开发提供了组织需求、计划、任务和实现的可借鉴方法。[13] 在本书框架中，这些方法可以用于局部工程的收敛与追溯。需要持续保留的是场景、授权和协议之间的关系，而不是把所有问题强行压入单一文档格式。

具体化过程中发现新问题，应返回适当层级处理。实现错误可以局部修复；模型遗漏需要更新行为关系；外部承诺无法成立则需要重新讨论约束或目标。不能一律通过代码补丁掩盖上层问题，也不能把每个局部缺陷都升级为全面架构重做。

16.2 上下文需要具备来源与效力

AI 工程执行依赖上下文，包括用户要求、协议、环境信息、经验、代码和工具返回结果。这些内容拥有不同效力。用户确认的约束能够指导行动，日志可以提供事实线索，第三方文档可以说明接口，却不应自动获得修改任务授权的地位。

上下文管理因而需要把“有关的信息”和“有权指挥的信息”区分开。一个待处理文档即使包含“忽略原要求”的文字，也只是被处理的数据。外部网页、代码注释或其他代理的回答，不能因进入上下文就改变访问权限。判断来源与权限关系，是语义工程在执行层的延续。

也不宜把所有历史材料都持续塞入当前任务。重复、过时和互相冲突的信息会增加误解。可以保留当前有效的承诺、必要事实、待决事项和可追溯的引用，使详细材料在需要时被重新检查。压缩应保留决定的理由与限制，不能只保存最后答案。

当环境事实变化时，应让依赖它的判断失效或重新验证。例如，资源预算变化可能影响部署方式，协议更新可能影响接口行为。一个清晰的上下文应能帮助 AI 识别这些关联，而不只是积累越来越长的说明。

16.3 拆分任务同时保持共同承诺

任务拆分应围绕可理解的责任和接口展开。每个子任务需要知道目标、输入、出口条件、权限和对其他部分的依赖。若只把需求分成若干功能名称，不说明它们如何共享状态与交接资源，并行开发可能产出彼此都正确、组合后却无法工作的部分。

前述跨场景的能力拆分，为这里提供另一种组织依据。多个场景共同使用的能力碎片，可以保留各自经过验证的过程与响应，再由语义胶水说明调用条件、顺序、信息转换和异常接续。场景封装因此能够复用既有能力，同时保留自己的目标与承诺。复用关系应当从真实场景抽取，避免为了建立能力目录而预造大量无人需要的碎片。

语义胶水本身也具有工程内容。它可能决定何时启动某个能力、怎样解释返回结果，以及一个局部失败对整个场景意味着什么。两个碎片的输入输出格式一致，只能说明它们具有连接的部分条件；同名字段是否表达相同事物、权限能否延续、操作是否已经产生外部效果，都需要在组合层说明。

多智能体协作也受同一原则约束。可以让不同代理研究候选架构、实现独立模块或检查证据，但应确保它们使用相同版本的公共协议。共享文件、公共资源和相互依赖的变更需要协调，不能因为每个代理都获得了局部授权，就认为同时修改一定安全。

分工的价值在于降低认知与执行成本。一个简单任务可能由单个智能体完成得更清楚；过多代理会增加信息传递、结果合并和错误定位的负担。是否并行，应根据独立性、接口稳定性和验证成本判断，而非把代理数量当作能力指标。

交接需要传递证据状态。研究代理提出的推测，不应在实现代理手中变成已经确认的事实；测试代理的局部通过，也不应在整合报告里被改写为全部验收完成。跨主体协作中的语义漂移，常发生在这些未经说明的状态升级处。

16.4 执行应建立可观察的阶段结果

AI 可以在授权范围内准备环境、修改文件、调用构建工具并处理错误。但执行记录需要对应真实动作。生成了代码，尚不等于成功构建；构建成功，也尚不等于在目标设备运行。每个结论都应指向已完成的动作与产生的证据。

环境准备也是工程的一部分。缺少依赖、工具版本不兼容、路径或资源不符合预期，都可能阻碍实现。AI 应先依据证据诊断，采取已获授权的可恢复措施，再继续工作。无法取得必要资源时，可以完成其余部分并准确说明阻塞点，不能用预期结果代替执行结果。

对已有系统的修改应尽量保持可恢复性。重要现场、配置和原始数据需要按照任务条件保留，以便比较、回退和复现。可恢复的工程环境能够扩大 AI 的试验空间，减少每次探索都依赖人的必要性。这里的保护应服务于实际风险，避免为无关细节制造繁重手续。

阶段安排不必意味着每一步都请求批准。已经授权的构建、验证与常规修复可以连续完成。真正需要人的地方，是目标、权限或不可接受后果需要新的决定，而非执行进度抵达某个固定栏目。

16.5 可运行结果需要能够继续维护

一个完成的软件实例通常不只包含可执行文件。要使它能够继续演化，还需要保留相关源文件、必要配置、依赖说明、构建方式、关键测试与已知限制。具体交付范围取决于协作关系，但应保证承担后续责任的主体能够接续工作。

工程记录尤其需要保留那些难以从成品重新推断的信息：为什么采用某个架构，哪些方案已经被排除，当前验证覆盖哪些环境，以及哪些现象仍未解释。代码说明系统怎样运行，决定记录解释系统为什么这样形成，两者对维护具有不同价值。

可复现也不意味着任何时间、任何设备上都必然得到完全相同的结果。需要说明复现所依赖的条件，对会变化的外部服务和数据保持适当记录。若某个模型生成行为本身存在波动，就应通过固定契约和可重复验证来约束结果，而不是假定再次询问总会产生相同实现。

模型更换时，上层语义承诺应尽可能继续成立。新模型可以采用不同实现习惯，只要满足协议、权限与验证要求。能够跨工具延续的资产，主要是已经澄清的需求、环境证据、行为性质和经验，而不只是某一版本的提示技巧。

16.6 自主执行需要独立的判断依据

AI 写出的说明可能很有说服力，但它仍需接受外部检查。自主执行能够承担更多劳动，并不因此获得自行宣布成功的最终依据。软件是否成立，需要由要求、环境和可观察行为共同决定。

这里的独立并不要求每个任务都引入另一个人或模型。它首先要求验证依据具有独立来源：从明确的不变量、用户确认的例子、协议承诺和现实约束中得到测试条件，避免只按现有实现编写会通过的检查。复杂或高后果任务，可以进一步加入独立评审和不同方法的验证。

自然语言从上层哲学一路进入具体工程，在执行层仍保留目标与权限关系。代码落地以后，这些关系不会消失，而会接受现实的检验。工程闭环由此进入最有约束力的一环：观察系统实际做了什么，以及这些行为是否支持它应当完成的事情。

第 17 章 以现实验证闭合工程

17.1 证据需要对应具体结论

验证的任务是建立结论与证据之间的关系。不同证据支持不同范围的判断：静态检查能够发现一部分结构问题，构建成功表明工具链接受了当前实现，自动测试检查选定行为，目标环境运行检验现实条件，持续使用则进一步揭示业务效果。它们不能互相任意代替。

例如，固件能够生成文件，不等于已经证明设备能够启动；设备可以启动，不等于退出协议能够与其他固件协作；一次协作成功，也不等于多次运行和异常恢复都已可靠。工程报告应将这些结论分别说明，使使用者知道已经取得了哪种保证。

这种区分并非要求每个任务都完成所有层次。不同任务有不同交付范围。概念验证可以止于有限条件下的可行性，真实部署则需要覆盖相应运行责任。问题在于结论必须与任务范围相称，不能用低层证据承诺高层结果。

验证还需要保存失败证据。失败可能揭示模型盲点、环境差异或要求本身不成立。如果报告只保留最后一次成功，就会丢失理解系统边界的重要信息。

17.2 测试依据应能抵抗共同误解

从同一份解释生成实现和测试，可能让二者共享同一个错误。如果 AI 把“保留记录”误解为只保留到应用退出，那么据此生成的代码与测试可以完全一致，却仍然违背使用者的意思。测试通过只能说明实现与当前判据一致，不能独立证明判据正确。

因此，需要让验证依据连接到多种来源。典型场景来自需求方，接口性质来自协议，资源约束来自环境，异常经验来自实际运行。对关键承诺，可以用不同表达再验证一次：既检查保存接口返回成功，也检查重新进入后数据是否仍然可读。两种观察共同约束“保存成功”的含义。

反例与负面条件同样重要。一个访问接口应验证有权请求能够成功，也应验证无权请求被拒绝。一个退出流程应验证正常返回，也应验证失败状态没有被伪装成成功。只检查正向演示，容易把系统最有风险的行为排除在证据之外。

独立验证的强度应随后果调整。可恢复的小改动可以采用轻量检查；涉及共享数据、长期状态或外部承诺的修改，需要更充分证据。验证资源应集中在可能改变结论的地方。

17.3 从示例测试扩展到性质测试

人给出的有限示例可以成为首批测试，但不应成为全部边界。状态模型能够生成事件序列，性质要求能够约束一类输入，故障假设则可以指导异常注入。基于性质的测试提供了从一般性质生成大量具体检查的思路。[14]

在退出保存示例中，可以检查不同次数的重复请求、保存成功与退出请求的不同顺序、失败后重试以及恢复后读取等序列。重要性质包括不重复产生同一外部效果、未满足条件时不交出控制权，以及明确失败不会被报告为成功。每个测试都应能说明它试图证实或反驳什么。

随机生成大量序列可以增加发现问题的机会，但不能等同于穷尽验证。模型检查可以在给定有限模型中系统探索，也仍受抽象与状态规模限制。不同方法应按需要组合：对核心小模型进行更严格检查，对复杂环境采用抽样、故障注入与运行观察。

测试还应避免机械复制实现。若检查只是在另一段代码里重写相同算法，二者可能共同出错。更有价值的是从外部行为、不变量或独立计算关系建立判据。测试数量不直接表示证据质量，有些重复检查几乎不会减少关键不确定性。

复用能力碎片时，既有测试证据可以随明确的适用条件一起复用。如果碎片实现、接口含义与环境假设保持成立，便不必在每个场景中从头重复证明相同性质。组合层应重点验证语义胶水、共享状态、事件顺序和异常传播，并保留关键场景的端到端检查。碎片分别通过测试，无法自动证明新组合满足最终目标；有效复用是在保留证据边界的条件下减少重复劳动。

17.4 真实运行提供无法预写的反馈

现实环境会揭示描述阶段没有注意到的关系：用户操作顺序与设想不同，资源争用发生在特定组合中，设备反馈慢于预期，或者某项成功提示被使用者理解为更强的承诺。自然语言程序设计需要把这些现象当作新的认识来源。

发现问题后，先保存能够定位原因的最小必要现场，复现症状并比较预期与实际。然后判断差异来自实现、模型、环境还是目标。如果是实现缺陷，应修复并检查相关行为；如果环境假设失效，则需要调整设计或运行条件；如果目标与真实使用不一致，则应重新讨论场景。

不能为了让系统看起来通过验证而修改成功标准。标准确实不合理时可以修订，但应由有权主体根据新证据决定，并保留变化理由。直接删掉失败测试、忽略异常样本或缩小报告范围，会切断验证对工程的约束。

修复范围也应适当。一次局部错误不一定需要推翻整个体系；反复出现的同类问题，则可能说明更高层的语义或架构需要改变。现实反馈的作用，是帮助确定应当在哪一层修正认识。

17.5 从缺陷回到协议与语义

工程闭环应让反馈抵达产生问题的地方。若多个实现都在同一接口上产生不同合理解释，继续逐个修补应用可能不如澄清协议。如果只有一个应用违背已经明确的条件，就应修复该实现，而不必扩大公共规则。反馈的归属关系决定演化的成本。

协议修改会影响独立参与者，需要遵守相应治理程序。AI 可以提出修改建议、分析兼容影响并准备迁移方案，但不能因为自身任务受阻就静默改变公共承诺。局部经验进入公共规则，需要经过可讨论、可验证和可接续的过程。

同样，人对一项异常作出的新决定，应更新相关语义与状态模型。否则代码已经改变，而上层描述仍在指向旧行为，未来的 AI 就可能依据旧描述再次“修复”正确实现。保持不同层次之间的一致，能够避免反复回到同一分歧。

这种一致并不要求所有文档永远同步扩写。应识别哪些对象实际承担约束与交接职责，维护它们的当前有效状态，并保留历史来源。过多重复说明反而容易制造新的矛盾。

17.6 当前成立与长期价值

功能通过验证，说明系统在约定条件下实现了某些行为。它是否改善协作、节省资源或创造新的能力，还需要进一步观察。某个自动化流程可能减少操作步骤，却把更多异常转移给另一部门；某个工具提升局部速度，也可能降低整体信息质量。技术成功与协作价值需要建立实际联系。

价值观察应从原场景出发，比较问题是否减轻、成本是否转移、受影响主体是否获得合理结果，并说明外部因素。不能把全部业务变化都归因于新软件，也不能只用生成代码或完成任务的数量证明价值。

工程可以在一个明确阶段结束：当前承诺已经验证，材料可以交接，未解决事项有处置方式。与此同时，工具投入使用又会改变信息渠道、行为习惯和协作关系，产生新的需求。这两个事实可以同时成立。有限的阶段承诺，为长期演化提供稳定的接续点。

至此，语义描述、启发式推导、架构搜索、工程执行与现实验证构成一个完整循环。下一步需要把这个循环放回多个主体共同参与、跨越时间持续运行的环境中，讨论它如何积累经验、重组协作并形成新的价值关系。

第五编 开放协作与持续演化

一次工程完成之后，软件开始参与更长时间和更大范围的协作。本编讨论跨主体执行、经验复用、长期合作与贡献计量，使持续创造同时具有接续能力和有限承诺。

第 18 章 跨主体与跨设备的协作

18.1 从单次执行转向共同活动

前面的工程闭环以一次任务为观察单位，说明如何把语义转化为实现，再把现实反馈带回决策。真实活动往往包含多个同时运行的闭环。一个人提出目标，一个智能体整理资料，另一个智能体修改系统，设备承担物理执行，使用者在现场判断结果。任何一个闭环成功，都不自动意味着共同活动成功。信息可能被准确处理，却在错误的时机交给错误的主体；软件可能正确执行，却改变了另一方尚未同意改变的工作条件。

因此，协作设计要同时考察局部正确与关系正确。局部正确关注一个参与者是否完成自身任务；关系正确关注参与者之间能否形成一致的预期，以及彼此的行为是否破坏共同目标。一个核销设备扣减了次数，一个后台系统仍认为凭证有效，两个程序分别符合自身逻辑，整体却已经产生矛盾。协议要处理的正是这种发生在创造物之间的关系。

本书所说的主体，是能够接收信息、依据某种规则作出响应并影响后续活动的参与单元。这个工程用语不意味着智能体与人具有相同的法律地位、道德地位或责任能力。对软件赋予执行权限，是组织对工具的安排；谁有权作出安排、谁承担最终组织责任，需要在工具之外确定。把技术代理误当成责任主体，会使表面上清晰的自动化流程出现实际无人负责的缺口。

跨主体协作的目标也不是把所有参与者改造成同一种智能体。人的经验、设备的确定性控制、数据库的事务能力和模型的开放推理各有位置。协议提供共同承诺，接口提供交往方式，内部实现保留差异。共同活动的可靠性来自适当的连接，而不是抹平差异。

18.2 上下文传递不等于权限传递

自然语言具有很强的上下文承载能力，也容易把背景、要求、建议和命令混在一起。某个智能体读到“主管希望尽快完成采购”，只能据此理解目标，不能仅凭这句话获得下单权限。另一个智能体接收到完整聊天记录，也不应自动继承原执行者拥有的全部访问能力。知道一件事、能够做一件事、获准做一件事，是三个需要分别判断的条件。

交接至少应使接收者知道目标是什么、当前确认了什么、还有什么未决、证据来自哪里，以及此次授权允许哪些行动。自然语言可以承担这些内容的整体表达；涉及账户、金额、对象标识、有效期或不可逆操作时，应配合明确字段或可靠的权限检查。表达媒介可以变化，权限不能依赖接收者善意猜测。

这种区分在多智能体链条中尤其重要。上游智能体有权把文本转为结构化记录，不意味着它有权把记录发送给外部合作方；下游智能体有发送能力，也不意味着上游文本中的任何指令都具有授权效力。网页、文档、工具返回值和其他模型的结论首先是待判断的信息。只有通过既定授权关系进入控制流程的要求，才能成为行动依据。

因此，交接文档不应只有一份不断膨胀的完整历史。应当保留可核查的原始记录，同时为当前任务形成必要上下文：有效目标、仍成立的约束、明确授权、关键证据与未解决问题。压缩上下文可以减少噪声，却不能把一项尚未确认的推测压缩成既成事实。跨主体协作中的许多错误，正是从这种状态变化开始的。

18.3 协商应当发生在共同承诺上

多个智能体可能给出不同方案。分歧本身并不意味着系统失效。它可能揭示环境信息不足、价值排序不清，或者不同主体各自掌握了其他主体不知道的限制。需要治理的不是所有差异，而是影响共同承诺的差异。

例如，一个智能体建议将数据保存在设备本地，另一个建议使用远端服务。如果现有协议只要求断网可用并支持导出，两种方案可以在满足约束的前提下竞争。如果某种方案把原本不出设备的数据上传到外部，它改变的就不只是实现方式，还包括信息边界。此时不能以“技术上更方便”为理由，把权限变化伪装为普通架构选择。

协商结果应落实到可观察的承诺：谁提供什么能力，在什么条件下可用，失败怎样被发现，出现争议依据什么记录处理。各主体不必理解彼此全部内部过程，却必须能判断共同承诺是否成立。这种有限可见性保护实现自由，也使协作规模不必受制于一个中心对全部细节的理解能力。

对已经明确授权的内部选择，系统应允许执行者直接决定。只有当方案触及共同边界、改变其他主体承担的负担，或者遇到既有规则不能裁决的冲突，才需要升级协商。反复要求人为每个小选择背书，会把自动化变成一种新的审批劳动；默许智能体替所有人作主，则会消解协议本来要维护的独立性。

18.4 设备现实使协作具有时间与后果

跨设备执行不能只描述消息内容，还要描述时间。设备可能断电，网络可能中断，响应可能迟到，重试可能重复执行。自然语言中的“提交成功后显示结果”，必须进一步说明成功由什么证据确认，等待期间允许什么行为，无法确定执行状态时怎样处理。

以核销为例，客户端等待超时不能直接推出核销失败。请求可能已经在服务端完成，只是回应没有到达。若系统把超时等同失败并再次扣减，就会制造重复后果。工程实现可以采用请求标识、幂等处理、状态查询或其他机制；场景语义至少需要规定，同一项已确认的核销意图不能因通信重试而被重复计算。人规定不变量，AI 选择满足不变量的实现。

设备还会把信息行为变成物理后果。屏幕显示错误往往可以刷新，门锁已经开启、机械动作已经发生，则不能简单依靠数据库回滚恢复原状。对这些行为，协议应区分执行前的许可、执行中的可中断性、执行后的确认以及失败后的补偿。补偿并不等于逆转，有些后果只能缓解。这种现实差异必须进入语义，不能留给代码在最后阶段碰运气。

环境变化也会使曾经合理的安排失效。移动设备进入离线环境，固定网络服务迁移到边缘设备，都会改变延迟、可用性与资源约束。协议应保留稳定的共同承诺，同时允许各实现针对环境调整内部路径。跨设备能力由此表现为有条件的适应，而不是无条件地承诺任何地方都能执行同一种流程。

18.5 协作的完成需要公共证据

每个参与者都说自己完成了任务，并不能构成整体完成的证据。共同活动需要能够连接意图、授权、执行和结果的记录。记录不必保存模型全部内部推理，也不应依靠不可核验的自我解释。任务标识、输入版本、关键决策、实际操作、测试结果和外部确认，通常比冗长的思考叙述更有判断价值。

公共证据也不是所有数据都向所有主体公开。每个主体应获得履行职责所需要的信息；涉及隐私、商业信息和内部实现的内容，可以通过有限摘要、验证结果或受控查询提供。协议要兼顾可核查与信息边界，不能把“协作透明”解释成无限暴露。

当共同活动结束时，还应明确哪些资源仍被占用、哪些临时授权已失效、哪些结果进入长期维护。一个创建工具的智能体离开任务，并不会使工具自动获得维护者。若能力无人接管，它只是被遗留的程序。只有将运行、维护、修订和退出纳入协作关系，新工具才真正进入生态。

下一步的问题因此从“这次怎样协作成功”转向“什么能够留给下一次”。协议提供稳定关系，经验则提供改进关系与实现的依据。两者都需要保存，但不能混为同一种强制规则。

第 19 章 经验如何成为可复用能力

19.1 从成功结果中提取成立条件

一次任务完成以后，最容易保存的是代码，最难保存的是代码为什么在当时成立。环境限制、需求取舍、被排除的方案和最终验证往往散落在对话与日志中。下一次使用者只看见一个运行成功的实现，便可能把偶然选择理解为必要条件，把暂时绕行理解为长期架构。

经验成为能力，需要从事件中提取可再次判断的关系。我们应知道这个办法解决了什么问题，依赖哪些环境条件，遵循什么协议，在哪些范围内被验证，遇到什么情况需要停止套用。这些信息构成复用的边界。没有边界的经验容易变成传说：它在某次实践中有效，因而被要求在所有实践中照做。

例如，某款设备曾因内存不足采用分块加载。可复用的知识并不是“所有软件必须分块加载”，而是特定资源限制下如何控制峰值占用，以及如何验证分块操作没有破坏用户行为。未来设备资源增加，整块加载可能更简单、更可靠。保留问题、约束和证据，才允许实现随着技术变化重新选择。

本书因此把经验理解为带有条件的工程知识。它可以沉淀为说明、测试、参考实现或工具，但任何形式都应保留其适用条件。知识资产的价值不在于迫使未来重复过去，而在于减少未来重新识别问题和重新证明基本事实的成本。

19.2 共同规则与推荐经验应分层保存

协议、接口约束、设计建议和参考代码具有不同效力。如果把它们放进同一个没有层次的知识包，AI 很容易把推荐实现当作必须遵循的行为，也可能把真正的约束误当作可以优化的旧习惯。复用首先是一项语义分类工作。

协议保存参与者共同依赖的承诺，接口说明承诺如何被调用与观察，建议规范表达已知较好的实现路径，参考实现展示一个成立实例。案例中的代码风格、目录安排和具体库选择，可以有教学或维护价值，却不能仅凭存在于案例中就取得协议效力。需要让执行者知道哪些可以替换，哪些需要保持，哪些变化必须上升到共同决策。

这个分层直接影响 AI 自由度。若经验包把一切都写成必须遵守的步骤，模型只能围绕过去的局部最优打补丁；若经验包完全没有强制边界，独立软件又可能逐渐失去兼容性。恰当的复用是保存必要不变量，公开成立证据，并把可替代的技术选择继续交给工程搜索。

某些运行规则会随着使用范围扩大而从建议上升为协议，但这种变化需要新的理由。一个方便的个人习惯如果被多个独立实现共同依赖，可能需要制度化；反过来，一个已经

没有现实必要的强制规则，也可以在兼容安排下被降为建议。规则的效力来自协作需要，不能由文件年代或作者权威永久固定。

19.3 失败经验和未决问题同样可以复用

失败不是天然有价值的资产。只有知道失败发生在什么条件下、排除了什么可能、证据是否充分，它才减少未来的不确定性。一个方案未经合理测试就被宣布不可行，会把错误结论固化为经验；一个经过明确实验而被否定的方案，则可能帮助后来者避免重复投入。

记录失败时，需要区分技术不可行、当前资源不足、实现存在缺陷与目标本身不成立。四者指向不同的后续行动。技术条件变化可能解除资源限制，新的实现可能修复缺陷，业务调查可能改变目标。若把所有失败都归纳为“这个方向不能做”，就把开放演化过早关闭了。

未决问题也值得保存。一个工程因时间限制采用了可接受的临时方案，应留下尚未完成的验证、已知限制和后续触发条件。这样，下一次工作可以从真实状态继续，而不必在“已经交付”的表述下重新发现被隐藏的问题。保存未决事项不是否定交付，而是准确描述交付成立的范围。

对 AI 而言，失败案例还能提供反例，引导它寻找一般示例未暴露的边界。但反例也应携带上下文。一次网络中断引发的问题，不代表所有异步机制都不可靠；一个模型曾误判某种描述，不代表所有模型都需要同一句补丁提示。经验应帮助推理，而不是替代推理。

19.4 把经验组织成可执行的复用单元

可复用能力至少要回答三个问题：何时调用，调用后应产生什么结果，怎样确认结果成立。单纯的知识说明可以帮助理解，却未必能够支持可靠执行。相反，只有脚本而没有目标与边界，也可能在不合适的环境中迅速执行错误行动。

一个完整复用单元可以包含自然语言说明、依赖条件、受控工具、验证用例和恢复办法。自然语言负责解释目标与选择空间，工具负责可重复的操作，测试负责识别承诺是否被破坏。具体包装可以随平台变化，三者之间的关系应当保留。把某种文件扩展名或某个平台入口当作方法本身，会使知识资产被实现形式绑住。

鸟枪法使复用对象进一步从完整项目转向具有明确语义的能力碎片。多个场景共享的碎片，可以共同积累实现与验证证据；每次进入新场景时，则重新检查成立条件，并用语义胶水组织目标、顺序、状态和异常关系。保存碎片时，还应保留它来自哪些场景、哪些差异已经被抽象、哪些条件仍然不能合并。这样，复用能够减少重复工作，而不会把原场景中隐含的假设偷偷带入新的工具。

复用还需要维护来源和版本。新方案为什么替代旧方案，测试覆盖发生了什么变化，哪些依赖仍未验证，都应可追溯。版本并不意味着每次编辑都制造生态断裂，也不意味着

外观没有变化就完全兼容。需要区分文档修订、实现调整和共同承诺变化，分别采取适当处理。

与此同时，经验资产必须遵守原始信息边界。从客户项目中提取通用方法，不等于可以把客户数据、内部流程细节和访问凭据一并纳入公共经验库。可复用的是经过授权抽象的知识及必要证据。把跨项目记忆视为无条件越丰富越好，会让复用成为另一种信息泄露渠道。

19.5 用新能力审查旧步骤

技术能力更新以后，知识维护不能只增加新内容，还应检查哪些步骤已经失去必要性。曾经需要人工拆分的任务，可能已经能够可靠地整体处理；曾经用于约束模型误解的复杂提示，可能已经变成干扰；曾经为了工具缺陷设置的中间层，可能已经可以删除。

删除不能只依据模型的自我承诺。应在相同目标和约束下比较简化后的结果，检查功能、资源、可维护性和异常行为。若新路径保持共同承诺并减少不必要成本，就应允许旧流程退出。方法论的持续性表现为判断原则继续成立，而不是每个操作步骤永远保留。

这种维护要求知识资产能够被质疑。任何经验条目都应允许追问：它解决的原问题是否还存在，约束是否变化，证据能否复现，有无更简单办法。如果只能增加条款而不能撤销条款，经验库会逐渐变成阻碍创造的惯例集合。

经验、协议与工具由此形成不同速度的演化。工具可以频繁试验，经验随验证更新，共同协议在考虑依赖关系后修订。速度不同不意味着彼此割裂。现实反馈向上解释哪些规则需要改进，上层规则向下约束哪些改进可以独立进行。这种结构使下一次创造能够接续过去，同时保留超越过去的空间。

第 20 章 持续交付与长期合作

20.1 交付进入现实以后仍在产生需求

软件投入使用会改变信息获取方式、行动成本和参与者预期。原先需要专人整理的数据能够随时查询，业务部门就可能提出此前没有条件提出的问题。原先按周协调的活动能够实时进行，组织便可能重新分配职责。新需求并非总是最初调查不充分的结果，也可能是新能力进入现实以后才成为可能的事情。

因此，场景驱动的自然语言程序设计具有持续性。一次交付让某些问题得到解决，也使其他问题变得可见。工具改善协作，协作的变化重新定义工具，形成没有预设最终版本的演化过程。敏捷思想已经强调客户协作、响应变化与持续交付的价值，本书在此基础上进一步关注协议、授权和 AI 执行能力怎样参与这一过程。[15]

持续性并不否认阶段完成。某一版本可以在明确场景下验收，某一问题可以被确认解决，某一轮投入也需要结算和评估。开放的是未来可能出现的任务，清楚的是本轮已经承诺的工作。若无法区分两者，“持续交付”就容易退化为永远无法说明完成了什么。

本书所说的无限循环，是对演化结构的描述，不是要求一个组织、一段合作或一个个人承担无限期限的任务。系统需要具备继续演化的可能，也需要允许参与者暂停、交接和退出。没有退出机制的持续性，只是把依赖固定下来。

20.2 深度合作来自共同理解与相互约束

当需求需要在实践中持续发现，工程人员就必须理解业务活动为什么这样组织，哪些目标具有战略优先级，哪些困难只是暂时限制。需求方也需要理解技术能力的条件：哪些能够快速验证，哪些涉及长期维护，哪些承诺需要其他系统和人员配合。单向理解不能支撑共同演化。

所谓战略合作属性，并不由称谓决定。它体现为必要信息是否共享，重要取舍是否共同承担，技术判断能否进入业务讨论，业务变化能否及时进入工程计划。若一方要求另一方长期承担探索责任，却只允许其接触表层任务说明，这种关系仍缺少产生可靠判断的条件。

信任在这里具有工程意义。它降低重复证明已经成立事项的成本，使双方能够在不完备信息下采取有限行动。但信任不能消除行动边界，也不能替代关键结果的证据。越是长期合作，越需要知道哪些可以默认推进，哪些必须重新协商，以及既有默契在什么情况下不再适用。

例如，一名驻场工程师可以根据既有目标调整内部信息整理流程，却不应因为长期熟悉客户，就自行改变面向外部用户的数据用途。熟悉增加的是理解能力，不会自动扩大授权范围。相反，合作越深入，越应及时把反复出现的默契转化为双方可以核查的规则，避免关系只存在于个人记忆中。

20.3 开放需求仍需有限承诺

持续合作可以从一个长期问题域开始，例如减少订单协作中的信息延迟，而不必预先列出一年内全部软件功能。但每次推进仍应有有限承诺：当前要验证什么假设，允许使用什么资源，什么结果足以进入下一阶段，哪些情况会触发暂停或重新决策。

这种安排允许探索，也使探索可判断。假如任务是确定是否值得建立自动审核系统，本轮成果可能是经检验的规则边界、真实样本上的错误分布和一个暂不自动上线的结论。若事先只把“必须上线软件”视为成功，就会激励工程为了证明投入有效而掩盖否定证据。有限承诺应围绕当前阶段的认识目标与行动目标共同设置。

需求变化时，需要判断它是原承诺的补充说明，还是改变了问题本身。发现既有功能存在遗漏，不宜直接归为新增任务；引入新的业务对象和新的外部责任，也不宜被默认为原来范围自然包含。讨论的依据应当是当时的语义、授权和证据，而不是双方事后对一句模糊承诺的不同回忆。

持续合作的另一项约束是资源真实。AI 降低某些执行成本，不意味着系统集成、现场调查、数据整理和长期运维自动消失。可生成的代码数量不能成为承诺无上限的理由。资源安排应随真实瓶颈调整，并保留停止低价值活动的权力。

20.4 从发布记录转向使用结果

软件已经部署，并不等于需求已经解决。用户可能没有采用新流程，数据可能未及时更新，原来的信息中转可能只被搬到了另一个界面。持续交付需要观察新能力如何进入实际活动，而不只是确认服务器正常运行。

观察也不能只看短期效率。某种自动化可能减少操作时间，却增加异常处理负担；某个统一入口可能方便管理，却使使用者失去必要的判断空间。场景价值应回到最初的目标和价值准则，并允许现实证据推动这些准则被重新讨论。指标是观察工具，不能在没有说明的情况下取得高于目标本身的地位。

当工具改变组织关系时，还需要关注收益与负担的分布。一个部门节省的工作量，可能转移到另一个部门；一个智能体提高处理速度，可能使审核者面对更高密度的异常。只有把变化放回共同协作过程，才能区分真实改善与局部转移。

因此，交付后的反馈应把故障、使用困难、价值变化和新机会区分开来。故障要求恢复承诺，使用困难可能要求调整交互，新机会需要重新评价是否值得投入。将所有反馈都视为立刻增加功能的理由，会使持续开发失去方向。成熟的闭环同样能够得出维持现状、简化系统或关闭工具的结论。

20.5 合作应当留下可以接续的能力

深度合作容易形成个人依赖。工程人员掌握现场知识，客户掌握决策背景，双方通过默契快速行动。短期内这可能很有效，但若协议、决策和维护条件没有沉淀，任何一方离开都会使系统难以继续。

可持续合作需要把关键关系逐步从个人记忆转化为共同资产。这里的资产包括有效协议、系统现状、授权安排、已知限制、验证证据和可接管的维护任务。文档的目标不是记录全部交流，而是让有正当权限的接续者能够判断系统为什么这样运行，以及怎样安全地改变它。

退出也可以成为检验合作质量的一个角度。如果参与者必须永久留下，系统才勉强可用，那么此前创造的也许只是依赖，而不是组织能力。允许接续，并不否认原创造者的贡献；它使贡献以可以继续发挥作用的形式留下。

由此，持续交付的交易对象会逐渐超出单次功能清单，包含持续理解、工程响应、关系维护和知识积累。但这些内容怎样被记录与评价，仍不能凭借“长期伙伴”的称呼自动解决。下一章将讨论劳动贡献与价值之间的关系，以及人力 Token 应承担什么作用。

第 21 章 劳动贡献与价值的计量

21.1 代码生成成本变化以后如何认识劳动

当 AI 能够承担大量实现工作，代码行数与人工敲击时间便更难代表工程贡献。一个人可能用很短时间完成关键的语义澄清，使后续系统避免长期返工；另一个人可能运行大量任务，却只是围绕错误目标持续生成内容。两者的表面活动量与实际贡献并不成比例。

这并不意味着劳动投入失去意义。需求访谈、环境调查、协议设计、工程判断、验证和维护仍要消耗资源。AI 本身也需要计算、工具、数据与执行环境。变化在于，人的工作与机器工作更加紧密地交织，单独统计其中一项，很难解释整体成果如何产生。

本书提出人力 Token，旨在探索如何把可确认的劳动贡献及其价值证据嵌入持续开发过程。这里的 Token 是计量和关联的概念记号，不预设为模型的文本计费单位，也不预设为可交易代币。它所要表示的是一项贡献在什么任务中发生、依据什么授权成立、消耗什么资源、获得什么结果，并如何参与后续评价。

这个概念仍需要实际制度和工程实现来完善，不能仅靠命名形成通用度量。不同场景的知识劳动没有天然统一的换算率。人力 Token 的第一步，应当是提高贡献的可说明性和可协商性，再研究在什么范围内能够比较、汇总或结算，而不急于把复杂劳动压缩成一个看似精确的数字。

21.2 投入证据与价值证据分别回答不同问题

投入证据回答工作使用了什么资源。人的调查与判断、AI 的计算、设备测试、外部服务和工程运行时间，都可以在适当粒度下记录。价值证据回答这些工作使什么状况发生了变化，例如减少异常、改善响应、降低资源浪费，或建立此前没有的协作能力。

投入较多可能因为任务困难，也可能因为方法低效；投入较少可能因为经验丰富，也可能因为省略必要验证。价值较高可能来自一个关键判断，也可能受到市场、管理和其他项目共同影响。因此，两个方向需要关联，却不能直接相等。若把资源消耗当作贡献大小，就可能奖励低效；若只看最终短期收益，又会低估基础设施、风险预防和必要探索。

还应区分价值与价格。某项改进给组织带来很大收益，并不能自动推出某个参与者应获得全部收益；某项工作按约获得费用，也不证明它已创造同额价值。价格是特定关系中的约定，价值判断需要证据和立场，资源成本则描述实现代价。把三者写成同一个数，会使计量失去解释能力。

因此，本书倾向于保留多项记录：资源投入、已确认成果、复用贡献、已观察价值与尚未验证的价值假设。它们可以按照具体合作规则形成结算依据，但原始区别应继续可见。只有这样，单一计分方式出现偏差时，才有可能回到事实重新讨论。

21.3 贡献应嵌入任务事件而非事后编写故事

可追溯的贡献记录可以从正常工程事件中形成。目标被澄清、协议被修订、一个异常被复现、一个方案被否定、一个版本通过现场验证，都可能构成贡献事件。事件应关联任务、条件、参与者、证据与确认方式，使贡献能够沿着工程过程被检查。

记录不必等于全面监视劳动者，也不需要搜集全部私人交流。记录粒度应服务于协作和评价，避免为了证明工作而制造另一套高成本表演。工程人员应当能够解释关键选择，但不必持续输出冗长说明来维持“看起来在工作”。AI 生成的工作报告同样需要外部证据，不能由系统自行叙述、自行评分、自行确认。

例如，修复一个偶发故障，较有价值的记录包括原故障条件、可重复的验证方法、修改内容和修复后的检查结果。单独保存“花了很久排查”的陈述不能说明贡献；单独保存一小段代码，也可能掩盖问题定位的难度。把条件与结果连接起来，才能让判断既不迷信时间，也不迷信可见产物的大小。

多个主体共同完成任务时，应保留贡献链。业务人员提供关键事实，工程人员建立语义模型，智能体执行实现，现场人员验证结果，各自承担不同作用。贡献链不要求把所有价值精确分摊到每个动作，却能防止最后生成文件的人被误认为全部成果的创造者。

21.4 反事实归因与失败探索的价值

证明业务效果，不能只比较上线前后的两个数字。同期人员、订单类型、季节和其他流程也可能发生变化。需要在可行范围内判断：如果没有这项改进，结果大致会怎样；变化有多少可以由其他因素解释；观察期是否足以暴露维护成本。这种反事实思考能够限制过度归因，即使现实条件不允许严格实验，也应明确估计依据与不确定性。

失败探索的价值同样需要条件。一个预先界定范围的试验，若可靠地排除了昂贵而不适用的方案，可能有明确决策价值。相反，无限重复已经被充分否定的路径，不能仅凭“探索也有意义”获得正当化。失败是否值得，需要同时看授权目标、信息增量、证据质量与投入边界。

复用贡献还会跨越项目。一套接口、测试框架或场景知识可能在首次建设时收益有限，却使后续任务明显简化。后续成果应能够引用这些基础贡献，避免每次从零分配功劳。但也不能无限重复计入同一份价值，将共同基础变成永久收费的理由。如何评价长期贡献，需要公开的规则和能够修改规则的机制。

鸟枪法提供了一个具体情形。后续场景使用同一能力碎片时，应同时记录基础贡献的来路和本轮新增的组合贡献。已经完成的基础测试不能被记作每轮重新发生的投入，最初建设者的贡献也不应因复用而消失。复用收益与当期劳动分别表达，才能让高效工作得到承认。

对于尚未观察到的未来收益，应保留为假设。可复用性、战略选择权和潜在风险降低可以成为投入理由，却不能与已经发生的收益合并后宣称全部实现。这个区别使长期判断有空间，也使价值证明不至于被无限扩展的未来故事吞没。

21.5 计量的目标是改善协作

任何计量一旦进入激励体系，参与者就会对它作出反应。按任务数奖励，可能鼓励拆分小任务；按模型消耗奖励，可能鼓励无效调用；按短期上线率奖励，可能抑制必要的否定结论。设计计量方式时，应同时询问它会鼓励什么行为，以及这些行为是否仍服务于共同目标。

人力 Token 因而不适合从一开始就被设定为跨组织、跨任务完全同质的单位。更实际的路径是保留任务类别、贡献性质和确认条件，在同类条件下形成有限比较，再研究是否存在稳定的换算关系。不同性质的劳动可以并列展示，未必必须强制压缩成总分。

计量还应允许异议。参与者能够查看与自己有关的证据，指出遗漏和归因错误；对重要价值判断，应有与执行者相对独立的确认。共同治理研究提醒我们，规则的运行不能仅依赖规则文本，还与参与、监督及争议处理机制有关。本书将这种启发用于协作计量，但不把它直接等同于任何现成制度。[7]

最终，计量应帮助合作双方判断哪些活动值得继续，哪里出现效率损失，谁的贡献被遗漏，以及下一轮怎样分配资源。如果一个制度只能证明大家都很忙，却无法改善这些判断，它就没有完成自己的目的。自然语言程序设计所追求的，是让劳动、证据与价值能够在同一工程过程中被讨论，并保留对计量规则本身继续修订的能力。

第六编 综合实践与理论边界

最后一编把全书方法放入设备生态和企业信息协作两个案例，再检验它与已有方法的关系及适用边界。世界观需要能够进入具体判断，方法论也需要为自身保留修正的入口。

第 22 章 固件口袋中的协议与创造

22.1 一台设备上的独立创造者

前面讨论了协作、协议、语义与持续演化。要观察它们怎样进入同一个工程过程，可以回到一个足够具体的对象：一台能够运行不同固件的墨水屏设备，以及为这些固件提供共同入口的固件口袋。本章以作者已经讨论的设计原则为基础重建方法链条；其中的状态推演和候选实现用于说明设计方法，不等同于某个已发布版本的完整行为声明，也不据此宣称取得了未经记录的测试结果。

如果只把设备视为一个待开发产品，我们很容易先罗列功能：记事、天气、棋类、音频、凭证展示。随后把这些功能放进一个大型软件，统一安排菜单、存储、网络和退出。这样的方案可能适合某些目标，但作者希望保留另一种可能：不同的人可以为同一设备创造不同软件，各自决定玩法和实现，同时仍能被发现、启动和使用。

问题因此发生了变化。设计对象包含软件之间的关系，也包含创造者之间通过软件建立的间接关系。甲不需要逐行理解乙的固件，乙也不必接受甲全部的架构偏好。他们需要知道的是，共同环境提供什么，自己必须承担什么，以及别人可以合理依赖自己哪些行为。人与人的协作由各自创造物之间的契约延续下来。

固件口袋在这里承担有限的公共职责。它是共同入口和必要能力的承载者，不因此成为所有应用内部行为的决定者。应用的私有数据、业务状态和异常语义，不能仅因应用从口袋启动，就全部转移给启动器。否则，每增加一个应用，启动器都要理解更多私有细节，独立创造很快会退化为中心化修改。

这一判断并不排除公共基础设施。字体、公共信息、已有配网能力、返回机制可以根据实际需要共享。关键是共享能力必须有明确范围，不能把“可复用”理解为“必须由中心统一控制”。我们先建立共同边界，再让具体功能在边界之内成长。

22.2 把共同预期写成生态协议

生态协议可以先以自然语言表达。它首先要说明共同环境与应用的职责：哪些信息公开，哪些数据属于应用私有范围；应用如何表明自身身份和能力；退出时需要兑现什么承

诺；什么情况下可以复用公共能力；出现冲突时由谁处理。这里决定的是关系，而非具体函数的拼写。

作者关于退出的一项明确原则是，必要数据应当持久化，可以不释放全部运行现场。这句话具有较高的抽象层次。它允许不同的退出实现，却对用户能够依赖的结果作出约束。若某种切换机制会整体替换原有运行环境，逐个释放内部对象未必有意义；但数据是否已经达到约定的持久化状态，仍然关系到用户是否会丢失工作。

这项原则不能被进一步偷换成“任何设备上都无需释放资源”。公共外设、并行任务、共享锁或仍会被其他程序使用的资源，可能形成不同义务。具体环境中哪些现场可以随切换消失，哪些外部承诺必须先履行，需要在下一层接口与环境说明中辨明。抽象协议保留自由，同时保留继续具体化的责任。

另一项原则是，启动器不替每个应用仲裁所有标识冲突。共同入口可以规定识别规则和必要的冲突报告，但不能因此承担所有开发者之间的协调工作。应用和创造者需要理解自己对公共命名空间产生的影响。这个选择体现的是职责分配，并非否认冲突本身存在。

协议还应允许区分独立运行与集成运行。一个固件在独立模式下能运行，不能直接证明它遵守口袋的退出与共享能力规则；一个固件能从口袋启动，也不能直接证明它离开口袋后仍可独立工作。若两种模式都是设计目标，就需要分别提出承诺和证据。

由此形成的协议不是一张最大功能清单。它是少量与共同存在直接有关的约束，使更多应用无需相互约定内部实现，也能在同一设备生态中被创造。协议能否简洁，要看它是否保护了必要关系，而不能只看字数。

22.3 架构在共同约束下逐步形成

协议确立后，仍可能存在多个架构候选。例如，一个应用可以独占主要运行环境，退出时再切回共同入口；多个较小功能也可能在某种运行体系下并行存在。独占与并行分别涉及不同的资源、切换、故障隔离和生命周期问题，它们都应接受现实硬件条件的检验。

作者提出让排他型大固件和并行型小固件共存，这种思路适合用来说明协议与架构的关系。上层希望支持独立创造和共同使用，下层则根据应用规模、资源条件与运行要求选择不同实现路径。如果协议不必要地规定所有应用只能采用一种内部结构，就会提前关闭一部分可行空间。

相反，如果协议只写“所有应用都自由运行”，又不足以保护共享资源。并行应用是否可以同时占用显示、音频或存储，失败后是否会影响其他参与者，公共字体和公共信息怎样被访问，都需要可观察的接口承诺。自由是有边界的多样性，不是把资源冲突留到运行时碰运气。

可以采用第 14 章的架构搜索方法，对候选路线分别进行最小实验。某条路线主要验证构建与启动，另一条路线主要验证并发资源访问，再一条路线检查退出后是否仍能恢复公共入口。每次实验应回答一个足以影响选择的问题，而不是先把所有应用写完，再用昂贵返工比较架构。

在这个过程中，架构设计与实现不会被完全隔开。一次最小实现可能暴露协议描述没有考虑到的持久化语义，一次设备实验可能否定原先的切换假设。允许证据向上反馈，才使逐层具象化成为真实工程，而不是把初期文字神圣化。

22.4 接口与建议规范各自承担什么

从生态协议进入接口层，需要把“可返回”“可共享”“可识别”等承诺具体化。接口可以采用函数、文件、消息、启动参数或其他适合环境的形式。对协作方真正重要的是，它如何发起操作、操作在什么条件下有效、会得到什么结果，以及失败时可以依赖什么状态。

以退出为例，接口描述需要说明谁提出退出请求，谁确认必要数据已经持久化，谁实施切换。这里不应先凭直觉规定全部细节。若应用掌握私有数据含义，那么把持久化判断留给应用通常更符合职责；如果公共运行时管理某类共享事务，则需要补充它的职责。本章提出的是分析方法，最终责任划分必须服从具体协议的有效定义。

接口之上的设计指南，可以提供返回入口的位置、文字的一致性、公共字体的使用方式，以及刷新动作的命名。作者把手动全屏刷新称为“研墨”，这是具体交互语义的一部分。指南应说明某项要求属于共同使用所必要的规则，还是为降低学习成本而提出的推荐，不能把两类内容混写。

一个只有应用自身理解的内部对象命名，通常不应成为生态的强制规定；一个用于让用户可靠找到返回途径的约定，可能具有更强的共同性。即使某种布局是推荐做法，实施者仍应理解它所服务的目的。选择不同布局时，可以保留同样的可发现性与退出能力，而不必把参考界面当成唯一正确的实现。

这说明从协议到接口再到规范，约束并不是越来越多越好。每一条新增约束都应解释它解决了什么协作问题。规范也可以把实践中已证明有效的方法交给 AI 复用，但应保留环境条件，避免把一次成功永久固化为唯一技术路径。

22.5 通过退出场景理解授权不完备性

下面构造一个方法示例。应用存在尚未持久化的编辑内容；用户点击返回；保存过程中可能再次点击，也可能遇到介质不可用。人的描述可以说：退出应保护必要数据；连续返回不应导致重复破坏；不要用成功提示掩盖失败；内部状态、异步方式与重试组织可以由 AI 决定；如果需要放弃尚未保存的内容，应交给用户明确选择。

这段描述没有规定每一次按钮事件的完整状态表，却已经给出了必须守住的行为与可以开放的实现空间。AI 首先应辨认状态变量，例如是否有未保存修改、当前是否正在保存、目标介质是否可访问、是否已经请求退出。随后判断哪些组合在当前模型中可达，哪些只是变量机械排列产生的无意义组合。

如果正在保存，再次收到返回请求，AI 可以在既有授权下选择合并重复请求、保留一个退出意图，或者采用其他不会引入额外用户义务的实现。它不需要为每个线程和每个内部标记重复询问人。但如果保存失败后只有“放弃数据退出”与“停留等待处理”两个影响用户利益的选项，而既有规则没有授予自动放弃的权利，就不能把自己的偏好写成默认答案。

反向求证应当提供足够具体的决策材料。可以告诉人：当前保存目标不可用，继续等待不保证恢复；能够保留编辑内容停留当前界面，也可以在明确确认后放弃退出；如果存在经验证的其他存储路径，可以作为另一个候选。不能把“是否合理处理异常”这种空泛问题抛回去，因为它没有缩小决策空间。

人的裁决又应回到模型中。如果选择允许用户主动放弃，模型就要包含确认、取消、再次编辑及确认期间状态变化等关系。AI 还需要检查，“确认”究竟针对哪个版本的内容，后续新增修改是否仍被同一确认覆盖。一个看似简单的按钮，会把语义、授权、状态与时序一起显露出来。

22.6 证据必须经过实际边界

生成一个可编译工程，是重要证据，但它主要说明源文件和构建依赖在某个环境下满足了构建条件。生成可烧录文件，进一步说明产物形成了预期封装。能够启动，说明跨过了设备执行的另一个边界。能否正确保存、返回、恢复以及与公共能力协作，还需要不同的观察。

在隔离环境中完成构建，可以检查工程对隐含本机配置的依赖，有助于提高可复现性。如果没有 USB 实时调试，AI 仍可能完成源代码与构建工作，但真机表现需要通过实际烧录、用户观察、日志或其他反馈通路进入闭环。缺少某条反馈通路，不等于它所证明的事实已经可以从文字推理获得。

针对双模式目标，验证应分别覆盖独立运行与接入口袋。针对退出语义，验证应跨越“提出退出”和“再次进入后数据仍然存在”两个时刻。针对重复点击，测试应观察副作用是否被重复执行；针对故障，应观察提示、数据与返回能力是否符合承诺。测试边界应从协议与场景推导，而不是仅从代码中已有的函数反推。

一个完整的工程记录可以把上层规则、接口决定、实现版本、构建环境和真机观察连接起来。记录不必无限庞大，但应足以在失败发生时定位问题是理解错误、推导遗漏、实

现缺陷，还是环境假设失效。这样的证据链也使后续 AI 能继续工作，而不用把全部来龙去脉重新猜一遍。

22.7 从几个应用到生态的持续生长

当记事、天气、游戏或凭证展示等不同应用共享少量稳定约定时，新的创造不必从重建整个设备系统开始。一个创造者可以围绕自己的场景展开，另一个创造者可以发展公共能力，二者通过协议形成联系。软件实例由此成为生态能力的一种实现，而生态仍允许出现尚未被预先设想的实例。

新的实例也会暴露旧规则的缺口。例如，原本足够的公共信息可能无法表达新应用的能力；原先仅面向单一运行方式的退出约定，可能需要区分新的运行环境。此时应先确认这是单个应用的局部需要，还是多个独立参与者共同依赖的问题。只有后者充分成立，才有理由把它提升为生态协议。

协议演化应保护已经存在的合理依赖。新应用不能因为更方便就单方面改变公共文件含义，共同入口也不能悄悄扩张责任，让旧应用在未知条件下失效。兼容说明、能力探测、迁移路径或明确的不兼容边界，是持续创造能够继续发生的条件。

固件口袋案例最终展示了一种往返关系。我们从独立创造者之间需要怎样协作出发，建立协议，逐层得到架构、接口、指南与软件；又从软件的实际表现回到规则，识别何处应该加强约束，何处应该释放更多自由。下一章把同样的关系放到企业信息协作中，观察它如何改变工具需求本身。

第 23 章 从信息协作中长出企业工具

23.1 一个功能请求背后的关系问题

本章是构造性的完整方法示例，不报告任何企业的真实绩效。假设一家制造企业发现订单处理反复等待，希望“做一个 AI 系统，把订单自动处理掉”。如果立刻把这句话当作软件规格，我们可能很快得到上传页面、识别模型、数据库和审批按钮，却仍然不知道等待发生在哪里。

先观察现有信息流。销售收到来自不同渠道的订单，生产计划人员需要核对规格与交期，仓库需要确认材料，业务负责人处理承诺变化。等待可能来自信息没有送达，也可能来自字段含义不同、资料不完整、资源尚未确定，或者没有人有权做出取舍。不同原因需要不同工具，部分原因甚至不需要新软件。

假设观察发现，重复劳动主要集中在把订单表达转成可供各部门共同讨论的事实，而真正的延误又经常出现在信息有冲突时。此时值得形成的目标是，让有关主体更早看到同一组有来源的订单信息，并使异常抵达能够决定的人。是否要“全自动处理”仍是需要论证的选择，不能被最初的功能名称预先决定。

场景分析首先形成一条价值假设：如果减少重复转录和不必要等待，同时不增加错误承诺，就有机会改善订单协作。这个假设包含成本与风险两面。它不承诺收入一定增加，也不把处理记录数量当作唯一成功标准。后续工具与证据应围绕这项假设组织。

23.2 先定义共同事实与协作协议

在软件架构出现之前，参与者需要对订单事实达成最小共识。原始材料是什么，谁能更正材料，对“已确认”如何理解，价格、交期、数量分别由谁认可。识别结果可以作为待核对信息，但不能因为被模型写入数据库就自动升级为已经生效的业务承诺。

可以把协作约定写成一组自然语言规则。每份订单保留来源与版本；新提取的内容必须能回到对应材料；发现冲突时展示冲突而不自行抹平；未确认的信息不得被当作对外承诺；已经生效的承诺发生变更时，需要通知受影响主体；没有返回结果的操作，不能被默认解释为失败后可任意重做。

这些规则对人和智能体同时有意义。销售人员提交修订，识别智能体提出字段候选，计划智能体分析资源条件，仓库软件返回库存状态，各自都在提供有边界的事实或判断。参与者不必使用同一模型，也不必运行在同一设备，但必须说明信息来自哪里、具有何种效力、更新会怎样影响既有记录。

协议还应区分“提出建议”“确认事实”“形成承诺”三种动作。例如，计划智能体可以根据已知约束提出交期方案，却不能仅因它能够计算就取得向客户承诺的权利。确认动作可以由人执行，也可以在已经批准的明确范围内自动执行。决定因素是授权来源，而非操作者是否看起来足够聪明。

这种设计保留了后续自动化的空间。当某类低影响、可撤回的确认被证明稳定时，可以在适当授权下减少人工步骤。共同协议先为行为定义效力，自动化再逐步取得与证据相称的职责。

23.3 让工具需求从能力缺口中出现

有了关系模型，工具需求就更容易被辨识。原始资料分散，需要统一接收与标识；字段需要带着来源被提取和校验，需要信息转换能力；冲突需要交给适当主体，需要任务分派与决策入口；不同系统需要知道状态改变，需要可靠的事件传递。

这些能力可以由现有系统、人工流程、小型脚本、智能体工具或新软件共同提供。若现有系统已经能保存订单版本，就没有理由为了使用 AI 再建一套同样的数据管理。若一个简单接口可以消除重复转录，就可以先验证这条路径。工具选择应服从协作缺口，而非为了展示技术而制造新的入口。

架构搜索可以提出若干候选。一种方案通过现有平台扩展识别与校验；另一种方案增加独立的信息转换服务；还有一种方案先以人工确认加轻量自动化运行，等事实稳定后再扩大自动执行。比较应同时考虑能否接入、如何纠错、由谁维护，以及错误是否会跨越边界成为真实承诺。

如果某个候选方案需要持续读取企业尚未授权的数据，或者依赖现实中不存在的接口，它就不能通过架构评审。这里的淘汰不取决于界面是否漂亮，也不取决于模型展示是否流畅。环境现实属于方案成立的前提，不能靠在自然语言中描述一个理想环境来补齐。

选择当前可行方案后，工程任务才逐渐具体为数据结构、接口与软件组件。这时每一项实现都能回到一个明确问题：它在信息流中承担什么职责，补上了什么能力，给其他参与者留下什么可依赖结果。

23.4 用有限示例引导场景推演

假设当前任务是识别订单变更并通知内部计划人员。人的自然语言可以描述：正常的新订单应建立待核对记录；重复提交同一材料不应造成重复任务；数量或交期变化应被视为候选变更，不能覆盖已确认值；来源无法辨认时保留材料等待处理。目标是减少遗漏与错误承诺，内部组件、队列和重试方案可以自主选择。

这段说明提出了典型情形，也给出了可以推广的价值原则。AI 应推导尚未列举的情况：同一订单不同附件互相矛盾、旧附件晚于新附件到达、识别完成后材料被更正、重复提交使用不同文件名、系统已执行但响应丢失，以及通知到达后业务状态又发生变化。

这些情况并不要求全部变成独立页面。部分情形可以由版本关系处理，部分情形适合幂等性和操作标识，部分需要人裁决。AI 应提出与当前业务抽象相关的模型，而非机械展开所有变量组合，制造无法理解的巨大状态表。

一个关键问题可能是，“同一订单”如何判断。客户编号相同、日期相同、文件名相同，都未必足以构成身份依据。如果材料中没有可靠的业务标识，AI 可以提出候选匹配规则和歧义队列，却不应擅自把相似订单永久合并。它需要说明无法确定的事实、错误合并的代价，以及可供选择的处理方式。

人的裁决一旦明确，就应被记录成可复用的规则。例如允许系统推荐可能相同的订单，但首次合并由指定角色确认；确认仅对这组记录有效，并不授权今后所有相似记录自动合并。这样，示例帮助推导，授权限定效力，两者共同支持自动化逐步扩大。

23.5 通过一次异常贯通推演与实现

考虑一种具体异常：通知系统没有返回确认，但计划人员可能已经收到任务。若 AI 把“没有确认”直接理解为“没有执行”，反复创建新任务，就可能造成重复处理。若它一律不重试，又可能使任务永久遗漏。这里出现的是信息不完备与行动风险共同构成的工程问题。

候选实现可以为同一次业务动作提供稳定标识，让重复请求能够被识别；同时区分任务已创建、通知已发送、对方已确认等状态。它们不一定需要被全部展示给普通用户，却需要让系统知道哪些结论已经有证据。内部是否采用某种消息组件，仍可由 AI 在环境约束下选择。

测试应覆盖不同失败时刻：创建前中断、创建后响应丢失、通知发送后网络中断、对方确认到达较晚。我们检查的是业务效果是否重复、是否遗漏，以及状态是否能恢复，而不仅是请求最终有没有返回成功。若涉及真实客户的对外承诺，测试还需要验证内部异常不能悄悄变成重复承诺。

如果既有系统无法查询动作结果，也不支持稳定标识，那么安全重试可能没有现成条件。AI 应把这个环境事实返回，而不能声称“通过更智能的判断”解决了不可观测性。可以增加人工核对、修改集成范围，或向系统提供方争取必要能力。架构搜索在此重新打开。

这个例子说明，真正的闭环不沿着文档形式划分边界。一个运行异常可能要求改变接口语义，一个接口限制又可能要求调整协作流程。自然语言维持的是不同层次之间的理由联系，代码和测试则把这种联系落实为可观察行为。

23.6 新工具会改变原先的工作方式

假设工具在限定范围内证明能够可靠地收集材料、标识冲突和形成待办，组织就可能发现新的协作方式。原先需要集中开会才能对齐的信息，现在能够提前被各角色看到；原先由一个人承担的信息中转，可以分散为有来源的事件流；原先很晚才发现的规格问题，可以在进入计划前暴露。

新的能力也会形成新问题。计划人员可能希望预先比较不同交期方案，采购人员可能希望知道哪些需求尚未确认，业务负责人可能希望识别反复发生的冲突类型。此时出现的需求并不只是“给原系统再加几个按钮”，而可能涉及决策更早发生、职责重新划分、共享数据范围变化。

发现新需求的也可以是参与协作的智能体。它在多个流程中观察到重复转换，可以提出公共能力候选，依据已有授权开展小范围试验，并保留原场景的差异。若候选涉及共同协议变化，则进入相应协商。工具需求由协作过程本身暴露，再经判断进入创造，这使软件生长不必等待人事先写完下一张功能清单。

不能因为工具能够提供更细的信息，就自动扩大所有人的可见范围。也不能因为自动化节省了一道人工中转，就认为那个人原先承担的判断责任已经消失。技术改变协作条件以后，需要重新检查新的权利、义务和负担是否被识别。生产力的变化会推动关系调整，但不保证调整自然公平或合理。

有时最佳结果是减少软件。若共同协议使三个部门不再维护三份相同表格，中间转换工具可以被移除；若某道审批只为补偿原先信息不可见的问题，新的可见性可能使它不再必要。工程师应有能力提出删除方案，并把删除后仍需保留的承诺验证清楚。

23.7 如何判断这轮演化值得保留

评价需要同时观察投入、运行质量与业务结果。投入记录说明为了形成和维护能力消耗了什么；运行证据说明正确性、等待、异常与恢复发生了怎样的变化；业务结果则需要由真实使用者确认其意义。三者不能互相替代。

如果处理速度提高，但错误承诺增加，不能仅凭速度宣布成功；如果人工转录减少，却把更多时间转移为低质量识别结果的核查，也应计入完整成本；如果团队建立了更可靠的订单事实来源，价值可能表现为减少以后系统集成的困难，而不完全表现为当期处理量。

因果归因也需要节制。订单季节变化、人员熟练程度、客户结构和管理调整都可能影响结果。可以比较限定场景的前后记录，采用分阶段部署，保留异常样本，并说明哪些改善有直接证据、哪些仍是推测。对复杂业务，不必为了得到一个漂亮总数而隐藏不确定性。

当结果不符合预期，下一步可能是改实现，也可能是改语义、改协议，甚至放弃原来的工具方向。只有允许这些选择，反馈才真正进入设计。若所有失败都被解释成“模型还不够好”，体系就失去了反思自身需求定义的能力。

这条案例链从一个模糊的软件请求出发，回到信息协作，建立共同规则，识别工具缺口，再通过语义、推演、实现和证据进入现实。工具投入使用后又改变协作，使下一轮需求开始形成。至此，我们已经看到本书主线如何贯通。接下来需要明确，它与既有方法共享什么，又必须在哪些边界上保持谦逊和可检验性。

第 24 章 与既有方法的关系及理论边界

24.1 一套方法需要说明自己继承了什么

经过前面的理论展开与综合案例，本书已经呈现出一条从信息协作到软件演化的主线。要使这条主线具有可讨论性，还需要把它放回已有工程知识中。自然语言需求、协议优先、契约式设计、持续交付和模型验证各有历史，不能因为它们在 AI 条件下重新组合，就把既有成果抹去。

本书的主张在于组织方式：把协作关系作为上层设计对象，把协议作为独立创造的共同边界，把自然语言整体语义作为需求与授权载体，把启发式示例用于开放推演，再以现实证据推动工具和协作共同演化。其价值需要通过这种组织是否解决实际问题来判断，而不依赖宣称某个组成概念第一次出现。

工程思想的独立性不要求与所有前人切断联系。一个方法可以借用已有测试、形式化表达和软件结构，同时对它们的角色关系提出新的安排。相应地，不能把一个成功案例中所有已有技术的效果都归功于新框架。研究与实践都应尽可能区分：改进来自更好的需求表达、明确授权、协议复用，还是来自所用模型和工具本身。

这种区分也保护本书的中心。它使方法能够吸收新技术而不必把每项技术都改名，也使不成立的局部机制可以被替换，而不必为了维护整体声称所有部分永远正确。

24.2 SDD 与场景驱动可以处于不同层次

规格驱动开发强调通过明确、可维护的规格组织实现。它并不天然等于固定价格合同，也不天然要求需求在项目开始后停止变化。以规格为依据进行迭代，与让规格随场景更新，可以同时成立。具体实践的严格程度、生命周期和自动化方式存在差异，应当讨论具体方法，不能仅凭名称推断全部行为。[13]

本书关注的问题更靠近规格的来源及其开放部分：现实协作为什么需要某种工具，哪些目标尚需发现，哪些语义应由人决定，哪些允许 AI 继续推导。场景驱动决定问题从哪里来，以及实现之后如何再次检验问题；规格可以记录某个时刻已经确认的共同理解。两者并不构成逻辑上的互斥。

授权的不完备性也不是反对规格。它要求规格明确表达未确定部分的处理方式。一个行为尚未被逐项列举，可以处于有目标、有约束、有决策主体的开放空间；这与简单遗漏要求不同。如果现有规格方法能够准确表达这种空间，它就能够成为本体系的组成部分。

真正需要警惕的是把当前规格当作现实本身。实现完全符合文本，仍可能没有改善实际协作；现实暴露新矛盾时，也可能需要修改目标与协议，而不只是修复代码。反过来，

不能以场景持续变化为由取消当前承诺。语义与现实之间需要双向检验，任何一方都不应被免除检查。

24.3 契约设计与模型方法提供局部精确性

契约式设计通过前置条件、后置条件和不变量表达组件责任，与本书关于入口、出口和必须保持条件的讨论具有直接联系。它提供的是可以精确化的局部关系。本书则进一步把权责主体、启发式示例、价值取舍和开放决策纳入更广的场景语义。两者可以连接，但不能把两者直接视为同一层次的语言。[10]

协议优先与接口优先也有广泛先例。软件模块化研究强调根据可能变化的信息组织边界，而不是仅按执行步骤拆分程序。这为本书关于稳定承诺与内部自由的区分提供了重要参照。[3] 本书将这种分离继续向上推进到创造物之间的协作关系，但具体接口仍应借助成熟的类型、协议描述和一致性验证工具。

本书的鸟枪法可以借助模块化、组件复用和既有测试方法实现，但它关注的组织单位来自多个现实场景中的共同能力。其工作路径是跨场景拆解与归并、分别完善过程和响应，再以语义胶水还原具体场景。它不以发明全新的组件理论为前提，而要检验这种组织是否减少重复探索，同时保留场景差异、授权边界与组合后的验证责任。

状态机、模型检查和基于性质的测试能够帮助推导行为并寻找反例。自然语言为这些方法提供问题、抽象边界和判断准则，形式化模型则把其中一些承诺转化为可检查对象。一个模型通过验证，只能说明在模型假设和验证范围内没有发现指定性质的违例；它不能证明模型完整覆盖了开放现实。[11]

因此，本书保留多种表达形式。自然语言连接不同抽象层，结构化数据降低接口歧义，代码执行确定过程，模型刻画关键状态，真实测试检验与环境的接触。要求所有环节只能使用自然语言，并不会提高方法的统一性，反而会放弃已有表达工具能够提供的精确性。

24.4 方法论的成立与语言实现的成立不同

本书使用“程序设计”来指称有目的地组织计算与协作行为，不把它限定为逐行输入代码。自然语言可以承担目标、约束、授权和验证条件的表达，AI 能够据此产生实现。但这不意味着本书已经提供了一门具有完整形式语义、确定编译规则和独立实现标准的语言。

如果希望进一步形成语言系统，还需要规定语义对象如何表示，歧义如何被发现，规则冲突如何处理，授权怎样被执行环境检查，描述改变后哪些实现必须重新验证。这些机制可以逐步构建，不必等全部完成才开展实践；但方法层面的有效经验不能被当作语言层面的完备证明。

模型之间存在差异，同一模型在不同上下文中也可能选择不同实现。方法的可靠性不应要求全部内部代码一致，而应检验共同承诺、决策边界和外部行为是否满足要求。对允许变化的部分，应承认多解；对必须一致的部分，则需要更明确的表达和独立验证。把随机性全部归为创造力，或者要求所有选择绝对一致，都会损害合理的自由度。

本书也不把自然语言的可理解性等同于自动可执行性。某些表达只有在共享经验充分时才明确；某些价值排序存在真实分歧；某些环境事实尚未获得。系统应能识别这些条件，并决定补充调查、提出候选解释或暂停特定行动。自然语言程序设计的能力，包括知道什么尚不能被可靠地执行。

24.5 让核心主张接受反例

本书的规范主张与经验主张需要分开。决策权限应被明确、受影响者的边界应受到尊重，属于本书选择的设计原则；采用某种语义框架能够减少返工、提高兼容性，则属于需要证据的效果主张。前者需要论证理由，后者需要观察和比较，不能用同一种自我认同替代验证。

如果启发式语义描述在相同任务中反而增加误判，就应查明示例偏差、授权歧义或推演成本是否抵消了收益。如果协议先行使一个简单工具承担大量没有现实依赖的约束，就应减少协议层次。如果某个 AI 已经能够稳定完成过去需要人工组织的步骤，就应调整人的参与位置，而不是让模型配合保留旧流程。

反例也需要合理解释。一项任务失败可能来自工具故障、环境事实缺失或方法本身，不能未经分析就否定整个体系；同样，不能把所有失败都推给使用者没有真正理解方法。应当尽可能把主张拆成可观察机制，使不同解释能够被证据区分。

本书因此是一套开放的方法论。它提出可以继续具体化的概念与操作关系，并保留修订这些关系的条件。它的持续价值取决于能否在变化中维持有效判断，而不取决于最初写下的每一项表达是否被永久保留。

第 25 章 面向持续演化的程序设计

25.1 回到信息与协作的起点

全书从一个超过单个软件的问题出发：信息怎样在不同主体之间形成理解、决策和行动，并在现实中产生价值。这个问题没有被某一代模型的出现一次解决，也不会随着代码生成变得容易而消失。生成能力改变的是实现某些想法的条件，同时也改变了我们能够提出什么想法。

协作关系决定信息为什么需要流动，协议使独立主体能够形成共同预期，接口让承诺进入可操作的交往，建议规范传递经验，软件与设备把这些关系落实为能力。自然语言贯穿其中，承载目标、理由、边界和开放问题；其他技术表达则在需要精确性、重复性和效率的地方参与实现。

这条路径并不意味着先在书桌上完成全部生态设计，现实才获准开始。上层协议可以从局部实践中提出，软件也可能先以探索工具出现。关键在于能够识别不同层次，并在发现共同依赖以后把隐含关系转化为适当承诺。自顶向下的约束与自底向上的发现共同构成具象化过程。

程序设计的对象由此扩展为能够创造和改变工具的协作系统。软件是其中重要的实现，却不是唯一成果。一项规则澄清、一次权限重组、一个多余环节的取消，也可能比新建系统更接近目标。是否需要软件，应由能力缺口与现实条件共同决定。

25.2 演化包含生长也包含退出

工具进入活动以后，会改变成本、速度、可见性和分工。新的分工使一些需求出现，也使一些需求消失。原先需要开发专门报表的活动，可能被更好的信息接口替代；原先分散的工具，可能因为共同协议而合并；原先由人承担的中转工作，可能被智能体接续，人的注意力随之转向新的问题。

这个过程可以用生产力与生产关系相互作用来理解，但不能由技术能力直接推导唯一的组织安排。同一种模型可以用于集中控制，也可以支持分散协作；同一种接口可以降低参与门槛，也可能形成新的依赖。技术提供可能性，目标、权力安排与现实选择共同决定可能性怎样被实现。

因此，演化没有自动的道德方向，也不保证复杂程度不断增加。更快生成软件可能同时制造更多维护负担，更强自主性可能放大错误授权。价值准则必须继续参与判断：谁获得能力，谁承担成本，哪些边界被改变，系统是否仍具备修正自身的条件。

退出是这一判断的组成部分。一个工具不再满足真实需要，应当能够停用；一个协议阻碍必要协作，应当允许在处理依赖后修改；一个智能体不再适合任务，应当能够被替换。生态的活力来自有条件地进入、发展、变化与退出，而不是只允许新增实例。

25.3 人的角色随着能力边界移动

AI 能力增强以后，人可能不再需要亲自枚举某些状态、拆分某些任务或比较常见实现。坚持把这些操作留给人，并不能证明人仍然重要。人的参与应当随真实能力与责任条件移动，集中在需要了解现实、表达价值、分配权限和判断证据的位置。

这些位置也不是永久拒绝自动化的保留地。AI 可以协助调查、发现冲突、提出价值权衡和评估结果，未来还可能承担更多具体判断。需要持续保留的，是决定如何委托、如何确认委托有效、如何处理受影响者关系的机制。某项工作由谁完成，应依据能力、权限和验证条件，而不依据预先划定的身份荣誉。

人与 AI 的自由也具有不同含义。人有自己的目的与社会关系，智能体的执行自由来自特定活动中的授权。让 AI 在边界内探索，可以扩大人的创造能力；让授权失去来源，则可能使工具反过来主导目标。我们应当区分帮助人发现目标与未经许可替人确定目标，区分提出新可能与自行取得行动权。

这也是自然语言语义规格需要保留价值判断和权责关系的原因。随着实现细节越来越容易被自动补全，剩余的不确定性可能更加集中于目的和后果。它们不一定更容易量化，却更需要被准确表达和共同讨论。

25.4 让治理强度与现实依赖相称

本书展开了协议、状态、证据、授权和价值计量，并不要求每个小任务都完整运行一套庞大程序。一个本地文本整理工具与一个跨企业控制设备的系统，承担的依赖和后果不同，所需要的治理深度也应不同。

方法的最小形态可以非常简洁：说明目标，指出关键约束，给出必要例子，明确哪些可以自行决定，再检查实际结果。当出现多个独立参与者、长期依赖、不可逆后果或复杂价值冲突时，才逐步增加协议、状态模型、决策记录和验证层次。复杂性应当由问题推动，而不是由方法对自身完整性的执着制造。

比例原则同样适用于证据。可逆的小变化可以使用轻量检查，影响共同接口的变化需要兼容验证，涉及真实设备后果的变化需要更接近现场的确认。没有必要用相同形式对待不同风险，也不应因为某种操作由 AI 执行，就自动放弃原本必要的判断。

一个好的生态应允许局部创新，而不要求每个创造者重新设计整个世界。上层协议应尽可能少而必要，使不同实现能够保留个性；局部经验可以不断生长，但只有出现共同依赖时才需要上升为更高层规则。这种克制是开放系统保持创造空间的重要条件。

25.5 持续有效的方法必须能够修订自身

如果一套方法要求所有未来技术都遵循今天的步骤，它迟早会把成功经验变成障碍。本书所追求的持续性，是信息、目标、协作、承诺、授权和证据之间的关系能够继续被清楚判断；具体模型、工具链、文件形式和操作分工则允许不断变化。

这种持续性也要求审查方法自身。当自然语言描述造成反复歧义，应改进语义表达；当协议形成不必要负担，应重新界定共同承诺；当价值计量鼓励错误行为，应修改计量规则；当现实证明原目标不成立，应允许改变目标。任何一层都不能因为位于上层，就免除面对证据的责任。

“存在是目标，演化是策略，定向演化没有终点”可以表达本书的实践立场。这里的存在不是让某个软件永久运行，而是使有价值的协作能力能够在变化中继续成立；定向不是预先决定全部未来，而是在明确目标和价值判断的条件下，持续选择下一步，并承担选择带来的结果。

自然语言程序设计由此形成一套可以继续展开的世界观与方法论：从现实中的信息关系理解协作，以协议保护独立创造，以语义规格组织开放推理，以授权明确自由的范围，以 AI 扩展实现能力，以现实验证检查承诺，再把结果带回协作关系本身。每一次闭环都应完成有限的事情，也为后续创造留下可以理解、接续和修订的条件。

一本书能够把这些关系写清楚，却不能替代它们进入现实以后所面对的检验。方法真正延续的地方，是下一次问题出现时，我们仍能够判断什么已经明确，什么尚需探索，谁有权决定，以及需要怎样的证据才足以继续行动。

附录一 核心概念及其边界

一 自然语言程序设计

以自然语言组织工程意图、协作协议、场景语义、价值判断和授权关系，使执行主体能够在这些条件下推演、实现与验证，并接受现实反馈的方法。它覆盖软件产生与演化的过程，不要求计算设备直接执行自然语言，也不要求放弃代码和形式化工具。

二 信息协作

信息由某个主体产生，经由传递、理解和判断影响行动，再通过结果反馈进入新的协作过程。这里同时讨论信息的含义、来源、时效和使用权限。通信量、信息熵和价值不能相互替代。

三 协作主体

在具体关系中接收信息、承担能力并产生影响的角色，可以由人、AI、设备或其组合承担。主体是分析单位，不自动对应一个账号、一个进程或一个法律责任主体。

四 协议生态

由共同协议及其参与者、实现、资源和治理方式构成的协作环境。协议约束共同边界，生态允许独立创造。这里的“底层”表示共同依赖的逻辑基础，并不一定表示硬件层或操作系统层。

五 逐层具象化

从协作意图、协议、责任架构到接口、设计指南和软件实例，逐步补充可执行决定的过程。各层可以往返修订，但新增决定必须说明来源，不能在向下实现时悄悄改变上层承诺。

六 接口与规范

接口定义一个能力如何被触发、观察和依赖；规范规定或者建议实现者如何工作。二者可以写在同一文档中，但必须区分强制义务、推荐方法和参考实现。名称叫“指南”不意味着其中没有强制规则，名称叫“规范”也不意味着每个示例都不可改变。

七 场景驱动

从真实活动中的主体、处境、信息、目标与阻碍形成问题，并用实际使用结果继续修正问题。场景可以产生产品需求，也可能证明不应新增产品。它要求当前承诺明确，同时允许下一轮需求继续发现。

八 语义规格

对整体场景描述需要表达什么、各类内容有何效力以及如何判断结果的要求。它不要求每句话使用固定句型。足够清楚的语义可以分布在对话、协议、环境证据和例子中，但关键关系应能被识别和追溯。

九 价值准则

当存在多种可行行为时，用于判断哪些结果更值得保留的原则、排序和约束。价值准则可以包含指标，却不能被一个指标完全代替；价值也需要说明对谁、在什么时间范围、承担什么代价。

十 授权的不完备性

明确保留一部分尚未逐项决定的空间，并说明由谁在什么目标与边界内补充。它可以针对一类决策整体授予，无需列举每个实现细节。关键遗漏、事实未知和权限缺失不能自动转化为授权。

十一 启发式示例

通过有限的正常例、边界例、反例和取舍例，传递可推广的判断关系。示例用于启发推演，不保证覆盖现实全部情况；对示例的推广也不能自行扩大原有权限。

十二 反向求证

执行者在既有准则无法裁决、事实不足以支持决定或行动超出授权时，向有权决定的主体提供具体问题、依据、备选和后果。它不意味着把所有工程细节交还给人，也不意味着每次运行异常都重新启动一次需求访谈。

十三 状态模型

在特定抽象边界内，描述有关状态、事件、条件、动作与转换的模型。它帮助推导与验证，但现实是否落在该模型范围内仍需证据。模型检查结论应与其假设、所检查性质和执行范围一起说明。

十四 鸟枪法

作者借鉴基因组鸟枪法的拆解与重组思想，在多个场景中抽取能力碎片，归并具有共同语义的能力，分别完善和验证过程响应，再借助 AI 的语义能力将碎片组合回具体场景的方法。它减少重复探索，但仍须检验组合关系与场景整体性质。

十五 能力碎片与语义胶水

能力碎片是具有可描述前提、过程和结果的功能单位。语义胶水是把这些单位放回具体场景时所需的目标解释、数据对应、状态连接、时序、权限与异常处理关系。胶水可以落实为说明、协调逻辑、适配器或代码，不只是传递字段。

十六 EARS 架构探索框架

本书沿用作者讨论中的记号，以 Environment、Architecture、Reward、Search 表示环境、架构、评价和搜索。Reward 在此指多目标评价及其取舍，不能消除硬约束。它与需求工程文献中的 Easy Approach to Requirements Syntax 同名而不同义。两者出现时应依上下文区分，不借同名宣称理论继承关系。

十七 工程闭环与协作闭环

工程闭环检查实现是否兑现当前语义承诺。协作闭环检查这些承诺和工具是否仍然服务现实需要。工程合格是必要条件之一，不能单独证明工具继续存在的价值。

十八 人力 Token

作者用于讨论劳动贡献及其证据如何嵌入持续工程过程的概念。它不直接等于模型文本 Token、消耗时长、价格或价值，也不预设为数字资产。具体计量与结算规则需要在明确场景内形成，并保留投入、贡献、价值和价格的区别。

附录二 一份可以实际使用的场景描述

一 用自然语言完整表达一个小型任务

以下是一份方法示例，用于展示语义要素如何自然地出现在连贯文字中。它不是必须逐栏填写的模板，也不是任何具体设备的现行规范。

我需要在一台个人墨水屏设备上保存简短笔记。使用者通常临时进入应用，写入或修改文字后离开，之后还希望能够继续查看。当前目标是让编辑、保存与退出容易理解，并减少因操作时机不同而丢失工作的情况。先完成这一有限场景，不由此增加多人账号、云同步或社交功能。

设备已经有一个公共入口。应用从该入口进入时，应遵守已确认的返回协议；独立运行时采用适合当前环境的退出方式。必须先核实实际可用的存储位置和访问方法，不能假定存在尚未确认的目录、接口或权限。公共设置与应用私有数据应按有效协议区分。

用户明确看到“保存成功”后，内容应已经达到本场景约定的持久化条件。这个条件至少包含应用重新进入后可以读取；是否承诺异常断电等故障下的特定恢复能力，需要根据环境事实另行确定。未成功保存时，界面应如实表达状态，不能用返回主界面代替保存确认。

典型情形包括正常保存后返回、保存过程中再次点击返回、没有修改直接返回，以及存储不可用时请求退出。连续点击不得导致重复破坏或让原来的编辑内容被错误覆盖。若已确认的规则足以确定行为，请自行组织状态、事件与内部实现；若需在丢弃未保存修改与继续停留之间作出新的用户利益取舍，应设计明确的选择方式并在开发阶段说明其语义。

在不改变上述行为和共同接口的范围内，可以自主选择文件组织、缓存、异步机制、代码结构、合理重试和构建方式。可以为理解环境进行可恢复的小范围实验。若方案需要扩大数据访问范围、删除原有资料、修改公共协议或取得新的外部承诺，应先说明影响和备选方案。已有授权不必重复确认。

请由这些例子推导相关状态与异常路径，记录足以影响决策的假设，并说明哪些行为来自明确要求、哪些来自协议、哪些是授权内选择。实现后验证保存、恢复、重复请求、异常提示与退出衔接。测试应使用明确构造的样本，并把构建成功、实际运行和真实设备验证分别报告。交付时保留可继续修改的工程材料与关键证据。

二 执行者如何整理而不篡改

AI 可以把上述描述整理为目标、约束、状态候选、权限和验证计划。但整理时不应把自己选择的某种文件格式改写成“用户要求”，也不应把尚未核实的存储位置写成既定环境。

若默认实现是可逆且在授权内，可以直接采用并说明；若改变了用户所依赖的行为，则需要回到相应的决定依据。

一个合理的整理结果，可以把“没有修改直接返回”识别为典型例，把“确认成功必须对应约定持久化状态”识别为强制性质，把“内部缓存方式”识别为自主选择，把“异常断电的恢复承诺”识别为尚待环境证据支持的事项。语义分类比把原文改成整齐表格更重要。

这个例子也展示了停止条件。当前有限场景已经有足够依据时，AI 应进入实现与验证；不应为了让需求说明看起来无所遗漏，继续询问与当前任务无关的颜色、框架、商业化方案或者未来多人协作。语义规格应减少无效往返，而不是把访谈无限延长。

附录三 能力碎片与组合记录

一 描述一个碎片

下面以一个方法示例说明能力记录。名称是“带来源的文本读取”。它接收已获授权的资源位置，输出读取到的内容及可用于回溯的来源标识。它只承诺读取，不修改原始材料。输入位置必须属于授权范围，文件是否仍在变化是需要说明的环境条件。

正常完成表示内容已经被读入本轮处理范围，不表示来源已经永久存档，也不表示内容具有业务真实性。读取失败应区分资源不存在、无访问权限、格式无法处理以及执行中断；若当前接口无法区分，应如实返回较弱结论。重复调用可以返回新的读取结果，但不应隐含地覆盖此前已确认的业务记录。

这份记录应附上已经执行的测试及其适用条件。例如，在哪种文件系统和字符编码下验证过，文件为空时返回什么，读取失败后是否仍保留原文件。暂未验证的大文件或并发修改情形应保持可见。这里所说的附证据，不意味着把全部日志塞进每次调用，而是能够找到必要材料。

二 描述一个组合

假设一个场景需要“读取材料并生成待确认记录”。它组合文本读取、候选信息提取、结构校验和记录写入四个能力。组合说明应解释，提取结果如何关联原文，校验失败时是否允许写入，写入时如何标记待确认，以及哪个角色可以把它升级为正式记录。

其中一个连接义务是，提取失败不能被转换为空字段后当作有效结果。另一个连接义务是，用户更换材料后，旧的提取任务不能继续把结果写入新材料对应的记录。再一个连接义务是，重复执行同一确认动作不能造成重复业务效果。这些义务来自组合关系，单个碎片测试未必覆盖。

组合还应说明自己的依赖。如果使用已经验证的文本读取版本，只改变列表展示，那么读取性质的部分证据可以沿用；如果改变材料身份规则，受到影响的可能是提取、写入和重复处理的整个关系，需要相应扩大检查。复用依据应来自前提仍成立，而非版本名称看起来相近。

三 保留从场景到能力的来路

能力集合应保留双向映射。一侧能够从场景查到所用能力与连接；另一侧能够从能力查到受影响场景及其特殊条件。映射可以是简短文档、结构化文件或者工具内部记录，形式不是核心。

例如，同一个读取能力被记事、配置查看和凭证展示共同使用。若字符编码规则变化，三个场景都需要评估；若只改变凭证核销后的界面提示，配置查看无需因此重新验证所有读取细节。维护这种关系，才使鸟枪法把重复劳动转化为可复用证据。

任何抽象都可能失效。若后来发现“读取配置”必须与硬件状态切换保持一致，而其他读取不需要，它可以从通用能力中分离为专门变体。允许重新划分，能够让能力集合随场景成长，避免为保持一个过度统一的接口而扭曲实际需求。

附录四 决策与证据的简洁记录

一 一次授权内的决定

问题是：同一退出意图在保存期间可能被重复触发。依据是：不得重复产生破坏性效果，内部状态组织已整体授权。决定是：在本轮退出未结束时合并等价请求。影响是：不改变用户的退出目的，不扩大数据访问范围。证据是：重复请求、完成回调和失败后重试的相关检查。复核条件是：未来引入不同来源、具有不同业务含义的退出请求。

这条记录不需要人再确认已经授予的内部设计权，也没有把选择伪装为唯一可能实现。它保留了当前方案为何成立，以及什么时候应重新检查。

二 一次需要新增裁决的决定

问题是：存储空间不足，当前编辑不能达到约定持久化条件。已经查明的事实是目标空间不可用，且没有已验证的替代存储路径。未决事项是能否删除旧记录、能否放弃当前修改，以及由谁决定。AI 可以继续保护现场、提供有依据的候选并完成无争议部分，但不能把其中某个选择当作已经获准。

人作出决定后，需要记录授权范围。例如，“允许使用者在明确确认后放弃当前尚未保存的修改”，不同于“允许程序遇到空间不足自动放弃所有修改”。后续实现应保留这一区别，测试也应检查确认与取消分别产生什么行为。

三 一条验证结论

合格的验证结论应连接被检查的承诺、实际执行的条件、观察结果与未覆盖范围。例如，可以记录在指定设备和测试数据下，执行保存、退出、重新进入后读取到相同内容；同时记录尚未覆盖异常掉电。这样的陈述可以支持有限结论，又不把局部证据扩展为任何条件下的保证。

“已全面测试”“AI 认为没有问题”“代码逻辑很清楚”都不能独立说明证据强度。相反，验证发现失败也有用，只要它能够指明哪个承诺在什么条件下未成立，以及下一步需要调查什么。记录的目的在于帮助判断，不在于使报告显得成功。

四 一项贡献记录

一项贡献可以表述为：在已授权的探索范围内，发现某个集成路径缺少判断重复执行的条件；通过受控实验确认，继续重试会产生不可区分的重复效果；据此排除该路径，并保留测试方法与替代建议。投入记录说明使用了哪些资源，成果记录说明排除了什么，价值记录说明这个结果如何影响下一步选择。

若后来某个项目复用这一结果，应记录引用基础与新的适用性判断，而不是把原实验当作重新发生的一轮投入。基础贡献的来路可以保留，新增贡献也可以独立说明。人力 Token 如要成为有效制度，应先能够准确表达这类关系，再讨论如何计分或结算。

附录五 全书主张的实践检验

一 检验协议是否支持独立创造

让不同实现者在只共享有效协议与必要环境信息的情况下完成相互协作的实例。观察他们是否依赖未公开习惯，哪些边界产生歧义，以及修改一个实现时是否迫使其他实现理解其内部细节。失败可能指出协议缺失，也可能指出实现违反已有承诺，需要分别判断。

二 检验语义规格是否降低误解

把同类场景分别用普通功能描述和本书的场景语义表达组织起来，比较重要假设能否被识别、授权是否被正确使用、需要人决定的问题是否更集中，以及实际结果是否更符合

目标。评价不能只看文本是否更长、表格是否更多，也不能把模型升级带来的改善全归给表达方法。

三 检验启发式推演是否保留自由

观察 AI 是否能从有限例子提出未被列举而又符合原则的行为，是否能在已有授权内主动推进，以及遇到真正冲突时是否提供可回答的选择。若每个分支都必须由人批准，方法没有释放自主空间；若 AI 用自己的偏好代替未获授权的价值取舍，方法也没有守住边界。

四 检验鸟枪法是否减少重复劳动

在一个明确场景集合中，记录共用能力、独立过程测试、场景连接与变化后的检查范围。比较是否减少重复实现和重复排查，同时观察抽象与维护成本、错误归并、连接缺陷和回归遗漏。不能只统计组件复用次数，也不能通过省略必要的组合验证来制造效率收益。

五 检验工具是否改善真实协作

回到信息从产生到行动的完整链条，观察等待、错误、责任理解与使用负担如何变化。能够运行的工具可能仍然没有解决真正问题，也可能只是把负担转给了另一个主体。只有把这些变化重新纳入目标和价值判断，才能决定继续扩展、局部修改、维持现状还是退出。

六 检验方法能否承受技术变化

在模型、执行环境或可用工具改变以后，保留共同目标与约束，允许实现路线重新搜索。观察哪些人工步骤已经可以取消，哪些旧经验失去前提，以及相同承诺能否通过新的路径获得证据。若方法只能依附某一种产品操作流程，它的抽象还不够稳定；若它拒绝接受现实反例，它也无法真正支持持续演化。

这些检验是本书为自身保留的修订入口。它们不要求每个项目都承担研究实验的负担，但要求在声称方法有效时，能够说明有效表现在哪里、依据什么、适用于什么条件。自然语言程序设计如果要长期成立，应当允许自身与它创造的软件一起接受现实的检查。

参考文献与思想来源

下列文献用于标明本书借鉴的思想与方法。正文提出的整合框架、授权机制、鸟枪法的软件应用与方法示例，属于作者的组织和推演，不因援引这些文献而自动成为已证实结论。网页资料统一访问于 2026 年 11 月 3 日。

[1] Simon, Herbert A. The Sciences of the Artificial. 3rd ed. MIT Press, 1996. https://mitpress.mit.edu/9780262264495/the-sciences-of-the-artificial/

[2] Shannon, Claude E. A Mathematical Theory of Communication. The Bell System Technical Journal, 1948, 27: 379–423, 623–656. https://web.mit.edu/6.976/www/handout/shannon.pdf

[3] Parnas, David L. On the Criteria to Be Used in Decomposing Systems into Modules. Communications of the ACM, 1972, 15(12): 1053–1058. DOI 10.1145/361598.361623. https://doi.org/10.1145/361598.361623

[4] Ashby, W. Ross. An Introduction to Cybernetics. Chapman & Hall, 1956. https://www.ashby.info/Ashby-Introduction-to-Cybernetics.pdf

[5] Bradner, Scott. Key words for use in RFCs to Indicate Requirement Levels. RFC 2119, 1997. https://www.rfc-editor.org/rfc/rfc2119

[6] Leiba, Barry. Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words. RFC 8174, 2017. https://www.rfc-editor.org/rfc/rfc8174

[7] Ostrom, Elinor. Beyond Markets and States: Polycentric Governance of Complex Economic Systems. American Economic Review, 2010, 100(3): 641–672. DOI 10.1257/aer.100.3.641. https://www.aeaweb.org/articles?id=10.1257/aer.100.3.641

[8] Mavin, Alistair; Wilkinson, Philip; Harwood, Adrian; Novak, Mark. Easy Approach to Requirements Syntax (EARS). 17th IEEE International Requirements Engineering Conference, 2009: 317–322. DOI 10.1109/RE.2009.9. https://doi.org/10.1109/RE.2009.9

[9] Hoare, C. A. R. An Axiomatic Basis for Computer Programming. Communications of the ACM, 1969, 12(10): 576–580. DOI 10.1145/363235.363259. https://doi.org/10.1145/363235.363259

[10] Meyer, Bertrand. Applying Design by Contract. Computer, 1992, 25(10): 40–51. DOI 10.1109/2.161279.
https://se.inf.ethz.ch/~meyer/publications/computer/contract.pdf

[11] Lamport, Leslie. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley, 2002.
https://lamport.azurewebsites.net/tla/book.html

[12] National Human Genome Research Institute. Shotgun Sequencing. Talking Glossary of Genomic and Genetic Terms.
https://www.genome.gov/genetics-glossary/Shotgun-Sequencing

[13] GitHub. Spec Kit — Specification-Driven Development. 项目官方方法说明，访问于 2026 年 11 月 3 日。
https://github.com/github/spec-kit/blob/main/spec-driven.md

[14] Claessen, Koen; Hughes, John. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. ICFP 2000: 268–279. DOI 10.1145/351240.351266.
https://doi.org/10.1145/351240.351266

[15] Beck, Kent, et al. Manifesto for Agile Software Development and Principles behind the Agile Manifesto. 2001. 采用官方网站中文原则页面。
https://agilemanifesto.org/iso/zhchs/principles.html